



République Tunisienne
Ministère de l'Enseignement Supérieur et de la
Recherche Scientifique
Université de Tunis El Manar
Faculté des Sciences de Tunis



MEMOIRE DE MASTER

Présenté en vue de l'obtention du
Diplôme National de Mastère Professionnel
En Système de Télécommunications et Réseaux

Réalisé par :

Yasmine Mouakhar

**Conception, développement et déploiement d'une
plateforme SaaS multi-tenant pour la gestion de
marketplaces bancaires avec mise en place d'un pipeline
CI/CD**

Soutenu le 26/06/2024

Devant le Jury :

Mr. Med Anouar Ben Messaoud	Maître de Conférences	Président
Mme. Rachida Bedira	Maître assistante	Examinatrice
Mr. Dhafer Mezghani	Maître de Conférences	Encadrant

Réalisé à :

BRI-Technology



Année Universitaire 2025 - 2026

Encadrant Académique

J'autorise l'étudiante à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant Académique : **Tarek CHAABANI**

Signature

Encadrant Professionnel

J'autorise l'étudiante à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant Professionnel : **Naja AGUILI**

Signature et cachet

Remerciements

Le travail exposé dans ce mémoire a été entrepris dans le cadre de la préparation du diplôme de Licence en Technologie de l'Information et de la Communication à la Faculté des Sciences de Tunis.

Au thème de ce projet, je tiens à exprimer ma sincère gratitude envers toutes les personnes qui m'ont soutenue tout au long de mon parcours.

A cette occasion, j'exprime ma profonde gratitude et mes sincères remerciements à mon encadrant académique, Monsieur Dhafer Mezghani, Maître de Conférences à la Faculté des Sciences de Tunis, qui a accepté de diriger ce travail et de donner ses précieux conseils.

Je tiens également à remercier Monsieur Achraf Smaoui, mon encadrant au sein de l'entreprise Orange Tunisie, pour le suivi qu'il a assuré sur mon projet, ainsi que pour sa disponibilité et les compétences qu'il a su m'apporter durant toute la durée de mon stage.

Mes remerciements s'adressent également à toute l'équipe de la Direction Réseaux et Services, pour leur soutien quotidien et la facilitation de mon intégration dans l'équipe, notamment à Monsieur Bilel Boussaa.

*Un remerciement particulier aux membres du jury pour avoir accepté de juger ce travail et de me faire profiter de leurs remarques et conseils. Je tiens tout particulièrement à exprimer ma gratitude envers le Président de jury, Monsieur **Med Anouar Ben Messaoud**, pour son implication et ses éclairages tout au long de l'évaluation de ce mémoire. Je suis également reconnaissante envers l'examinatrice, Madame **Rachida Bedira**, pour ses précieuses contributions et ses commentaires constructifs.*

Dédicace

C'est avec une profonde émotion et une grande reconnaissance que je dédie ce travail à toutes les personnes qui ont contribué, de près ou de loin, à son accomplissement

A mes chers parents

Kamel & Fatma

Pour leur amour, leurs sacrifices, leur soutien permanent et leurs encouragements tout au long de mon parcours. Vous avez toujours été ma source de force et de motivation. J'espère rester toujours à la hauteur de votre confiance et ne jamais vous décevoir.

Que Dieu vous procure bonne santé et longue vie.

A mes chers frères

Hamdi & Yassine

Pour leur soutien moral, leurs encouragements et leurs précieux conseils, qui m'ont accompagnée durant toutes les étapes de ce travail.

A toute personne qui a marqué ma vie par un mot, un conseil ou un geste qui m'a guidée vers le bon chemin.

Merci pour tout l'amour et la confiance et pour votre énorme support pendant la réalisation de mon projet.

Table des matières

<i>Liste des figures</i>	VIII
<i>Liste des tableaux</i>	IX
<i>Liste des acronymes</i>	X
Introduction générale	1
Chapitre 1 : Cadre général du projet	2
1. Introduction	2
2. Présentation de l'organisme d'accueil	2
2.1. BRI Technology	2
2.2. Présentation des services offerts par BRI Technology	2
2.3. Les solutions proposées par BRI Technology	3
3. Présentation du projet	4
3.1. Contexte du projet	4
3.2. Étude de l'existant	4
3.3. La problématique	5
3.4. La solution proposée	6
4. Méthodologie de Gestion de Projet : SCRUM	6
4.1. Le cycle de vie de Scrum	6
4.2. L'équipe de Scrum	7
5. Planification Agile du projet	7
5.1. Release 1 — SaaS bancaire, onboarding et activation de la marketplace	8
5.2. Release 2 — Parcours des concessionnaires, des clients et assistants conversationnels 8	
5.3. Correspondance entre les titres de réalisation et les sprints	9
6. Conclusion	9
Chapitre 2 : Analyse et spécification des besoins	11
1. Introduction	11
2. Identification des Acteurs	11
3. Concepts du SaaS et de la multi-tenant	12
3.1. Présentation du SaaS	12
3.2. Principe de multi-tenant	12
3.3. Principales stratégies multi-tenant	12
3.4. Choix d'architecture	14
4. Besoins fonctionnels	15

4.1. En tant que Administrateur SaaS	15
4.2. En tant que Administrateur Banque	16
4.3. En tant que Concessionnaire	16
4.4. En tant que Client.....	17
4.5. En tant que Internaute.....	17
5. Besoins non fonctionnels	18
Chapitre 3 : Architecture et Conception	19
1. Introduction	19
2. Environnement de développement et technologies utilisées	19
2.1. Environnement matériel	19
2.2. Environnement logiciel	19
Visual Studio Code	19
Visual Studio Code est un éditeur de code source développé par Microsoft. Léger et extensible, il prend en charge plusieurs langages de programmation.	19
IntelliJ IDEA	19
IntelliJ IDEA est un environnement de développement intégré développé par JetBrains. Il offre de nombreux outils facilitant le développement d'applications Java.....	19
Postman	20
Postman est une plateforme dédiée au développement et au test des API. Elle permet d'envoyer des requêtes HTTP et de vérifier les réponses retournées par le serveur.....	20
pgAdmin	20
pgAdmin est une plateforme open source d'administration spécialement conçue pour PostgreSQL. Elle permet notamment de créer, consulter et administrer les bases de données.	20
DBeaver	20
DBeaver est un outil de gestion de bases de données permettant de consulter les données et d'exécuter des requêtes SQL à travers une interface graphique.....	20
GitHub	20
GitHub est une plateforme d'hébergement et de gestion de code source basée sur Git. Elle facilite le suivi des versions et le travail collaboratif.....	20
2.3. Technologies utilisées	20
React	20
React est une bibliothèque JavaScript permettant de construire des interfaces utilisateur à partir de composants réutilisables.....	20
Spring Boot	21
Spring Boot est un framework Java basé sur l'écosystème Spring. Il fournit un ensemble de bibliothèques facilitant la création rapide et la gestion simplifiée des projets.	21
PostgreSQL	21

PostgreSQL est un système de gestion de base de données relationnelle open source permettant de stocker, organiser et manipuler les données de manière fiable et sécurisée. Il utilise principalement le langage SQL et prend en charge des fonctionnalités avancées. 21



Docker est une plateforme de conteneurisation permettant d'emballer une application avec ses dépendances dans des conteneurs. Elle facilite ainsi l'exécution de l'application dans différents environnements. 21

Docker Compose 21

Docker Compose est un outil de l'écosystème Docker permettant de définir et de lancer plusieurs conteneurs à partir d'un seul fichier de configuration. Il est utilisé pour orchestrer les différents services nécessaires au fonctionnement d'une application. 21

3. Architecture de la solution	22
3.1. Vue d'ensemble et choix architectural.....	22
3.2. Architecture logique.....	22
3.3. Architecture multi-tenant et isolation des données	23
3.4. Sécurité et contrôle d'accès	24
3.5. Architecture physique et intégrations externes.....	24
3.6. Synthèse des choix d'architecture.....	24
4. Modélisation fonctionnelle par cas d'utilisation	25
4.1. Diagramme global	25
4.2. Diagrammes et scénarios de cas d'utilisation détaillés.....	26

Chapitre 4 : Mise en œuvre du cycle de vie des marketplaces bancaires : onboarding, configuration, abonnement et activation..... 60

1. Introduction	60
2. Authentification, autorisation et contexte multi-tenant.....	60
3. Demande de création d'une marketplace bancaire	63
3.1. Renseignement des informations de la banque.....	66
3.2. Informations du futur Administrateur Banque et vérification de l'adresse e-mail	66
3.3. Configuration initiale de la marketplace, sélection des stores et modules	66
3.4. Récapitulatif et soumission de la demande.....	67
4. Back-office SaaS : administration et supervision de la plateforme	67
4.1. Tableau de bord.....	67
4.2. Gestion des stores et des modules.....	68
4.3. Gestion des demandes de marketplace	68
4.3.1. Traitement d'une demande de marketplace	68
4.3.2. Approbation et déclenchement du paiement.....	70

4.3.3. Paiement sécurisé par Stripe et activation de la marketplace	70
4.4. Gestion des banques	72
4.5. Gestion des utilisateurs et des rôles	72
4.6. Gestion des concessionnaires	72
4.7. Gestion des marketplaces	72
4.8. Gestion du contenu des marketplaces	72
4.9. Gestion des offres et des abonnements	72
4.10. Audit et logs	73
4.11. Paramètres de la plateforme	73
5. Back-office Banque : administration de la marketplace	73
5.1. Personnalisation de la marketplace	74
5.2. Gestion des stores, modules et paramètres	74
5.3. Renouvellement et l'ajout des services	74
6. Marketplace publique	75
7. Conclusion	75
Chapitre 5 : Réalisation des parcours métier : concessionnaires, financement et assistant IA....	77
1. Introduction	77
Ce chapitre a présenté les différentes fonctionnalités mises en œuvre pour les Concessionnaires et les Clients, depuis la gestion des produits et des partenariats jusqu'au processus de financement, ainsi que l'intégration de l'assistant IA et du chatbot au sein de la plateforme.	
2. Parcours Concessionnaires	77
2.1. Inscription et validation du concessionnaire	77
2.2. Partenariats et contrats	79
2.3. Produits et publications multi-banque	80
2.4. Suivi des demandes de financement	82
3. Parcours client et demande de financement	82
3.1. Consultation, comparaison et simulation	82
3.2. Inscription du client et activation du compte	83
3.3. Constitution et soumission du dossier	84
3.4. Traitement de la demande par la banque	85
4. Assistants conversationnels de la plateforme	87
4.1. Assistant IA pour le pilotage du SaaS	87
4.2. Fonctionnement et sécurisation de l'assistant IA	87
4.3. Chatbot public des marketplaces	88
5. Conclusion	89
Chapitre 6 : Mise en place de la démarche DevOps, du pipeline CI/CD et déploiement de Matchia sur Microsoft Azure	91

1. Introduction	91
2. Démarche DevOps et principes de la CI/CD.....	91
2.1. Démarche DevOps.....	92
2.2. Intégration et déploiement continu	92
3. Préparation et validation de la conteneurisation.....	92
3.1. Gestion des versions avec Git et GitHub	92
3.2. Conteneurisation du backend Spring Boot	93
3.3. Conteneurisation du frontend React	94
3.4. Orchestration locale avec Docker Compose.....	95
4. Mise en place du pipeline CI/CD avec Jenkins.....	96
4.1. Rôle de Jenkins	96
4.2. Pipeline as Code avec le Jenkinsfile	97
4.3. Déclenchement automatique avec GitHub Webhook et Cloudflare Tunnel.....	98
5. Exécution automatisée du pipeline CI/CD	99
5.1. Récupération du code et construction des applications	99
5.2. Exécution automatique des tests backend.....	100
5.3. Analyse statique avec SonarQube.....	101
5.4. Construction et publication des images Docker.....	102
6. Déploiement de Matchia sur Microsoft Azure.....	102
6.1. Préparation de l'environnement Cloud.....	102
6.2. Base de données PostgreSQL managée	103
6.3. Stockage persistant avec Azure Files.....	104
6.4. Hébergement avec Azure Container Apps.....	105
6.5. Déploiement automatisé avec Azure CLI.....	106
7. Architecture complète et validation du processus	107
7.1. Validation du processus complet.....	108
8. Conclusion.....	108

Liste des figures

Figure 1: Logo de BRI Technology	2
Figure 2: Cycle de vie de la méthodologie agile SCRUM	7
Figure 3: Principales stratégies d'architecture multi-tenant.....	14

Liste des tableaux

Liste des acronymes

SaaS : Software as a Service

NLP : Natural Language Processing

LLM : Large Language Model

CI : Continuous Integration

CD : Continuous Deployment

Introduction générale

Le secteur des télécommunications est un domaine en constante évolution, caractérisé par une demande croissante de services efficaces et une expérience utilisateur optimale. Cette évolution est marquée par une explosion du volume de données générées par les réseaux, les services et les abonnés. Face à ce défi, les opérateurs télécoms se tournent vers les technologies Big Data pour gérer efficacement ces données et optimiser leurs performances.

Dans ce contexte, Orange Tunisie, l'un des principaux acteurs du marché des télécommunications en Tunisie, s'est engagé dans une démarche d'innovation en concevant et en déployant une solution Big Data automatisée. Cette solution vise à améliorer et surveiller les indicateurs clés de performance des réseaux et des services. En garantissant une gestion efficace du trafic de données et de voix, ainsi qu'un suivi précis des abonnés dans le système de tarification, Orange Tunisie aspire à maintenir des niveaux optimaux d'efficacité opérationnelle. Ceci lui permettra de répondre de manière proactive aux besoins évolutifs des clients, assurant ainsi leur entière satisfaction.

L'objectif de ce rapport de stage est de fournir une analyse détaillée de la procédure de mise en œuvre et de développement de la solution chez Orange Tunisie, en incluant une synthèse de toutes les tâches effectuées. Il est divisé en trois chapitres répartis comme suit :

- **Présentation générale du projet** : Ce premier chapitre présente l'organisme d'accueil, Orange Tunisie. Nous abordons ensuite le sujet en étudiant et en critiquant l'existant, en précisant la problématique et la solution proposée pour y répondre. De plus, nous décrivons la méthodologie de travail adoptée pour notre projet.
- **État de l'art** : Le deuxième chapitre se concentre sur les concepts clés du Big Data, les outils nécessaires pour la réalisation de notre solution, ainsi que sur une analyse conceptuelle approfondie.
- **Réalisation concrète de la phase de mise en œuvre** : Dans ce troisième chapitre, nous examinons en détail les réalisations concrètes de la phase de mise en œuvre. Nous commençons par expliquer les étapes de déploiement, englobant la création de l'environnement de travail. Ensuite, nous explorons les différentes étapes de réalisation de notre solution.

Chapitre 1 : Cadre général du projet

1. Introduction

Ce projet est situé dans son contexte global en commençant par une introduction de l'organisme d'accueil. Nous examinons ensuite la situation actuelle pour identifier la solution appropriée au problème mentionné, tout en soulignant les objectifs à atteindre. Par la suite, nous étudierons la méthodologie appliquée.

2. Présentation de l'organisme d'accueil

2.1. BRI Technology

BRI Technology est une entreprise tunisienne fondée en 2010, spécialisée dans l'intelligence artificielle et la transformation digitale. Elle accompagne principalement les banques, les compagnies d'assurance, les opérateurs télécoms et les institutions publiques dans la modernisation de leurs services numériques. Grâce à son expertise technologique, l'entreprise propose des solutions intelligentes, sécurisées et évolutives adaptées aux besoins de chaque client. Elle intervient en Tunisie ainsi que dans plusieurs pays, notamment en Afrique, en Europe et en Amérique du Nord [1].



Figure 1: Logo de BRI Technology

2.2. Présentation des services offerts par BRI Technology

BRI Technology propose un ensemble de services destinés à accompagner les entreprises dans leur transformation numérique. Parmi ses principaux services, on distingue :

- **Conseil** : BRI Technology accompagne les organisations dans leur transformation numérique à travers la réalisation d'études stratégiques et l'élaboration de feuilles de route détaillées. Cette approche permet d'orienter l'évolution des systèmes d'information et de transformer les processus traditionnels en écosystèmes numériques modernes, efficaces et performants.
- **Recherche et développement** : BRI Technology intervient dans la conception, le développement et la mise en œuvre de solutions numériques avancées, adaptées aux

besoins spécifiques de différents secteurs d'activité. Ses réalisations couvrent notamment le commerce en ligne, les marketplaces et les plateformes digitales sur mesure. Chaque solution est conçue de manière personnalisée afin de répondre efficacement aux exigences et aux objectifs de chaque client.

- **Audit** : BRI Technology met à disposition son expertise pour analyser et évaluer les systèmes d'information existants. Elle accompagne également les établissements publics et privés dans la définition de plans directeurs stratégiques et opérationnels, favorisant ainsi l'amélioration continue, la performance et l'alignement avec les objectifs à long terme.
- **Formation et certification** : BRI Technology propose des programmes de formation adaptés aux besoins de ses clients et de leurs équipes. Ces formations sont accompagnées, si nécessaire, de services de certification permettant de valider les compétences acquises, de renforcer le savoir-faire des collaborateurs et d'assurer la conformité aux standards exigés.

2.3. Les solutions proposées par BRI Technology

Les solutions proposées par BRI Technology couvrent plusieurs domaines technologiques. L'entreprise développe des solutions d'intelligence artificielle et d'analyse de données permettant d'exploiter les données afin de faciliter la prise de décision, la prédiction et l'automatisation intelligente. Elle accompagne également les organisations dans leur stratégie de transformation digitale à travers l'optimisation des processus, la modernisation des systèmes existants et l'adoption de nouvelles technologies.

Dans le domaine bancaire, BRI Technology propose des solutions de digital banking permettant l'intégration des systèmes bancaires, la mise en place de plateformes web et mobiles, les solutions de paiement et les portefeuilles numériques. Son produit phare, **Matchia**, est une marketplace de financement intelligente basée sur l'intelligence artificielle. Cette solution permet la simulation de produits financiers, l'évaluation d'éligibilité en temps réel, l'onboarding client automatisé et l'intégration avec différents partenaires.

L'entreprise propose également des solutions de cybersécurité, de développement logiciel sur mesure, d'applications web et mobiles, d'architectures API et microservices, ainsi que des solutions cloud natives. Ces solutions visent à offrir aux entreprises des plateformes modernes, performantes, sécurisées et capables d'évoluer selon leurs besoins.

3. Présentation du projet

3.1. Contexte du projet

Le secteur bancaire connaît une évolution continue portée par la digitalisation des services et par l'apparition de nouveaux usages numériques. Dans ce contexte, les banques cherchent à enrichir leur offre en proposant à leurs clients des services complémentaires accessibles à travers des plateformes digitales.

Les marketplaces bancaires s'inscrivent dans cette évolution en permettant de regrouper, au sein d'un même environnement, différents produits et services proposés en collaboration avec plusieurs partenaires. Leur mise en œuvre nécessite toutefois une organisation capable de gérer efficacement la diversité des acteurs, des offres et des configurations associées à chaque établissement bancaire.

Avec l'augmentation du nombre de banques et de partenaires, la gestion de ces environnements peut rapidement devenir complexe. Il devient alors essentiel d'adopter une approche permettant de simplifier leur administration, de faciliter leur évolution et d'améliorer la centralisation des différentes opérations.

3.2. Étude de l'existant

Avant la mise en place de la solution proposée, la gestion des marketplaces bancaires reposait principalement sur une approche spécifique à chaque établissement. Chaque banque disposait de son propre environnement, configuré et adapté selon ses besoins.

Cette organisation permettait de répondre aux besoins particuliers de chaque banque, mais elle reposait fortement sur des interventions manuelles et sur la multiplication d'environnements indépendants. Avec l'augmentation du nombre de banques et de partenaires, plusieurs limites sont apparues.

Les principales caractéristiques de cette solution existante peuvent être résumées comme suit :

- **Marketplace spécifique à chaque banque :** Dans l'approche existante, chaque banque disposait d'une version spécifique et personnalisée de la marketplace, adaptée manuellement par l'équipe de développement selon ses besoins.

Chaque nouvelle mise en place nécessitait donc une configuration dédiée ainsi que, dans certains cas, des modifications ou des corrections du code. Cette multiplication des versions rendait la maintenance et l'évolution plus complexes, puisque certaines modifications devaient être reproduites sur plusieurs environnements.

- **Demande de marketplace et onboarding traditionnel :** La création d'une marketplace nécessitait des échanges directs entre la banque et le fournisseur de la solution afin de définir les besoins, les services souhaités et les différentes configurations. Ce processus restait donc dépendant de plusieurs interventions humaines.
- **Paiement non intégré :** Le paiement n'était pas intégré au processus de gestion de la marketplace et était traité manuellement ou à travers un système externe. Il n'était donc pas directement lié à l'activation automatique des services, ce qui nécessitait une intervention humaine pour confirmer le règlement, activer les services concernés et assurer le suivi des abonnements.
- **Gestion séparée des concessionnaires :** La gestion des concessionnaires était répartie entre les différentes marketplaces. Lorsqu'un concessionnaire collaborait avec plusieurs banques, il devait gérer un compte et un espace distinct pour chaque marketplace. Les informations sur les produits et les stocks devaient alors être mises à jour dans plusieurs environnements, ce qui entraînait une dispersion des données et une charge de gestion supplémentaire.

3.3. La problématique

L'étude de l'existant met en évidence plusieurs limites majeures :

- **Multiplication des applications et complexité de maintenance :** chaque banque disposant d'une marketplace spécifique, les évolutions et corrections devaient être reproduites sur plusieurs environnements.
- **Processus de création peu automatisé :** la mise en place d'une nouvelle marketplace nécessitait plusieurs échanges et interventions entre la banque et le fournisseur de la solution.
- **Paiement et activation non intégrés :** le règlement, sa vérification, la mise à jour de l'abonnement et l'activation des services nécessitaient des opérations supplémentaires.

- **Gestion décentralisée des concessionnaires** : un même concessionnaire devait gérer plusieurs comptes, produits et stocks dans différentes marketplaces, entraînant une dispersion des données et une charge de gestion supplémentaire.

3.4. La solution proposée

Afin de répondre aux besoins identifiés, la solution proposée consiste à mettre en place **Matchia**, une plateforme **SaaS multi-tenant** permettant de centraliser la gestion de plusieurs marketplaces bancaires au sein d'un même système. Chaque banque dispose de son propre espace personnalisable, avec ses stores, modules, contenus et paramètres, tout en s'appuyant sur un socle applicatif commun. La plateforme intègre également le processus d'adhésion en regroupant les étapes de demande, de validation, de paiement et d'activation dans un même parcours. Par ailleurs, la gestion des concessionnaires est centralisée grâce à un compte unique leur permettant de collaborer avec plusieurs banques et de gérer leurs produits et partenariats depuis un seul espace. La solution offre ainsi une plateforme centralisée, configurable, automatisée et évolutive, adaptée à la gestion de plusieurs établissements bancaires et de leurs différents partenaires.

4. Méthodologie de Gestion de Projet : SCRUM

Dans le cadre de notre projet, nous avons choisi d'appliquer la méthodologie agile **Scrum** pour structurer le développement. Le travail a ainsi été réparti en plusieurs sprints, chacun regroupant un ensemble cohérent de fonctionnalités à concevoir, développer et valider. Cette organisation nous a permis d'assurer un avancement progressif du projet, de mieux gérer les priorités et d'intégrer les ajustements nécessaires au fur et à mesure de l'évolution des besoins.

4.1. Le cycle de vie de Scrum

Le cycle de vie Scrum débute par la définition des **User Stories** par le Product Owner, qui les ajoute et les priorise dans le **Product Backlog** selon leur importance et leur valeur métier. Lors de la **Sprint Planning**, l'équipe sélectionne les User Stories à réaliser pendant le sprint et les décompose en tâches constituant le **Sprint Backlog**. Le sprint démarre ensuite pour une durée déterminée, généralement comprise entre deux et quatre semaines. Durant cette période, l'équipe réalise les tâches planifiées et organise quotidiennement un **Daily Scrum** afin de suivre l'avancement et d'identifier les éventuels obstacles. À la fin du sprint, l'incrément développé

est présenté et évalué lors de la **Sprint Review**, puis l'équipe analyse son organisation et les améliorations possibles durant la **Sprint Retrospective**. Le cycle reprend ensuite avec la planification du sprint suivant, permettant ainsi une évolution progressive et continue du produit [2].

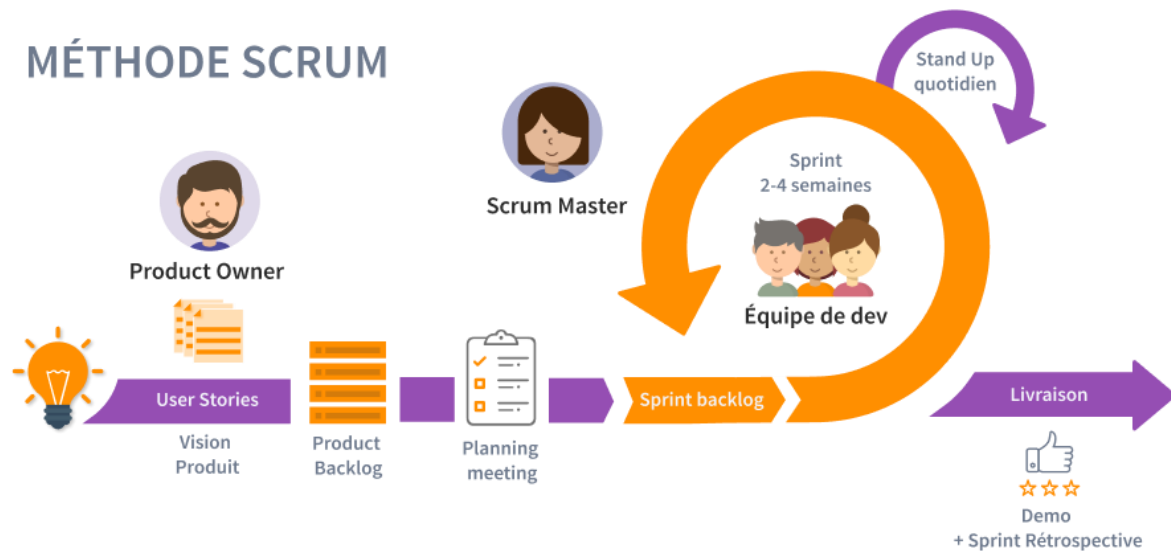


Figure 2: Cycle de vie de la méthodologie agile SCRUM

4.2. L'équipe de Scrum

Le projet repose sur les principaux rôles définis par la méthodologie SCRUM:

- **Product Owner** : identifie les besoins fonctionnels, définit les priorités du Product Backlog et valide les fonctionnalités réalisées.
- **Scrum Master** : facilite l'application de Scrum, accompagne l'équipe et veille au bon déroulement des différentes itérations.
- **Équipe de développement** : assure la conception, le développement, les tests et l'intégration des fonctionnalités de la plateforme.

5. Planification Agile du projet

La planification du projet s'appuie sur l'approche Agile Scrum. Le Product Backlog est organisé par incréments fonctionnels cohérents avec la structure des chapitres de réalisation. Chaque sprint, prévu sur deux semaines, produit un ensemble de fonctionnalités testables et directement rattachées à une section ou à un sous-titre du rapport.

Deux releases structurent cette planification. La première couvre la réalisation du socle SaaS et le cycle de vie complet de la marketplace bancaire. La seconde regroupe les parcours des concessionnaires et des clients, ainsi que les assistants conversationnels. Selon l'ampleur fonctionnelle, un titre de chapitre peut être réalisé par plusieurs sprints, tandis qu'un sous-titre ciblé peut correspondre à un sprint unique.

5.1. Release 1 — SaaS bancaire, onboarding et activation de la marketplace

Cette release correspond aux principales sections du chapitre 4. Elle permet de livrer progressivement le socle multi-tenant, la demande de création de marketplace, l'administration SaaS, le paiement puis les espaces bancaire et public.

Sprint	Objectif	Éléments clés du backlog
S1	Socle SaaS multi-tenant	Authentification, gestion des sessions, autorisation par rôles et application du contexte multi-tenant afin d'isoler les données de chaque banque.
S2	Demande de création de marketplace	Informations de la banque, futur Administrateur Banque, vérification de l'e-mail, slug, identité visuelle, stores, modules, récapitulatif et soumission.
S3	Back-office SaaS	Tableau de bord, stores et modules, banques, utilisateurs et rôles, concessionnaires, marketplaces, contenus, offres, abonnements, audit, logs et paramètres.
S4	Traitement, paiement et activation	Consultation des demandes, approbation ou rejet avec motif, e-mails, lien Stripe, contrôle du paiement, activation de la banque, de la marketplace, de l'abonnement et du compte Administrateur Banque.
S5	Back-office Banque et marketplace publique	Personnalisation et contenus, configuration des stores et modules, renouvellement ou ajout de services, accueil public, produits, comparateur, simulateur, chatbot et demande de financement.

5.2. Release 2 — Parcours des concessionnaires, des clients et assistants conversationnels

Cette release correspond aux sections du chapitre 5. Elle traite les parcours métier qui complètent l'exploitation de la marketplace, depuis l'inscription des concessionnaires jusqu'au dépôt et au traitement des demandes de financement des clients.

Sprint	Objectif	Éléments clés du backlog
S6	Inscription et validation du concessionnaire	Demande d'inscription, dépôt des pièces justificatives, contrôle SaaS, création du compte et e-mail des identifiants.

S7	Partenariats et contrats	Demandes initiées par la banque ou le concessionnaire, validation, cycle de vie du contrat et gestion des conditions de collaboration.
S8	Produits, publications et suivi des dossiers	Catalogue centralisé, stock, publications par marketplace et consultation des demandes de financement associées aux produits.
S9	Parcours client et demande de financement	Inscription et vérification du compte, consultation, comparaison, simulation, constitution et traitement du dossier.
S10	Assistants conversationnels et validation	Assistant IA du SaaS, chatbot public à règles métier, contrôle des accès, tests et validation fonctionnelle globale.

5.3. Correspondance entre les titres de réalisation et les sprints

Le tableau suivant explicite le rattachement entre la structure fonctionnelle du rapport et les incréments prévus dans le Product Backlog.

Titre ou sous-titre de réalisation	Sprint	Incrément fonctionnel livré
Chapitre 4 — Authentification, autorisation et contexte multi-tenant	S1	Accès sécurisé, rôles et cloisonnement des données par
Chapitre 4 — Demande de création de marketplace bancaire	S2	Saisie, vérification e-mail, configuration, stores, mod soumission.
Chapitre 4 — Back-office SaaS : administration et supervision	S3	Tableau de bord et gestion des acteurs, catalogues, co abonnements, audit et paramètres.
Chapitre 4 — Traitement, approbation et paiement	S4	Décision, e-mails, paiement Stripe et activation conditi des services.
Chapitre 4 — Back-office Banque et marketplace publique	S5	Configuration bancaire, services souscrits et parcours jusqu'à la demande de financement.
Chapitre 5 — Inscription et validation du concessionnaire	S6	Onboarding contrôlé du concessionnaire.
Chapitre 5 — Partenariats et contrats	S7	Collaboration banque-concessionnaire et contractualis
Chapitre 5 — Produits, publications et suivi des demandes	S8	Gestion centralisée du catalogue et des dossiers liés au produits.
Chapitre 5 — Parcours client et demande de financement	S9	Du compte client jusqu'à la décision bancaire sur le dos
Chapitre 5 — Assistants conversationnels	S10	Assistant IA du SaaS, chatbot public et validation finale

À l'issue de chaque sprint, les fonctionnalités réalisées font l'objet de tests techniques et fonctionnels avant leur présentation en Sprint Review. Les retours recueillis permettent d'ajuster les priorités du Product Backlog et de préparer le sprint suivant. Cette planification maintient ainsi une progression cohérente entre les titres du rapport, les besoins fonctionnels et les incréments effectivement livrés.

6. Conclusion

En conclusion, ce chapitre a établi les bases du projet en présentant l'entreprise d'accueil, **BRI-Technology**. Il a également inclus une étude de l'existant, défini la problématique à résoudre et introduit la solution proposée ainsi que la méthodologie adoptée

Chapitre 2 : Analyse et spécification des besoins

1. Introduction

Ce chapitre est consacré à l'analyse et à la spécification des besoins de la plateforme. Il présente les différents acteurs du système, leurs besoins fonctionnels et non fonctionnels, ainsi que les principaux choix liés à l'architecture SaaS multi-tenant.

2. Identification des Acteurs

L'identification des acteurs permet de déterminer les différents intervenants de la plateforme et de préciser leur rôle dans le système. Elle constitue une étape essentielle pour structurer les besoins fonctionnels et définir les interactions propres à chaque type d'utilisateur.

Tableau 1: Acteurs de la plateforme Matchia

Acteur	Description
Administrateur SaaS	Acteur principal chargé de l'administration globale de la plateforme. Il gère notamment les banques, les demandes d'adhésion, les demandes concessionnaires, les stores, les modules, les abonnements, les paiements, ainsi que la configuration générale du système.
Administrateur Banque	Responsable de la gestion de la marketplace de sa banque. Il administre les stores, les modules, les produits, les contenus, les clients, les concessionnaires partenaires et les demandes de financement.
Concessionnaire	Partenaire commercial pouvant collaborer avec plusieurs banques depuis un espace centralisé. Il gère ses produits ainsi que ses demandes, invitations et partenariats avec les banques.

Client	Utilisateur authentifié de la marketplace bancaire. Il peut gérer son profil, effectuer des simulations, déposer des demandes de financement, fournir les documents nécessaires et suivre l'état de ses demandes.
Internaute	Utilisateur non authentifié pouvant consulter les marketplaces, les stores et les produits, ainsi qu'utiliser certaines fonctionnalités publiques telles que la comparaison et la simulation.

3. Concepts du SaaS et de la multi-tenant

3.1. Présentation du SaaS

Le **Software as a Service (SaaS)** est un modèle de distribution dans lequel une application est accessible en ligne sous forme de service, généralement hébergée dans le Cloud. Le fournisseur prend en charge l'hébergement, la maintenance, les mises à jour et l'exploitation technique, tandis que les utilisateurs accèdent à la solution à distance, souvent via un navigateur web et selon un modèle d'abonnement sans avoir à gérer directement l'installation, l'infrastructure ou la maintenance technique[3].

3.2. Principe de multi-tenant

Le multi-tenant est un principe d'architecture fréquemment associé au modèle SaaS. Elle permet à plusieurs clients ou organisations d'utiliser la même application, tout en conservant chacun un environnement logique qui lui est propre. Chaque client, appelé **tenant**, partage ainsi le même socle technique, tandis que ses données restent isolées de celles des autres tenants.

Ce principe repose sur deux notions essentielles : la mutualisation des ressources et l'isolation des données. L'application est développée et maintenue une seule fois, puis mise à disposition de plusieurs tenants. Des mécanismes de contrôle permettent ensuite d'identifier le tenant concerné et de garantir que chaque utilisateur accède uniquement aux données et aux fonctionnalités relevant de son propre périmètre. Cette approche permet ainsi de combiner centralisation, personnalisation et séparation logique au sein d'une même plateforme.

3.3. Principales stratégies multi-tenant

Plusieurs stratégies peuvent être adoptées pour mettre en œuvre une architecture multi-tenant. Elles se distinguent principalement par la manière dont les données des différents tenants sont stockées, isolées et administrées [4].

1. Base de données dédiée par tenant

Dans cette stratégie, chaque tenant dispose de sa propre base de données. Les données de chaque organisation sont donc physiquement séparées de celles des autres tenants, ce qui offre une isolation maximale des données, ce qui renforce la sécurité et la confidentialité, et facilite les sauvegardes ou restaurations propres à un client. En revanche, elle engendre un coût d'infrastructure élevé, puisqu'il faut gérer autant de bases que de clients, et devient plus difficile à maintenir et à faire évoluer à mesure que le nombre de tenants augmente.

2. Schéma dédié par tenant

Cette approche repose sur une base de données commune, mais chaque tenant possède son propre schéma. Les données restent ainsi séparées de manière structurelle, tout en partageant la même infrastructure de base de données. Cette stratégie constitue un compromis entre isolation et mutualisation, car elle réduit le nombre de bases à administrer. Toutefois, la gestion des schémas, des migrations et des évolutions peut devenir complexe lorsque le nombre de tenants devient important.

3. Base et schéma partagés

Dans cette stratégie, tous les tenants utilisent la même base de données et le même schéma. Les données sont stockées dans des tables communes et sont distinguées à l'aide d'identifiants permettant de déterminer à quel tenant chaque donnée appartient. L'isolation est donc principalement assurée au niveau logique et applicatif. Cette approche facilite la mutualisation des ressources, réduit les coûts d'infrastructure et simplifie la maintenance globale. Elle exige toutefois des contrôles rigoureux afin de garantir qu'un tenant ne puisse jamais accéder aux

données

d'un

aut

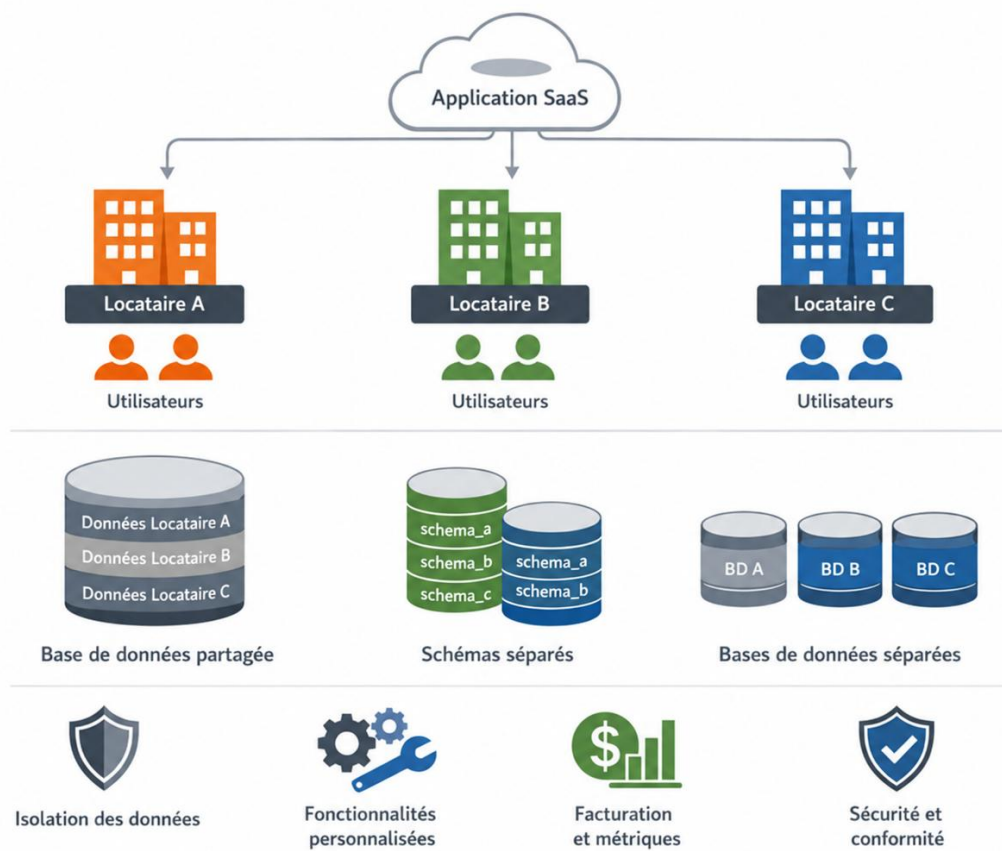


Figure 3: Principales stratégies d'architecture multi-tenant

3.4. Choix d'architecture

Après l'analyse des différentes stratégies de multi-tenant, le choix d'une base de données unique et partagée avec une isolation logique des données constitue la solution la plus adaptée à notre projet. Cette architecture permet de centraliser les données des différentes banques dans une même infrastructure, tout en maintenant une séparation stricte entre les tenants grâce à des mécanismes d'identification, de filtrage et de contrôle d'accès. Elle simplifie l'administration de la plateforme, facilite l'intégration de nouvelles banques et réduit la complexité liée à la gestion de plusieurs bases indépendantes. Ce choix offre ainsi un bon compromis entre mutualisation, maintenabilité, évolutivité et maîtrise des coûts, tout en exigeant une gestion rigoureuse de l'isolation des données au niveau applicatif.

4. Besoins fonctionnels

Les besoins fonctionnels traduisent les capacités que la plateforme doit offrir à chacun de ses acteurs. Ils permettent de structurer les fonctionnalités par profil utilisateur et de délimiter précisément les opérations accessibles selon les responsabilités et les droits associés à chaque rôle.

4.1. En tant que Administrateur SaaS

L'Administrateur SaaS dispose du niveau d'administration le plus élevé. Il assure la gestion globale de la plateforme et supervise les différents tenants.

Ses principales fonctionnalités sont les suivantes :

- S'authentifier et gérer son profil ;
- consulter le tableau de bord global de la plateforme ;
- consulter et traiter les demandes d'adhésion des banques ;
- approuver ou rejeter une demande ;
- affecter un espace backoffice pour la demande approuvée
- gérer les banques enregistrées dans la plateforme ;
- gérer les principales configurations de la plateforme ;
- gérer les marketplaces associées aux banques ;
- gérer les stores disponibles ;
- gérer les modules proposés;
- associer les modules aux différents stores ;
- gérer les offres et les abonnements ;
- consulter et traiter les demandes de création de comptes concessionnaires ;
- consulter les notifications ;
- utiliser l'assistant intelligent pour interroger certaines informations autorisées de la plateforme ;
- Consulter les logs et l'historique d'audit afin d'assurer la traçabilité des actions réalisées sur la plateforme.
- gérer les principales configurations de la plateforme ;

L'espace SaaS constitue ainsi le point central d'administration permettant de superviser l'ensemble de l'écosystème Matchia.

4.2. En tant que Administrateur Banque

L'Administrateur Banque assure la gestion de la marketplace appartenant à son établissement.

Il doit notamment pouvoir :

- s'authentifier et gérer son profil
- consulter le tableau de bord
- gérer les clients et les utilisateurs de la marketplace
- gérer les stores associés et activés pour sa marketplace
- gérer la personnalisation de la marketplace
- gérer et configurer les modules disponibles dans chaque store
- gérer les produits proposés
- gérer les contenus de la marketplace
- gérer les abonnements et suivre les services souscrits
- gérer les concessionnaires partenaires
- gérer les demandes de financement et consulter les documents associés
- Consulter et gérer les notifications
- L'administrateur Banque agit uniquement dans le périmètre de son établissement et ne doit pas accéder aux données appartenant aux autres banques.

4.3. En tant que Concessionnaire

Le Concessionnaire dispose d'un espace centralisé lui permettant de collaborer avec plusieurs banques. Il doit pouvoir :

- s'inscrire et accéder à son espace après validation
- s'authentifier et gérer son profil
- consulter son tableau de bord de son espace
- gérer ses produits et leurs stocks
- gérer ses publications auprès des banques partenaires
- gérer les demandes de partenariat envoyées et les invitations reçues
- gérer les partenariats et consulter les contrats associés avec les banques partenaires
- recevoir des notifications relatives aux demandes, invitations et partenariats

L'un des besoins majeurs de cet espace est de permettre au concessionnaire de travailler avec plusieurs banques à partir d'un compte unique, sans multiplier les comptes et les espaces d'administration.

Cette centralisation facilite également la gestion des produits et réduit les opérations répétitives liées aux différents partenariats bancaires.

4.4. En tant que Client

Le Client est un utilisateur authentifié d'une marketplace bancaire. Son espace est principalement orienté vers les demandes de financement.

Le client doit pouvoir :

- S'inscrire, s'authentifier et gérer son profil
- Consulter les produits et leurs détails
- Comparer les produits disponibles
- Effectuer une simulation de financement
- Créer et soumettre une demande de financement
- Gérer les documents associés à la demande
- Consulter et suivre l'état de ses demandes
- Recevoir les notifications liées au traitement de ses demandes

4.5. En tant que Internaute

L'Internaute représente un utilisateur non authentifié consultant les espaces publics des marketplaces.

Il peut notamment :

- accéder aux marketplaces bancaires publiques;
- consulter les stores disponibles ;
- parcourir les produits proposés ;
- consulter les informations détaillées d'un produit ;
- comparer plusieurs produits lorsque le module correspondant est actif ;
- utiliser le simulateur de financement lorsque celui-ci est disponible;

L'internaute peut consulter et simuler sans être connecté. En revanche, lorsqu'il souhaite déposer une demande de financement, le système doit l'inviter à s'inscrire ou à s'authentifier.

5. Besoins non fonctionnels

Les besoins non fonctionnels définissent les qualités attendues de la plateforme et les exigences nécessaires pour assurer un fonctionnement fiable, sécurisé, performant et adapté à une architecture SaaS multi-tenant. Parmi les principaux besoins non fonctionnels retenus pour la solution proposée, nous citons :

- **Sécurité** : La plateforme doit assurer une authentification sécurisée des utilisateurs et appliquer un contrôle d'accès basé sur les rôles. La séparation logique des données entre les différentes banques doit également être garantie afin d'éviter tout accès non autorisé aux informations d'un autre tenant.
- **Scalabilité** : La solution doit être capable d'intégrer progressivement de nouvelles banques, marketplaces, concessionnaires, d'utilisateurs et de données, sans nécessiter la création d'une nouvelle application pour chaque établissement.
- **Performance** : La solution doit assurer des temps de réponse satisfaisants et une navigation fluide.
- **Maintenabilité** : L'architecture de la plateforme doit faciliter les corrections, les évolutions et l'ajout de nouvelles fonctionnalités.
- **Disponibilité** : La plateforme doit garantir une disponibilité suffisante des services et assurer un fonctionnement stable des différents composants.

Chapitre 3 : Architecture et Conception

1. Introduction

2. Environnement de développement et technologies utilisées

2.1. Environnement matériel

Pour développer notre solution, nous avons utilisé un ordinateur dont les principales caractéristiques matérielles sont présentées dans le tableau suivant.

Processeur	11th Gen Intel(R) Core(TM) i5-11300H @ 3.10GHz (3.11 GHz)
RAM	16,0 Go
Système d'exploitation	Windows 11

Tableau : Caractéristiques de l'environnement matériel

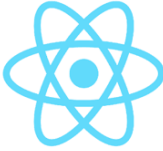
2.2. Environnement logiciel

Pour assurer le développement et les tests de notre solution, nous avons utilisé plusieurs outils logiciels présentés ci-dessous.

 Visual Studio Code	Visual Studio Code est un éditeur de code source développé par Microsoft. Léger et extensible, il prend en charge plusieurs langages de programmation.
 IntelliJ IDEA	IntelliJ IDEA est un environnement de développement intégré développé par JetBrains. Il offre de nombreux outils facilitant le développement d'applications Java

 Postman	Postman est une plateforme dédiée au développement et au test des API. Elle permet d'envoyer des requêtes HTTP et de vérifier les réponses retournées par le serveur.
 pgAdmin	pgAdmin est une plateforme open source d'administration spécialement conçue pour PostgreSQL. Elle permet notamment de créer, consulter et administrer les bases de données.
 DBeaver	DBeaver est un outil de gestion de bases de données permettant de consulter les données et d'exécuter des requêtes SQL à travers une interface graphique.
 GitHub	GitHub est une plateforme d'hébergement et de gestion de code source basée sur Git. Elle facilite le suivi des versions et le travail collaboratif.

2.3. Technologies utilisées

 React	React est une bibliothèque JavaScript permettant de construire des interfaces utilisateur à partir de composants réutilisables.
---	---

 Spring Boot	Spring Boot est un framework Java basé sur l'écosystème Spring. Il fournit un ensemble de bibliothèques facilitant la création rapide et la gestion simplifiée des projets.
 PostgreSQL	PostgreSQL est un système de gestion de base de données relationnelle open source permettant de stocker, organiser et manipuler les données de manière fiable et sécurisée. Il utilise principalement le langage SQL et prend en charge des fonctionnalités avancées.
 Docker	Docker est une plateforme de conteneurisation permettant d'emballer une application avec ses dépendances dans des conteneurs. Elle facilite ainsi l'exécution de l'application dans différents environnements.
 Docker Compose	Docker Compose est un outil de l'écosystème Docker permettant de définir et de lancer plusieurs conteneurs à partir d'un seul fichier de configuration. Il est utilisé pour orchestrer les différents services nécessaires au fonctionnement d'une application.

3. Architecture de la solution

L'architecture de Matchia a été conçue pour mutualiser un socle technique commun tout en préservant l'autonomie fonctionnelle et l'isolation logique de chaque banque. Elle adopte une architecture client-serveur en couches, mise en œuvre sous la forme d'un monolithe modulaire. Ce choix répond aux besoins du projet : centraliser l'administration SaaS, déployer plusieurs marketplaces bancaires à partir d'une même application et faire évoluer séparément les principaux domaines métier.

3.1. Vue d'ensemble et choix architectural

La plateforme repose sur une application web monopage et une API REST sécurisée. Les espaces public, Client, Concessionnaire, Administrateur Banque et Administrateur SaaS partagent le même socle applicatif ; leurs routes, leurs données visibles et leurs actions autorisées varient cependant selon le rôle et, lorsque cela s'applique, selon la banque de rattachement. Cette séparation limite le couplage entre l'interface, les règles métier et la persistance.

Le monolithe modulaire a été retenu afin de conserver un déploiement simple et des transactions cohérentes, sans renoncer à une organisation claire par domaines : authentification, banques et marketplaces, concessionnaires, catalogue et véhicules, clients, financements, paiements, notifications et assistants intelligents. Cette modularité facilite les tests, la maintenance et une éventuelle extraction future de services indépendants.

3.2. Architecture logique

L'architecture logique s'organise en cinq niveaux complémentaires. La couche de présentation, réalisée avec React et TypeScript, gère les vues, les formulaires et la navigation. La couche API expose les contrôleurs REST et les objets d'échange, puis valide les requêtes avant de les transmettre aux services métier. Spring Security et le filtre JWT assurent transversalement l'authentification, l'autorisation par rôle et la prise en compte du contexte multi-tenant.

La couche métier porte les règles propres à chaque domaine fonctionnel. Elle orchestre notamment la création et le traitement des demandes de marketplace, l'inscription des concessionnaires et des clients, la gestion du catalogue, les demandes de financement, les paiements et les échanges avec les assistants. Enfin, la couche de persistance s'appuie sur Spring Data JPA et PostgreSQL ; les dépôts encapsulent les accès aux données et appliquent les critères de filtrage nécessaires.

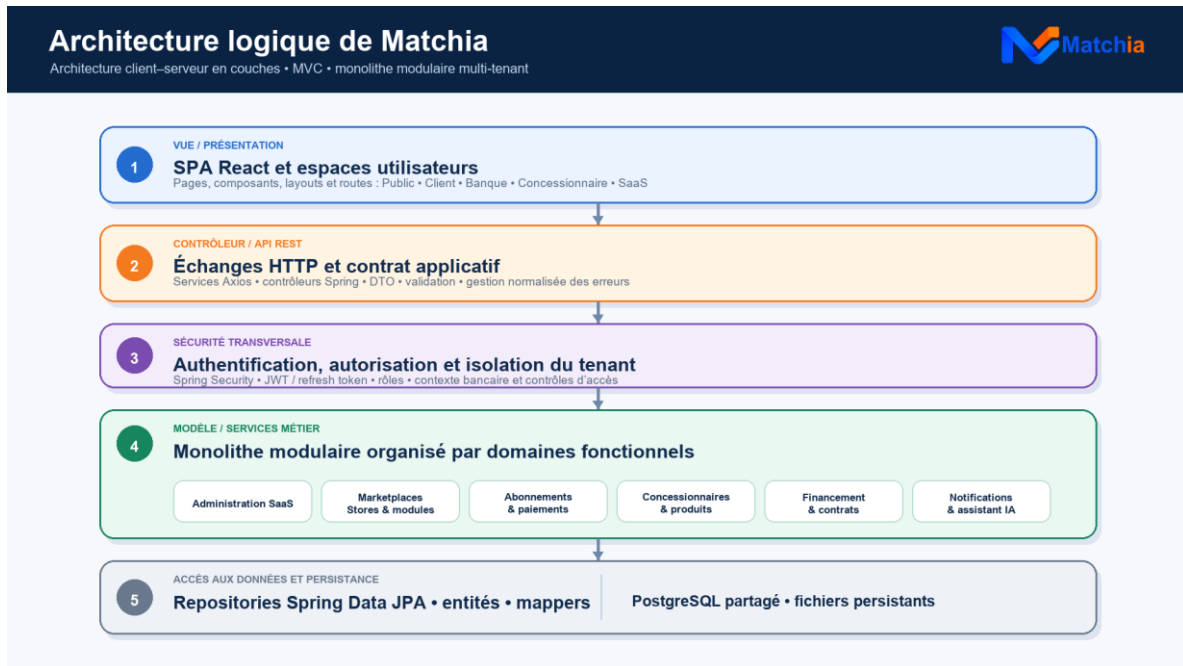


Figure 3.1 — Architecture logique en couches de la plateforme Matchia

3.3. Architecture multi-tenant et isolation des données

Matchia utilise une application et une base de données partagées par plusieurs banques. Chaque banque constitue un tenant disposant de sa propre marketplace, de ses paramètres, de ses administrateurs et de ses données métier. Les ressources globales du fournisseur SaaS sont distinguées des ressources rattachées à une banque, ce qui permet de mutualiser l'infrastructure sans dupliquer l'application.

Le contexte du tenant est déterminé à partir du compte authentifié pour les espaces privés et à partir de l'identifiant public de la marketplace, tel que le slug ou l'hôte, pour les parcours publics. Les entités concernées conservent une référence vers la banque. Les contrôleurs, les services et les dépôts vérifient cette référence et filtrent systématiquement les recherches par banque. Ainsi, une ressource appartenant à un autre tenant n'est ni exposée ni modifiable, même lorsque son identifiant technique est connu.

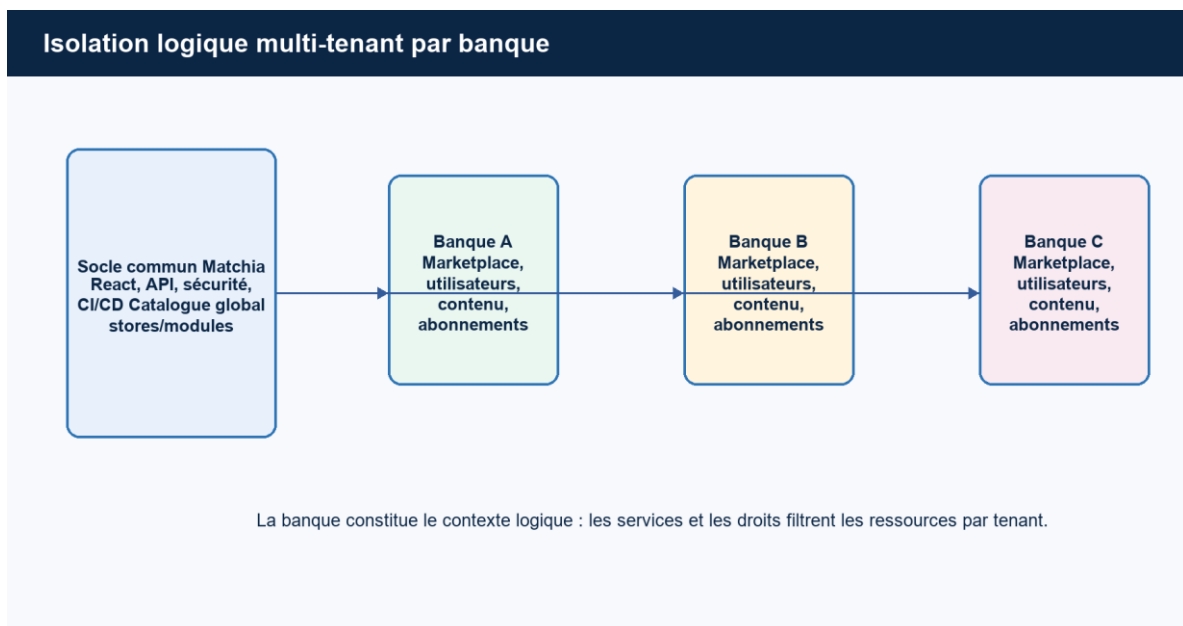


Figure 3.2 — Isolation logique des données dans l'architecture multi-tenant

3.4. Sécurité et contrôle d'accès

La sécurité repose sur Spring Security et une authentification stateless par jetons JWT. Après validation des identifiants et de l'état du compte, l'API délivre un jeton d'accès ; le mécanisme de renouvellement permet de maintenir la session selon les règles définies. Chaque requête protégée est interceptée afin d'identifier l'utilisateur, son rôle et son contexte d'accès.

Le contrôle d'accès combine les autorisations liées aux rôles — Administrateur SaaS, Administrateur Banque, Concessionnaire et Client — avec les vérifications liées au tenant. Les opérations sensibles sont contrôlées dans la couche métier, tandis que les dépôts limitent les lectures aux données autorisées. Les événements significatifs peuvent être consignés dans les journaux d'audit. Les secrets techniques et les clés des services externes sont fournis par la configuration de l'environnement de déploiement.

3.5. Architecture physique et intégrations externes

Du point de vue physique, l'utilisateur accède à Matchia depuis un navigateur web. Le frontend React, construit avec Vite et servi par Nginx, communique en HTTPS avec l'API Spring Boot. Le backend, exécuté avec Java 17, applique les règles de sécurité et les traitements métier avant d'accéder à PostgreSQL par l'intermédiaire de JPA. Les documents fonctionnels, notamment ceux liés aux demandes de financement, sont gérés par le backend et associés aux enregistrements métier correspondants.

Trois intégrations complètent le système. Stripe prend en charge le paiement associé au traitement d'une demande et retourne au backend l'état de la transaction. Gemini alimente l'assistant SaaS et le chatbot de marketplace, dans le respect du contexte transmis par l'application. Enfin, le serveur SMTP assure l'envoi des codes de vérification et des notifications. Les composants déployés, leur hébergement et le stockage persistant sont détaillés dans le chapitre consacré au déploiement.

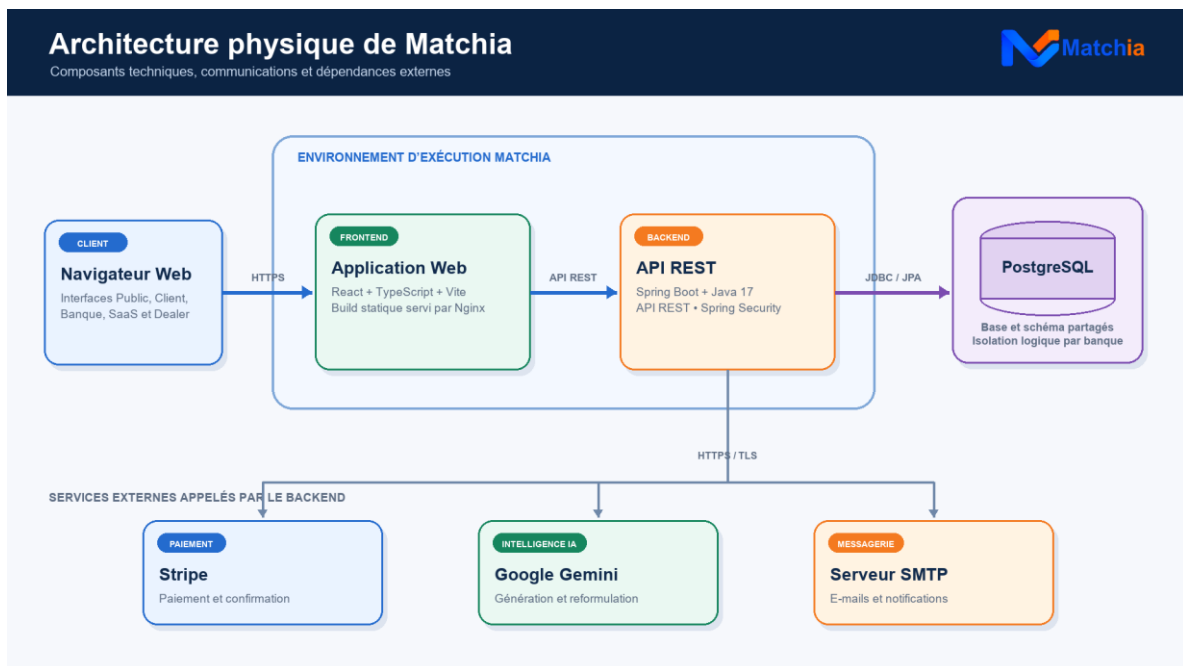


Figure 3.3 — Architecture physique et intégrations externes de Matchia

3.6. Synthèse des choix d'architecture

Cette architecture concilie simplicité de déploiement, séparation des responsabilités et évolutivité. Le monolithe modulaire facilite la maintenance, tandis que la mutualisation réduit les coûts. Chaque accès aux données tenantisées demeure néanmoins associé au contexte de la banque et soumis aux contrôles d'autorisation.

4. Modélisation fonctionnelle par cas d'utilisation

Cette section présente les interactions entre les acteurs de Matchia et les principales fonctionnalités de la plateforme. Les acteurs humains sont placés à gauche, les systèmes externes à droite et les relations UML « include » et « extend » distinguent les comportements obligatoires des comportements conditionnels.

4.1. Diagramme global

Le diagramme global regroupe les parcours de l'Internaute, du Client, du Concessionnaire, de l'Administrateur Banque et de l'Administrateur SaaS, ainsi que les interactions avec les services externes.

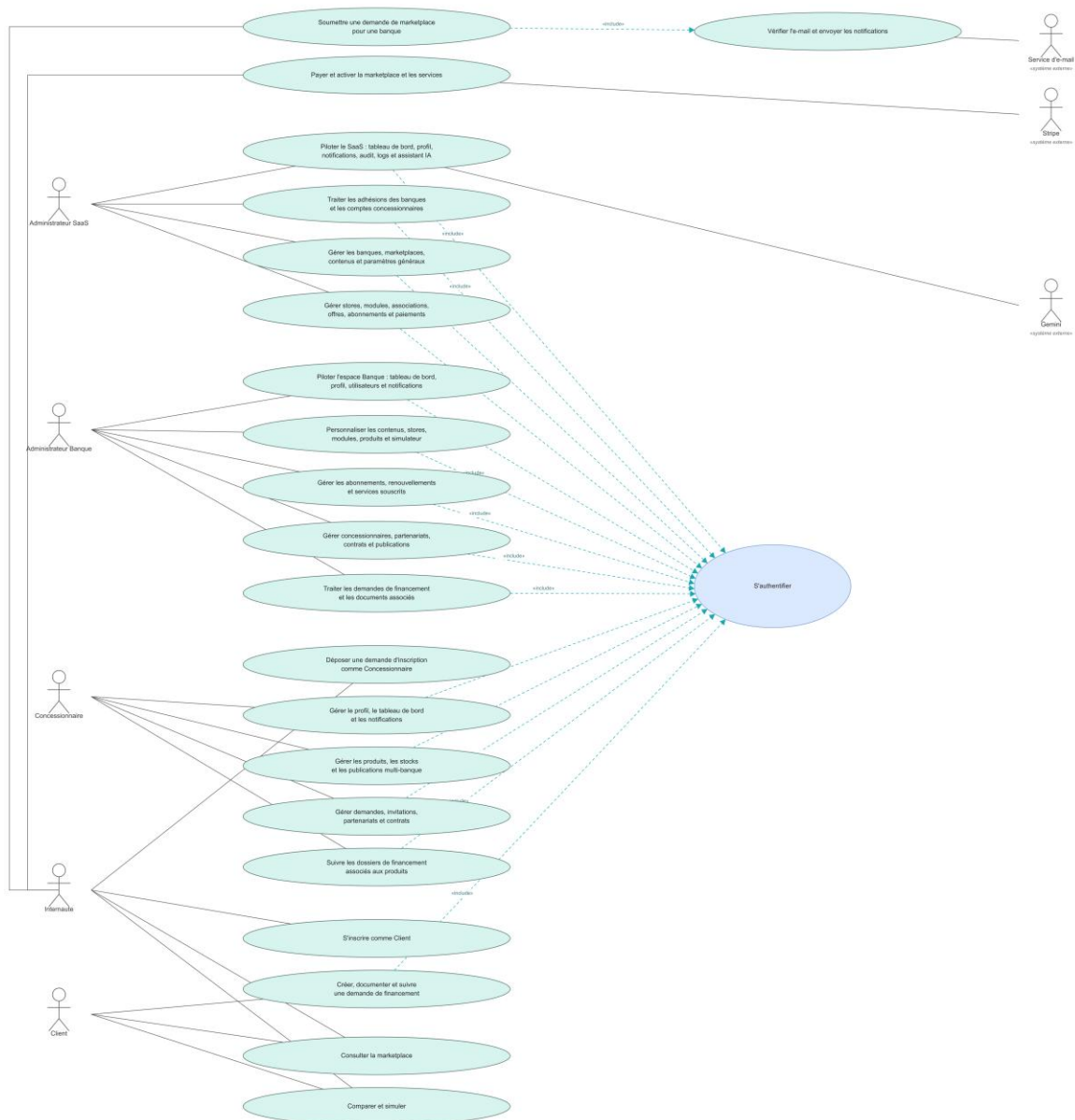


Figure 3.4 — Diagramme global des cas d'utilisation de Matchia

4.2. Diagrammes et scénarios de cas d'utilisation détaillés

Chaque diagramme est suivi d'un tableau de scénario précisant les acteurs, les conditions d'exécution, le déroulement nominal et les principales alternatives. Les cas sont regroupés par objectif métier et l'authentification n'est incluse que pour les parcours protégés.

4.2.1. UC-01 — S'authentifier

Ce cas d'utilisation décrit le processus « S'authentifier ». La figure présente les relations UML et le tableau formalise le scénario fonctionnel correspondant.

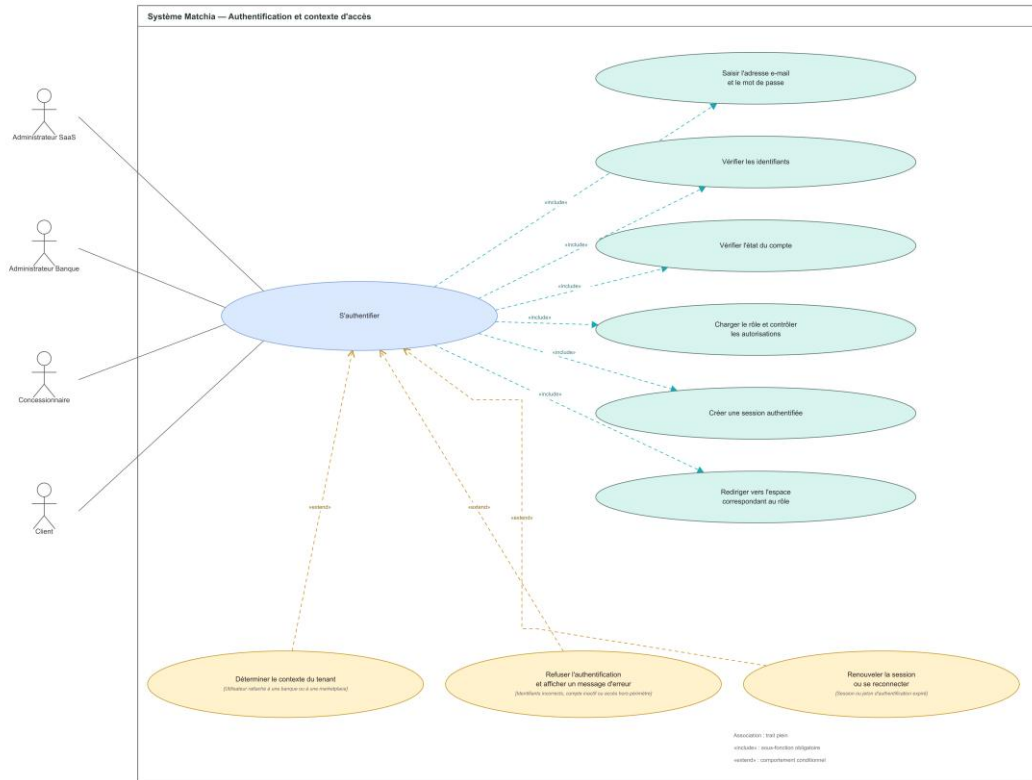


Figure 3.5 — Diagramme de cas d'utilisation détaillé : S'authentifier

Tableau 3.1 — Description du scénario UC-01

Élément	Description
Cas d'utilisation	UC-01 — S'authentifier
Acteur principal	Utilisateur : Administrateur SaaS, Administrateur Banque, Concessionnaire ou Client.
Acteurs secondaires	Aucun acteur secondaire direct.
Objectif	Permettre à un utilisateur d'accéder à l'espace correspondant à son rôle, à ses autorisations et, si nécessaire, au tenant auquel il est rattaché.
Préconditions	L'utilisateur possède un compte enregistré et accède au formulaire de connexion. Le service d'authentification est disponible.
Déclencheur	L'utilisateur saisit ses identifiants et soumet le formulaire de connexion.
Postconditions en cas de succès	Une session authentifiée est créée. Le rôle, les autorisations et, le cas échéant, le contexte du tenant sont associés à la session. L'utilisateur est redirigé vers l'espace approprié.
Postconditions en cas d'échec	Aucune session valide n'est créée ou conservée. L'accès à la ressource demandée est refusé et un message adapté est affiché.
Scénario nominal	1. L'utilisateur accède au formulaire de connexion. 2. Il saisit son adresse e-mail et son mot de passe. 3. Il soumet le formulaire de connexion. 4. Le système vérifie les identifiants fournis et l'état du compte. 5. Le système récupère le rôle et contrôle les autorisations. 6. Si l'utilisateur est rattaché à une banque ou à une marketplace, le système détermine le contexte du tenant. 7. Le système crée la session authentifiée. 8. L'utilisateur est redirigé vers l'espace correspondant à son rôle.
Alternatives et exceptions	1. Identifiants incorrects : l'authentification est refusée et un message d'erreur est affiché. 2. Compte inactif : l'accès est refusé. 3. Contexte du tenant invalide ou accès hors périmètre : la ressource est refusée. 4. Session ou jeton expiré : l'utilisateur doit renouveler la session ou se reconnecter.

4.2.2. UC-02 — Soumettre une demande de marketplace bancaire

Ce cas d'utilisation décrit le processus « Soumettre une demande de marketplace bancaire ». La figure présente les relations UML et le tableau formalise le scénario fonctionnel correspondant.

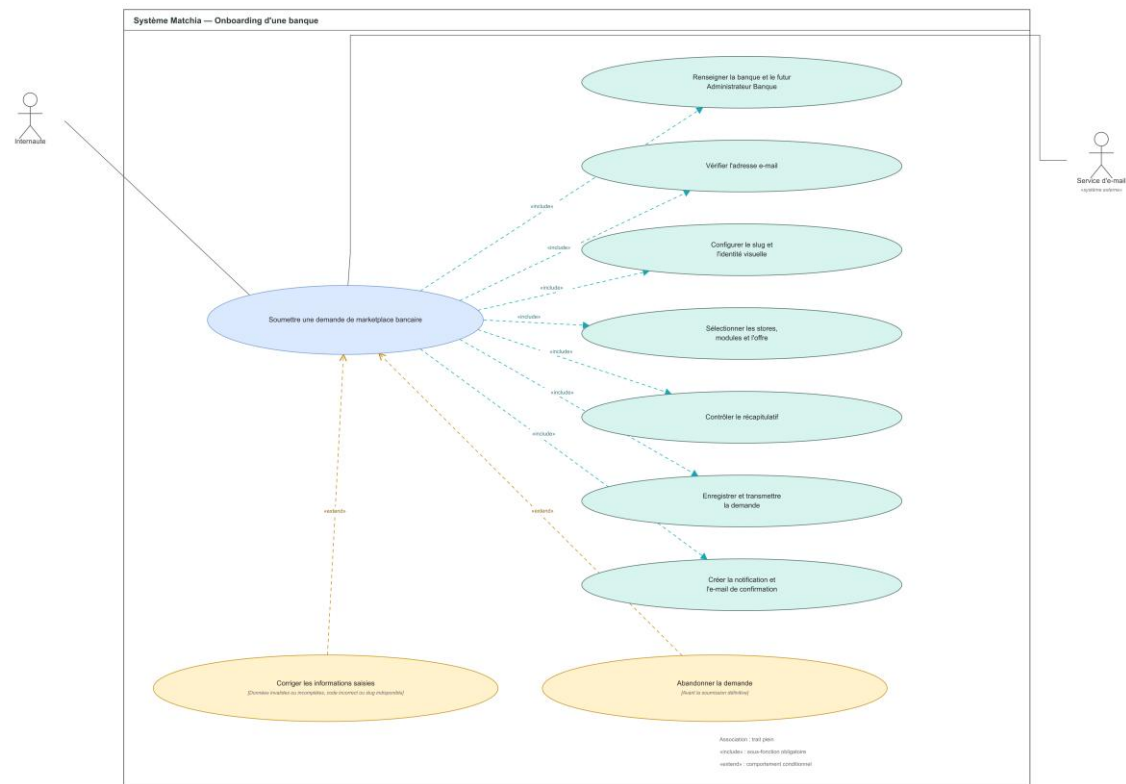


Figure 3.6 — Diagramme de cas d'utilisation détaillé : Soumettre une demande de marketplace bancaire

Tableau 3.2 — Description du scénario UC-02

Élément	Description
Cas d'utilisation	UC-02 — Soumettre une demande de marketplace bancaire
Acteur principal	Internaute agissant comme représentant de la banque ou futur Administrateur Banque.
Acteurs secondaires	Service d'e-mail.
Objectif	Enregistrer une demande complète de création de marketplace bancaire afin qu'elle puisse être examinée par l'Administrateur SaaS.
Préconditions	Le formulaire public est accessible. Le demandeur dispose d'une adresse e-mail valide et les stores, modules et offres proposés sont disponibles.
Déclencheur	L'Internaute démarre une nouvelle demande de marketplace.
Postconditions en cas de succès	La demande est enregistrée avec son récapitulatif et son statut initial. L'Administrateur SaaS est notifié et le demandeur reçoit une confirmation.
Postconditions en cas d'échec	Aucune demande incomplète ou non vérifiée n'est transmise pour traitement.
Scénario nominal	1. L'Internaute renseigne les informations de la banque et du futur Administrateur Banque. 2. Le système envoie un code de vérification à l'adresse e-mail indiquée. 3. L'Internaute saisit le code reçu et le système valide l'adresse. 4. L'Internaute configure le slug, l'identité visuelle, les stores, les modules et l'offre souhaités. 5. Le système affiche un récapitulatif et contrôle la complétude des données. 6. L'Internaute confirme et soumet la demande. 7. Le système enregistre la demande, crée la notification et envoie l'e-mail de confirmation.
Alternatives et exceptions	1. Données invalides ou incomplètes : le système signale les champs à corriger. 2. Code incorrect ou expiré : la vérification est refusée et un nouveau code peut être demandé. 3. Slug indisponible : le demandeur doit choisir une autre valeur. 4. Abandon avant soumission : aucune demande définitive n'est créée.

4.2.3. UC-03 — Administrer et superviser la plateforme SaaS

Ce cas d'utilisation décrit le processus « Administrer et superviser la plateforme SaaS ». La figure présente les relations UML et le tableau formalise le scénario fonctionnel correspondant.

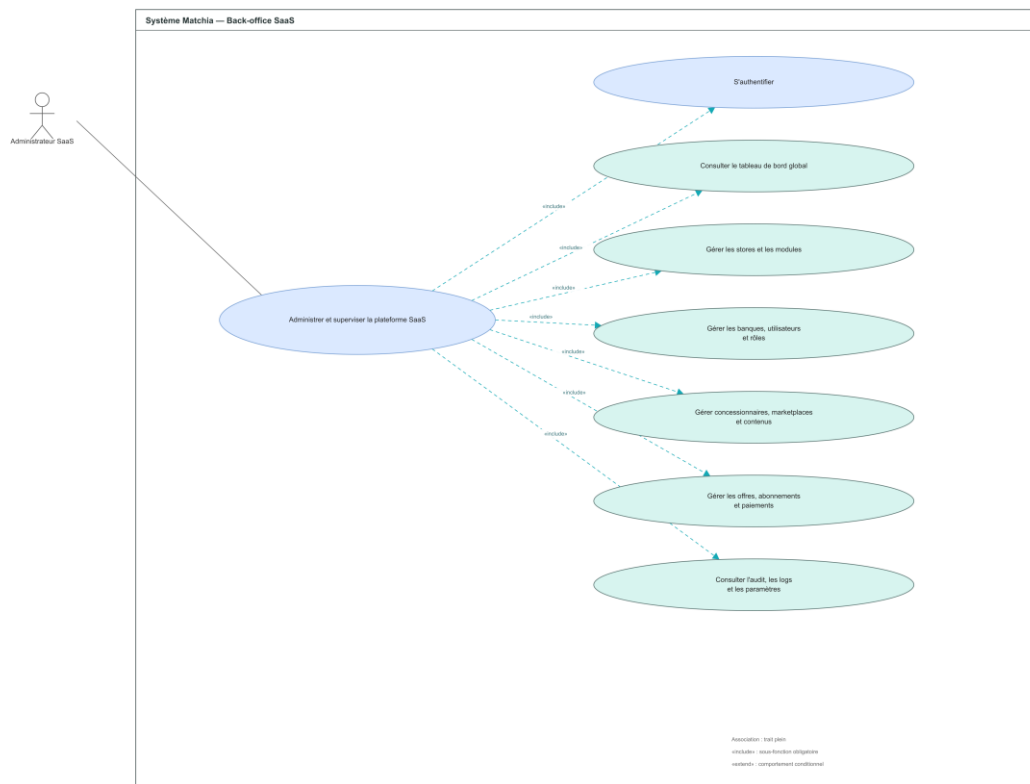


Figure 3.7 — Diagramme de cas d'utilisation détaillé : Administrer et superviser la plateforme SaaS

Tableau 3.3 — Description du scénario UC-03

Élément	Description
Cas d'utilisation	UC-03 — Administrer et superviser la plateforme SaaS
Acteur principal	Administrateur SaaS.
Acteurs secondaires	Aucun acteur secondaire direct.
Objectif	Piloter l'écosystème Matchia et administrer les ressources globales, les tenants, les offres, les abonnements et les paramètres de la plateforme.
Préconditions	L'Administrateur SaaS possède un compte actif, est authentifié et dispose des autorisations d'administration globale.
Déclencheur	L'Administrateur SaaS ouvre le back-office SaaS ou sélectionne une fonction d'administration.
Postconditions en cas de succès	Les consultations et modifications autorisées sont exécutées, persistées dans le périmètre approprié et tracées dans l'audit.
Postconditions en cas d'échec	Aucune modification non autorisée ou incohérente n'est enregistrée.
Scénario nominal	1. L'Administrateur SaaS s'authentifie et accède au tableau de bord global. 2. Il choisit le domaine à administrer : stores, modules, banques, utilisateurs, rôles, concessionnaires, marketplaces, contenus, offres, abonnements, paiements ou paramètres. 3. Le système affiche les données et indicateurs correspondants. 4. L'administrateur consulte, crée ou met à jour les ressources autorisées. 5. Le système valide les données et les transitions de statut. 6. Le système enregistre l'action et ajoute une trace d'audit.
Alternatives et exceptions	1. Données invalides : l'enregistrement est refusé avec indication des erreurs. 2. Transition de statut interdite ou ressource introuvable : l'opération est annulée. 3. Droit insuffisant : l'accès à la fonction est refusé.

4.2.4. UC-04 — Traiter une demande de marketplace

Ce cas d'utilisation décrit le processus « Traiter une demande de marketplace ». La figure présente les relations UML et le tableau formalise le scénario fonctionnel correspondant.

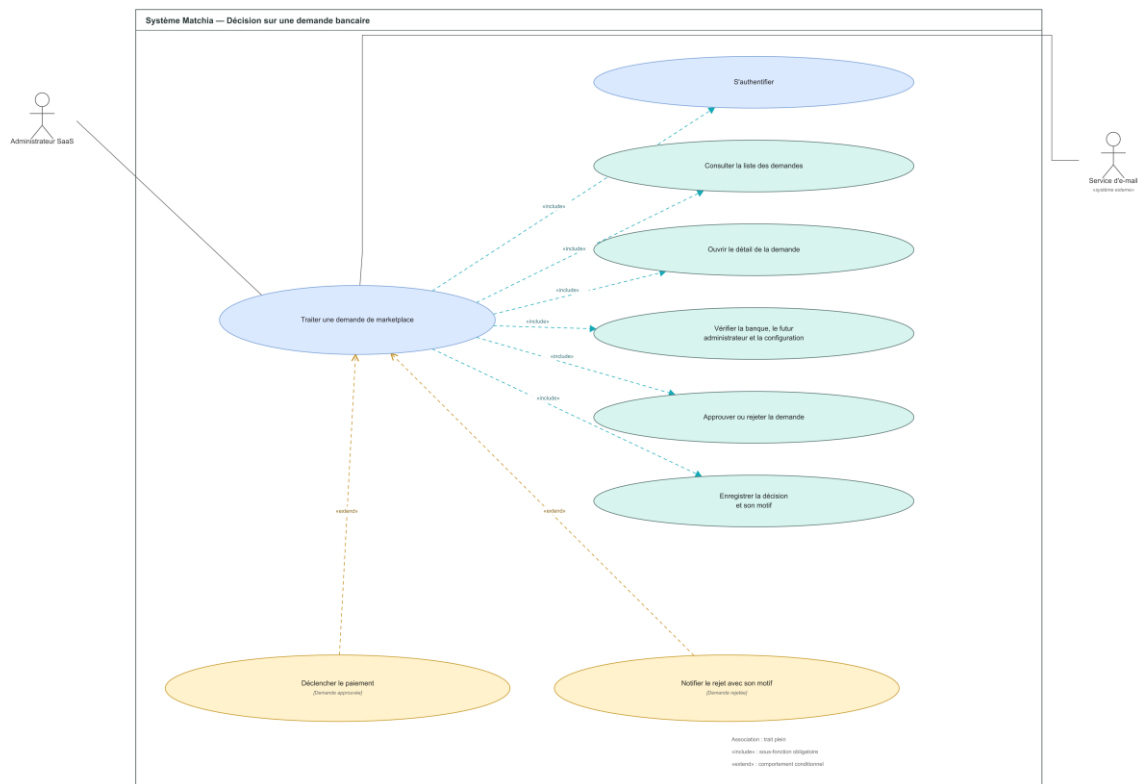


Figure 3.8 — Diagramme de cas d'utilisation détaillé : Traiter une demande de marketplace

Tableau 3.4 — Description du scénario UC-04

Élément	Description
Cas d'utilisation	UC-04 — Traiter une demande de marketplace
Acteur principal	Administrateur SaaS.
Acteurs secondaires	Demandeur bancaire et service d'e-mail.
Objectif	Examiner une demande de marketplace, enregistrer une décision motivée et déclencher le paiement lorsqu'elle est approuvée.
Préconditions	L'Administrateur SaaS est authentifié. Une demande complète existe au statut en attente de traitement.
Déclencheur	L'Administrateur SaaS ouvre une demande depuis la liste des demandes reçues.
Postconditions en cas de succès	La décision est enregistrée. Une approbation déclenche le parcours de paiement ; un rejet conserve le motif et le communique au demandeur.
Postconditions en cas d'échec	La demande reste dans son état précédent et aucun paiement ni service n'est activé.
Scénario nominal	1. L'administrateur consulte la liste des demandes. 2. Il ouvre le détail de la demande sélectionnée. 3. Le système affiche les informations de la banque, du futur administrateur et de la configuration souhaitée. 4. L'administrateur vérifie la cohérence et la complétude du dossier. 5. Il approuve la demande ou la rejette en indiquant un motif. 6. Le système enregistre la décision. 7. En cas d'approbation, le système déclenche le paiement ; en cas de rejet, il notifie le demandeur.
Alternatives et exceptions	1. Demande approuvée : un lien de paiement est généré et transmis. 2. Demande rejetée : le motif est obligatoire et envoyé au demandeur. 3. Dossier incomplet ou statut incompatible : la décision est bloquée. 4. Service d'e-mail indisponible : la décision reste enregistrée et l'envoi est signalé en erreur.

4.2.5. UC-05 — Payer et activer la marketplace

Ce cas d'utilisation décrit le processus « Payer et activer la marketplace ». La figure présente les relations UML et le tableau formalise le scénario fonctionnel correspondant.

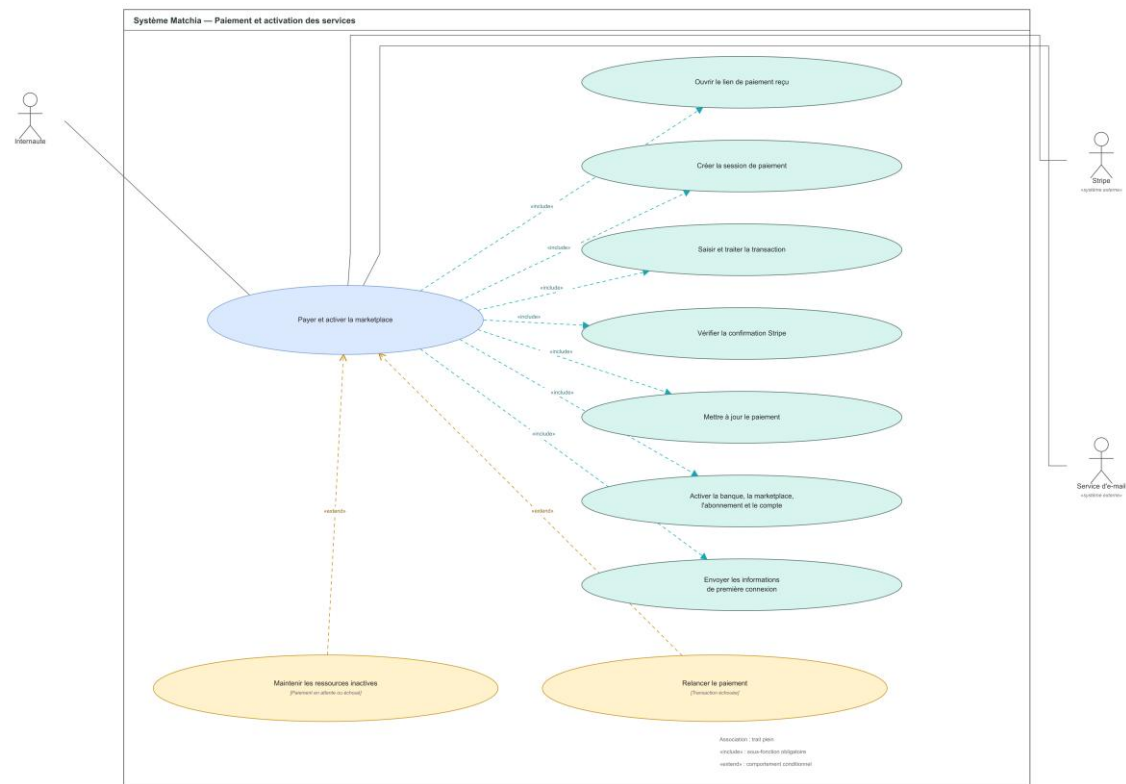


Figure 3.9 — Diagramme de cas d'utilisation détaillé : Payer et activer la marketplace

Tableau 3.5 — Description du scénario UC-05

Élément	Description
Cas d'utilisation	UC-05 — Payer et activer la marketplace
Acteur principal	Internaute représentant la banque et destinataire du lien de paiement.
Acteurs secondaires	Stripe et service d'e-mail.
Objectif	Finaliser le paiement d'une demande approuvée puis activer la banque, la marketplace, l'abonnement et le compte Administrateur Banque.
Préconditions	La demande de marketplace est approuvée, le lien de paiement est valide et les ressources à activer sont encore inactives.
Déclencheur	Le représentant de la banque ouvre le lien de paiement reçu.
Postconditions en cas de succès	Le paiement est confirmé. Les ressources souscrites et le compte Administrateur Banque sont activés, puis les informations de première connexion sont envoyées.
Postconditions en cas d'échec	Les ressources restent inactives et aucun accès privé à la marketplace n'est accordé.
Scénario nominal	1. L'Internaute ouvre le lien de paiement reçu par e-mail. 2. Le système crée une session de paiement Stripe. 3. L'Internaute saisit les informations requises et confirme la transaction. 4. Stripe traite le paiement et renvoie son statut. 5. Le backend vérifie la confirmation Stripe et met à jour le paiement. 6. Le système active la banque, la marketplace, l'abonnement, les stores, les modules et le compte Administrateur Banque. 7. Le service d'e-mail transmet les informations de première connexion.
Alternatives et exceptions	1. Paiement en attente, annulé ou échoué : les ressources restent inactives. 2. Transaction échouée : le demandeur peut relancer le paiement. 3. Confirmation Stripe absente ou incohérente : l'activation est bloquée. 4. Nouvelle notification de paiement déjà traitée : le système évite une seconde activation.

4.2.6. UC-06 — Administrer la marketplace bancaire

Ce cas d'utilisation décrit le processus « Administrer la marketplace bancaire ». La figure présente les relations UML et le tableau formalise le scénario fonctionnel correspondant.

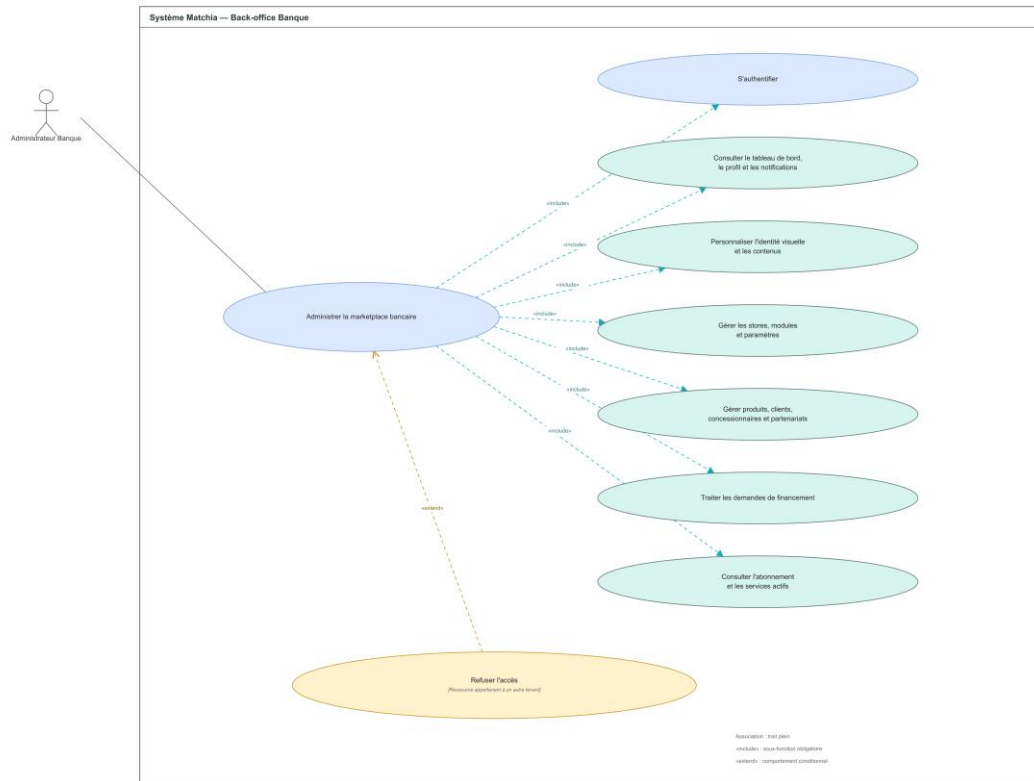


Figure 3.10 — Diagramme de cas d'utilisation détaillé : Administrer la marketplace bancaire

Tableau 3.6 — Description du scénario UC-06

Élément	Description
Cas d'utilisation	UC-06 — Administrer la marketplace bancaire
Acteur principal	Administrateur Banque.
Acteurs secondaires	Aucun acteur secondaire direct.
Objectif	Configurer et exploiter la marketplace de la banque exclusivement dans le tenant auquel l'administrateur est rattaché.
Préconditions	Le compte Administrateur Banque, la banque, la marketplace et l'abonnement sont actifs. L'utilisateur est authentifié dans le bon tenant.
Déclencheur	L'Administrateur Banque ouvre son espace d'administration ou sélectionne une fonction de gestion.
Postconditions en cas de succès	Les informations et décisions autorisées sont enregistrées dans le tenant de la banque et deviennent visibles dans les espaces concernés.
Postconditions en cas d'échec	Aucune ressource d'un autre tenant ou aucun service non souscrit n'est modifié.
Scénario nominal	1. L'Administrateur Banque s'authentifie et accède à son tableau de bord. 2. Il consulte son profil, les notifications et les indicateurs de sa marketplace. 3. Il personnalise l'identité visuelle et les contenus. 4. Il configure les stores, modules et paramètres autorisés par l'abonnement. 5. Il gère les produits, clients, concessionnaires, partenariats et contrats de son tenant. 6. Il traite les demandes de financement et consulte l'état de l'abonnement. 7. Le système enregistre et trace les actions réalisées.
Alternatives et exceptions	1. Accès à une ressource d'un autre tenant : l'opération est refusée. 2. Module ou service non actif : la fonction n'est pas accessible. 3. Donnée invalide ou transition interdite : l'enregistrement est annulé.

4.2.7. UC-07 — Renouveler ou ajouter des services

Ce cas d'utilisation décrit le processus « Renouveler ou ajouter des services ». La figure présente les relations UML et le tableau formalise le scénario fonctionnel correspondant.

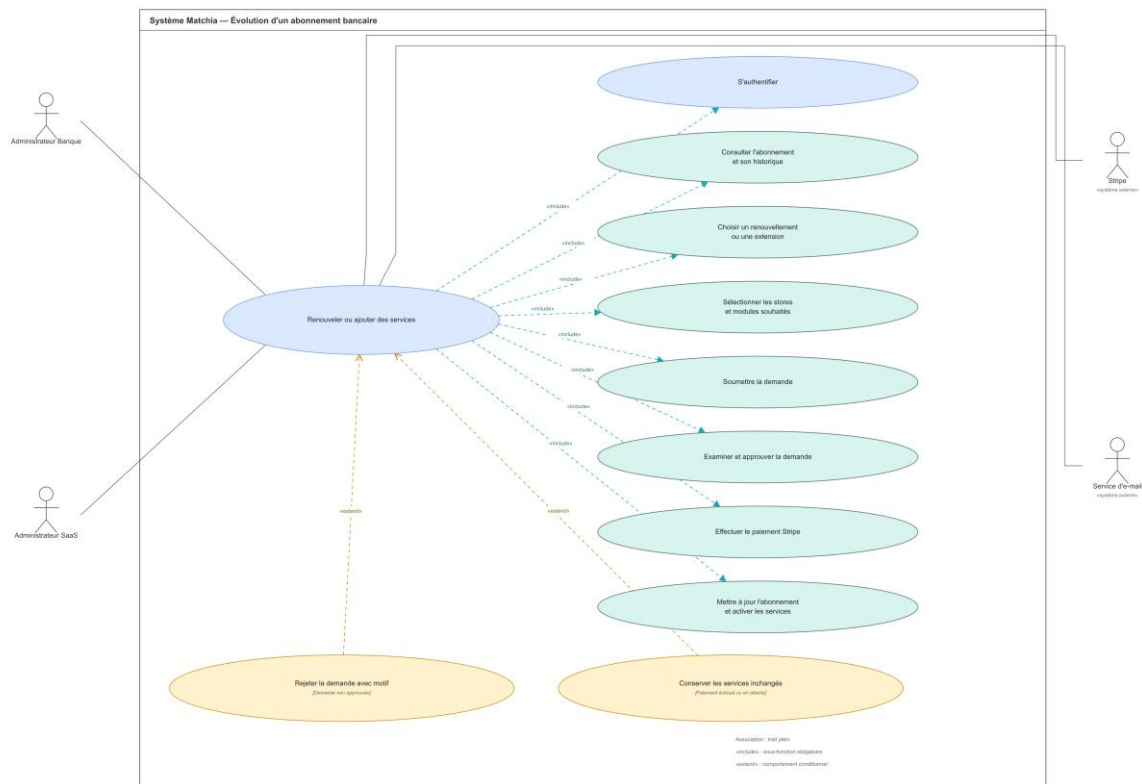


Figure 3.11 — Diagramme de cas d'utilisation détaillé : Renouveler ou ajouter des services

Tableau 3.7 — Description du scénario UC-07

Élément	Description
Cas d'utilisation	UC-07 — Renouveler ou ajouter des services
Acteur principal	Administrateur Banque.
Acteurs secondaires	Administrateur SaaS, Stripe et service d'e-mail.
Objectif	Renouveler l'abonnement d'une banque ou ajouter des stores et modules, après approbation et paiement confirmé.
Préconditions	L'Administrateur Banque est authentifié et un abonnement existe pour sa marketplace.
Déclencheur	L'Administrateur Banque sélectionne l'action de renouvellement ou d'extension de services.
Postconditions en cas de succès	L'abonnement est mis à jour et les services approuvés et payés sont activés pour la marketplace.
Postconditions en cas d'échec	L'abonnement et les services existants restent inchangés.
Scénario nominal	1. L'Administrateur Banque consulte l'abonnement et son historique. 2. Il choisit un renouvellement ou une extension. 3. Il sélectionne les stores et modules souhaités, puis soumet la demande. 4. L'Administrateur SaaS examine et approuve la demande. 5. Le système transmet le lien de paiement. 6. L'Administrateur Banque effectue le paiement via Stripe. 7. Le backend confirme le paiement, met à jour l'abonnement et active les services. 8. Les acteurs concernés reçoivent une notification.
Alternatives et exceptions	1. Demande non approuvée : l'Administrateur SaaS enregistre un motif de rejet. 2. Paiement échoué ou en attente : les services restent inchangés. 3. Store ou module indisponible : la sélection doit être corrigée. 4. Demande annulée avant paiement : aucune modification n'est appliquée.

4.2.8. UC-08 — Consulter la marketplace, comparer et simuler

Ce cas d'utilisation décrit le processus « Consulter la marketplace, comparer et simuler ». La figure présente les relations UML et le tableau formalise le scénario fonctionnel correspondant.

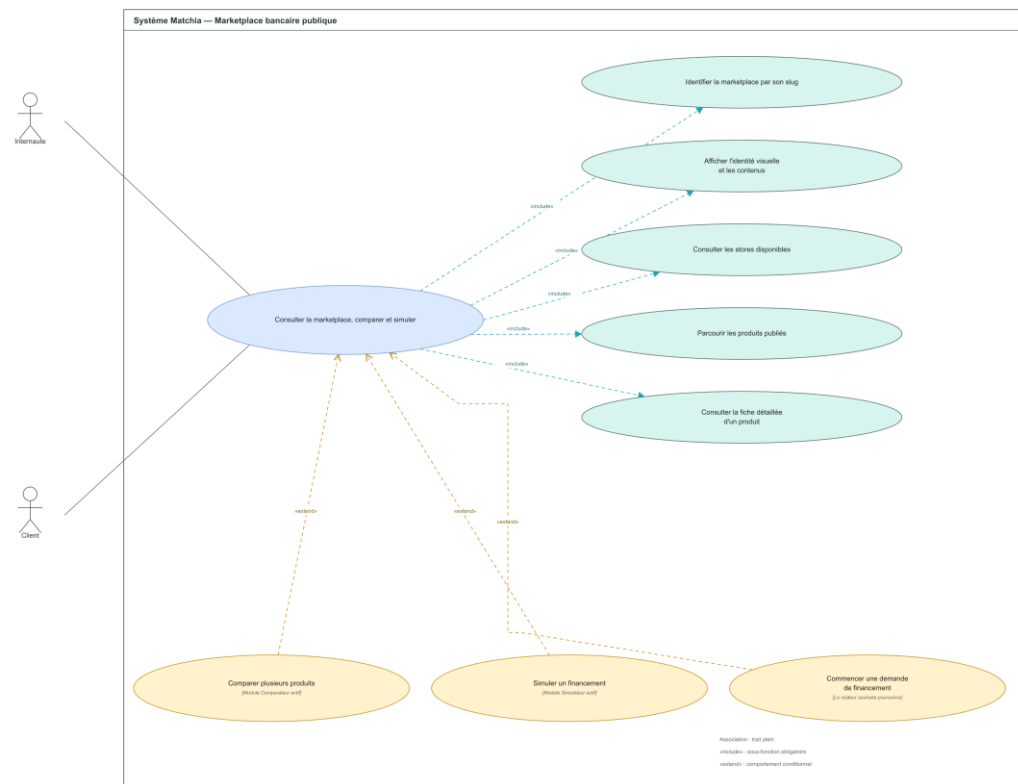


Figure 3.12 — Diagramme de cas d'utilisation détaillé : Consulter la marketplace, comparer et simuler

Tableau 3.8 — Description du scénario UC-08

Élément	Description
Cas d'utilisation	UC-08 — Consulter la marketplace, comparer et simuler
Acteur principal	Internaute ou Client.
Acteurs secondaires	Aucun acteur secondaire direct.
Objectif	Permettre la consultation publique du catalogue d'une banque et proposer la comparaison ou la simulation lorsque les modules correspondants sont actifs.
Préconditions	Le slug identifie une marketplace active disposant d'au moins un store accessible publiquement.
Déclencheur	L'utilisateur accède à l'adresse publique de la marketplace ou ouvre une fiche produit.
Postconditions en cas de succès	Les contenus publics, les produits et, le cas échéant, les résultats de comparaison ou de simulation sont affichés sans exposer de données privées.
Postconditions en cas d'échec	Aucune donnée d'un tenant inexistant, inactif ou non autorisé n'est affichée.
Scénario nominal	1. Le système identifie la marketplace à partir de son slug. 2. Il affiche l'identité visuelle et les contenus publics de la banque. 3. L'utilisateur consulte les stores disponibles et parcourt les produits publiés. 4. Il ouvre la fiche détaillée d'un produit. 5. Si le module Comparateur est actif, il sélectionne plusieurs produits et les compare. 6. Si le module Simulateur est actif, il saisit les paramètres et consulte le résultat de simulation. 7. S'il souhaite poursuivre vers un financement, le système l'oriente vers l'inscription ou l'espace Client.
Alternatives et exceptions	1. Module Comparateur ou Simulateur inactif : l'option correspondante n'est pas proposée. 2. Produit retiré, stock indisponible ou publication suspendue : la fiche n'est plus accessible à la sélection. 3. Slug inconnu ou marketplace inactive : une page d'indisponibilité est affichée.

4.2.9. UC-09 — Déposer une demande d'inscription comme Concessionnaire

Ce cas d'utilisation décrit le processus « Déposer une demande d'inscription comme Concessionnaire ». La figure présente les relations UML et le tableau formalise le scénario fonctionnel correspondant.

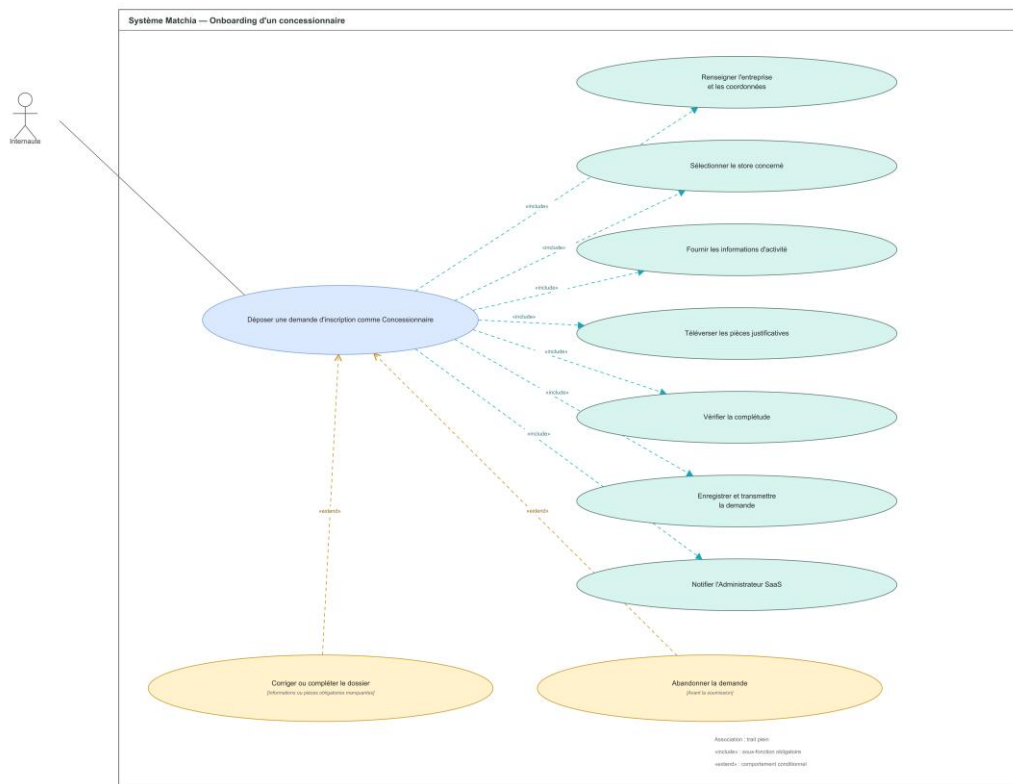


Figure 3.13 — Diagramme de cas d'utilisation détaillé : Déposer une demande d'inscription comme Concessionnaire

Tableau 3.9 — Description du scénario UC-09

Élément	Description
Cas d'utilisation	UC-09 — Déposer une demande d'inscription comme Concessionnaire
Acteur principal	Internaute représentant le concessionnaire.
Acteurs secondaires	Administrateur SaaS.
Objectif	Transmettre un dossier d'inscription complet afin d'obtenir ultérieurement un compte Concessionnaire après validation.
Préconditions	Le formulaire public est accessible et le représentant dispose des informations et pièces justificatives requises.
Déclencheur	L'Internaute démarre une demande d'inscription Concessionnaire.
Postconditions en cas de succès	La demande et ses documents sont enregistrés au statut en attente et l'Administrateur SaaS est notifié.
Postconditions en cas d'échec	Aucune demande incomplète n'est transmise pour validation.
Scénario nominal	1. L'Internaute renseigne l'entreprise et ses coordonnées. 2. Il sélectionne le store correspondant à son activité. 3. Il complète les informations d'activité demandées. 4. Il téléverse les pièces justificatives obligatoires. 5. Le système vérifie la complétude du dossier. 6. L'Internaute confirme et transmet la demande. 7. Le système enregistre le dossier et notifie l'Administrateur SaaS.
Alternatives et exceptions	1. Informations ou pièces manquantes : le dossier doit être corrigé ou complété. 2. Entreprise déjà enregistrée : le système refuse la duplication. 3. Échec du téléversement : le document concerné doit être chargé de nouveau. 4. Abandon avant soumission : aucun dossier définitif n'est transmis.

4.2.10. UC-10 — Traiter une demande d'inscription Concessionnaire

Ce cas d'utilisation décrit le processus « Traiter une demande d'inscription Concessionnaire ». La figure présente les relations UML et le tableau formalise le scénario fonctionnel correspondant.

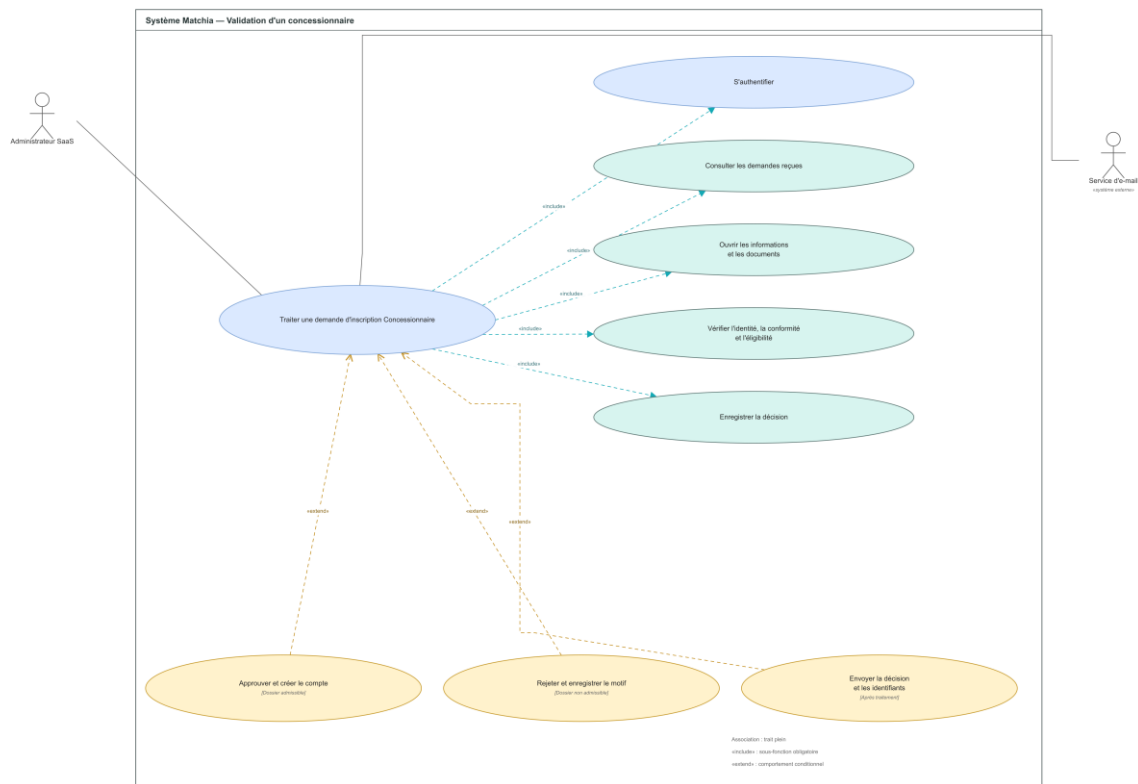


Figure 3.14 — Diagramme de cas d'utilisation détaillé : Traiter une demande d'inscription Concessionnaire

Tableau 3.10 — Description du scénario UC-10

Élément	Description
Cas d'utilisation	UC-10 — Traiter une demande d'inscription Concessionnaire
Acteur principal	Administrateur SaaS.
Acteurs secondaires	Demandeur concessionnaire et service d'e-mail.
Objectif	Contrôler l'identité, la conformité et l'éligibilité du demandeur, puis approuver son inscription ou la rejeter avec un motif.
Préconditions	L'Administrateur SaaS est authentifié et une demande Concessionnaire existe au statut en attente.
Déclencheur	L'Administrateur SaaS ouvre le détail d'une demande d'inscription.
Postconditions en cas de succès	Le dossier est approuvé et un compte Concessionnaire actif est créé, ou la demande est rejetée avec une décision tracée.
Postconditions en cas d'échec	La demande conserve son statut précédent et aucun compte n'est créé.
Scénario nominal	1. L'administrateur consulte les demandes reçues. 2. Il ouvre les informations et les documents du dossier sélectionné. 3. Il vérifie l'identité, la conformité des pièces et l'éligibilité de l'entreprise. 4. Il enregistre sa décision. 5. Si le dossier est admissible, le système crée le compte Concessionnaire. 6. Le système transmet la décision et, en cas d'approbation, les identifiants de première connexion.
Alternatives et exceptions	1. Dossier non admissible : l'administrateur saisit un motif de rejet. 2. Pièce illisible ou information insuffisante : la décision est différée jusqu'à correction. 3. Compte ou entreprise déjà existant : la création du compte est bloquée. 4. Service d'e-mail indisponible : la décision reste enregistrée et l'échec d'envoi est signalé.

4.2.11. UC-11 — Gérer les partenariats et les contrats

Ce cas d'utilisation décrit le processus « Gérer les partenariats et les contrats ». La figure présente les relations UML et le tableau formalise le scénario fonctionnel correspondant.

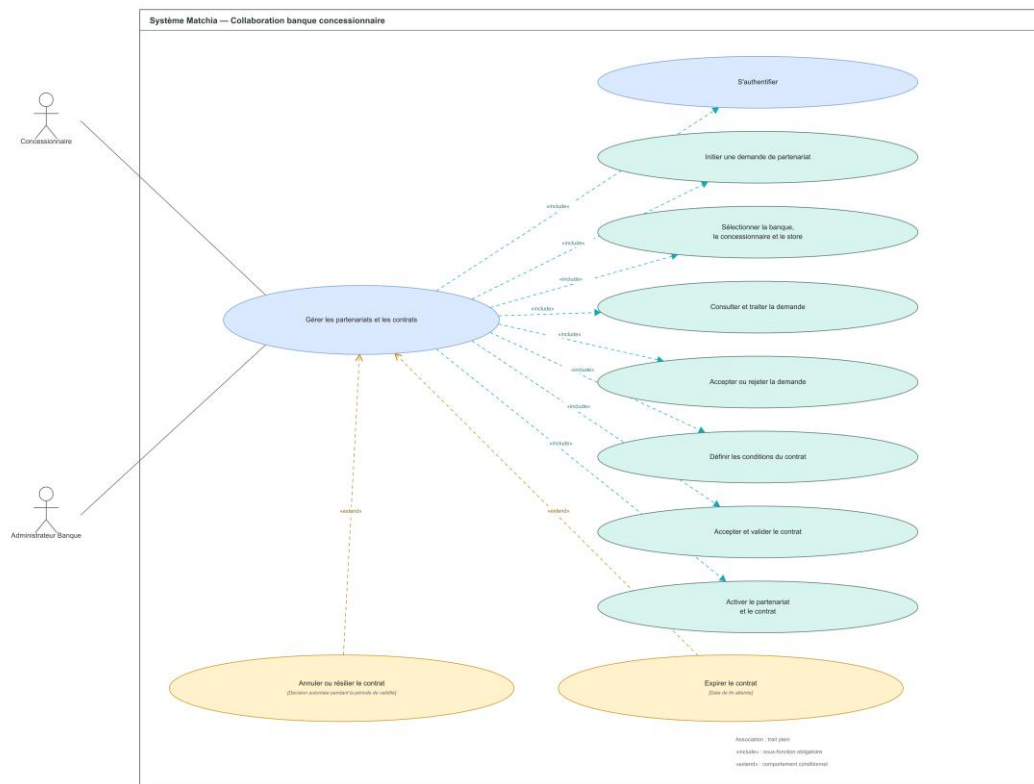


Figure 3.15 — Diagramme de cas d'utilisation détaillé : Gérer les partenariats et les contrats

Tableau 3.11 — Description du scénario UC-11

Élément	Description
Cas d'utilisation	UC-11 — Gérer les partenariats et les contrats
Acteur principal	Concessionnaire.
Acteurs secondaires	Administrateur Banque.
Objectif	Établir et administrer une relation commerciale permettant au concessionnaire de publier ses produits dans la marketplace d'une banque.
Préconditions	Les deux acteurs sont authentifiés. Le concessionnaire est approuvé, la banque est active et le store sélectionné est compatible.
Déclencheur	Le Concessionnaire initie une demande de partenariat ou l'Administrateur Banque ouvre une demande à traiter.
Postconditions en cas de succès	Le partenariat et le contrat sont activés, ou une décision de rejet, d'annulation, de résiliation ou d'expiration est enregistrée.
Postconditions en cas d'échec	Aucune publication n'est autorisée en l'absence d'un partenariat et d'un contrat valides.
Scénario nominal	1. Le Concessionnaire s'authentifie et initie une demande de partenariat. 2. Il sélectionne la banque et le store concernés. 3. L'Administrateur Banque consulte et traite la demande. 4. La banque accepte la demande. 5. Les conditions du contrat sont définies et le contrat est transmis au Concessionnaire. 6. Le Concessionnaire accepte le contrat. 7. Le système active le partenariat et le contrat.
Alternatives et exceptions	1. Demande rejetée : le motif est enregistré et la relation reste inactive. 2. Contrat refusé ou annulé : le partenariat ne permet pas la publication. 3. Résiliation autorisée : le contrat et les droits associés sont désactivés. 4. Date de fin atteinte : le contrat expire automatiquement.

4.2.12. UC-12 — Gérer les produits, les stocks et les publications

Ce cas d'utilisation décrit le processus « Gérer les produits, les stocks et les publications ». La figure présente les relations UML et le tableau formalise le scénario fonctionnel correspondant.

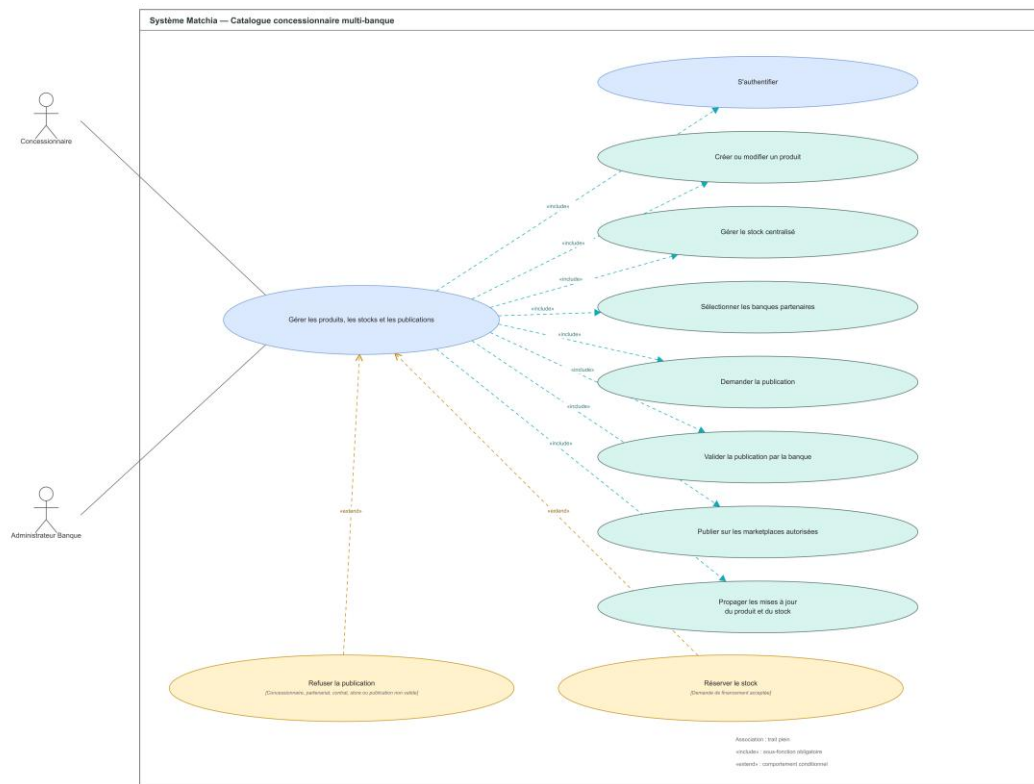


Figure 3.16 — Diagramme de cas d'utilisation détaillé : Gérer les produits, les stocks et les publications

Tableau 3.12 — Description du scénario UC-12

Élément	Description
Cas d'utilisation	UC-12 — Gérer les produits, les stocks et les publications
Acteur principal	Concessionnaire.
Acteurs secondaires	Administrateur Banque.
Objectif	Centraliser le catalogue et le stock du concessionnaire, puis publier les produits dans plusieurs marketplaces après validation de chaque banque.
Préconditions	Le Concessionnaire est authentifié et actif. Pour chaque banque ciblée, un partenariat et un contrat valides existent.
Déclencheur	Le Concessionnaire crée ou modifie un produit, met à jour son stock ou demande une publication.
Postconditions en cas de succès	Le produit et le stock sont mis à jour. Chaque publication approuvée devient visible dans la marketplace autorisée.
Postconditions en cas d'échec	Une publication non conforme reste invisible et aucune donnée d'une banque non autorisée n'est modifiée.
Scénario nominal	1. Le Concessionnaire s'authentifie et crée ou modifie la fiche produit. 2. Il renseigne le stock centralisé et les documents nécessaires. 3. Il sélectionne les banques partenaires visées. 4. Il soumet une demande de publication pour chaque banque. 5. L'Administrateur Banque examine et valide la publication. 6. Le système publie le produit dans les marketplaces autorisées. 7. Les mises à jour ultérieures du produit et du stock sont propagées aux publications actives.
Alternatives et exceptions	1. Publication refusée : le motif est enregistré et le produit reste non visible dans la banque concernée. 2. Partenariat, contrat, store ou produit invalide : la soumission est bloquée. 3. Demande de financement acceptée pour un produit concessionnaire : le stock correspondant est réservé. 4. Stock insuffisant : la réservation ou la disponibilité publique est ajustée.

4.2.13. UC-13 — S'inscrire comme Client

Ce cas d'utilisation décrit le processus « S'inscrire comme Client ». La figure présente les relations UML et le tableau formalise le scénario fonctionnel correspondant.

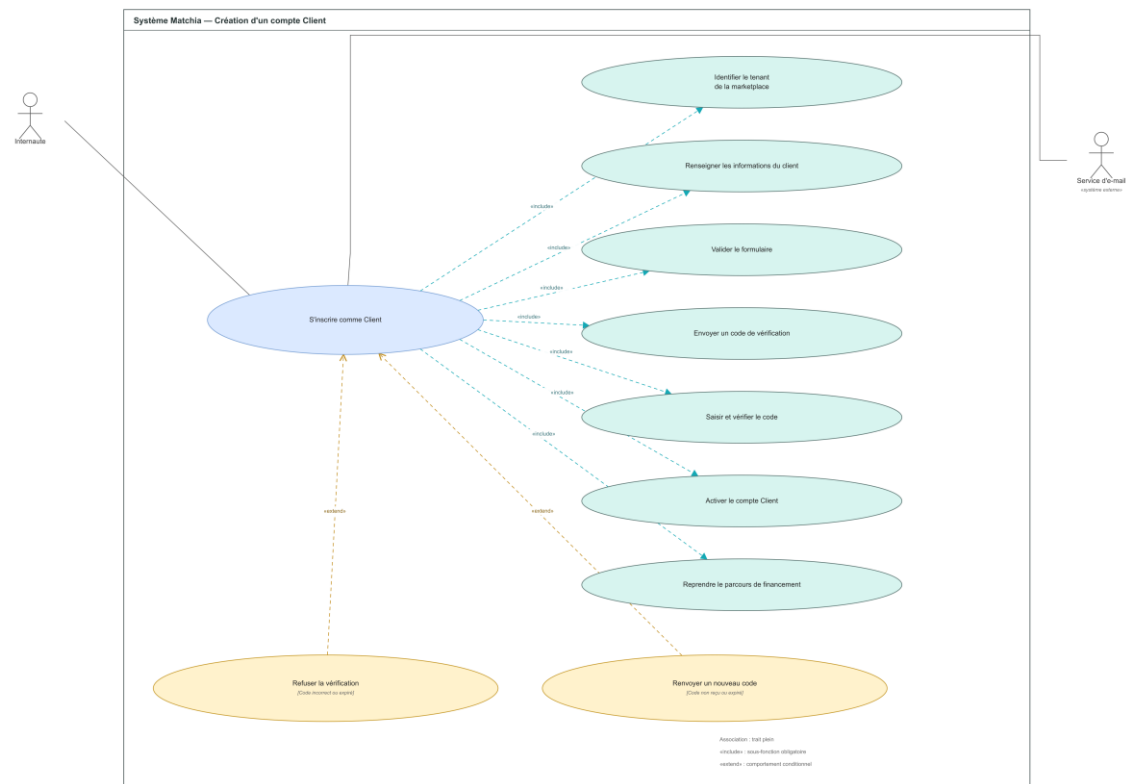


Figure 3.17 — Diagramme de cas d'utilisation détaillé : S'inscrire comme Client

Tableau 3.13 — Description du scénario UC-13

Élément	Description
Cas d'utilisation	UC-13 — S'inscrire comme Client
Acteur principal	Internaute.
Acteurs secondaires	Service d'e-mail.
Objectif	Créer et vérifier un compte Client rattaché à la marketplace bancaire depuis laquelle l'inscription est lancée.
Préconditions	La marketplace est active, l'inscription Client est disponible et l'adresse e-mail fournie n'est pas déjà utilisée dans le contexte concerné.
Déclencheur	L'Internaute sélectionne l'inscription Client ou poursuit un parcours de financement nécessitant un compte.
Postconditions en cas de succès	Le compte Client est vérifié, activé et rattaché au tenant de la marketplace. Le parcours initial peut être repris.
Postconditions en cas d'échec	Aucun compte actif n'est créé tant que l'adresse e-mail n'est pas vérifiée.
Scénario nominal	1. Le système identifie le tenant à partir de la marketplace courante. 2. L'Internaute renseigne ses informations personnelles et son adresse e-mail. 3. Le système valide le formulaire et envoie un code de vérification. 4. L'Internaute saisit le code reçu. 5. Le système vérifie le code et active le compte Client. 6. Le Client est redirigé vers son espace ou vers la reprise du parcours de financement.
Alternatives et exceptions	1. Code incorrect ou expiré : la vérification est refusée. 2. Code non reçu ou expiré : un nouveau code peut être envoyé. 3. Adresse e-mail déjà utilisée : le système propose la connexion plutôt qu'une nouvelle inscription. 4. Tenant ou marketplace inactif : l'inscription est indisponible.

4.2.14. UC-14 — Constituer et soumettre une demande de financement

Ce cas d'utilisation décrit le processus « Constituer et soumettre une demande de financement ». La figure présente les relations UML et le tableau formalise le scénario fonctionnel correspondant.

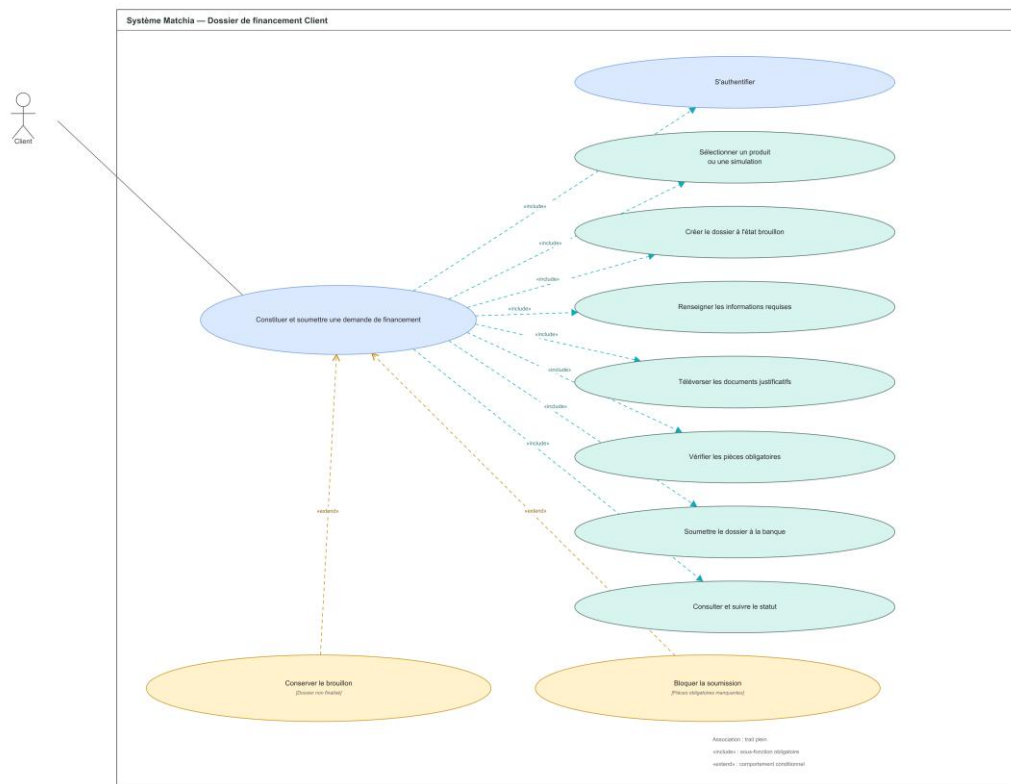


Figure 3.18 — Diagramme de cas d'utilisation détaillé : Constituer et soumettre une demande de financement

Tableau 3.14 — Description du scénario UC-14

Élément	Description
Cas d'utilisation	UC-14 — Constituer et soumettre une demande de financement
Acteur principal	Client.
Acteurs secondaires	Administrateur Banque, destinataire du dossier.
Objectif	Permettre au Client de préparer un dossier complet, de joindre les pièces requises et de le soumettre à la banque concernée.
Préconditions	Le Client est authentifié. La marketplace, le store et le produit sélectionné sont actifs et les exigences documentaires sont disponibles.
Déclencheur	Le Client choisit de poursuivre un produit ou une simulation par une demande de financement.
Postconditions en cas de succès	Le dossier est soumis à la banque avec le statut en attente et reste consultable par le Client pour le suivi.
Postconditions en cas d'échec	Le dossier reste en brouillon et aucune instruction bancaire ne commence.
Scénario nominal	1. Le Client s'authentifie et sélectionne un produit ou une simulation. 2. Le système crée un dossier à l'état brouillon. 3. Le Client renseigne les informations financières et personnelles requises. 4. Il téléverse les documents justificatifs demandés. 5. Le système vérifie la présence des pièces obligatoires. 6. Le Client confirme et soumet le dossier à la banque. 7. Le système place le dossier au statut en attente et le rend accessible au suivi.
Alternatives et exceptions	1. Dossier non finalisé : il est conservé en brouillon. 2. Pièces obligatoires manquantes : la soumission est bloquée. 3. Produit retiré ou store inactif avant soumission : le parcours ne peut pas être finalisé. 4. Le Client peut corriger les informations tant que le dossier n'est pas soumis.

4.2.15. UC-15 — Traiter une demande de financement

Ce cas d'utilisation décrit le processus « Traiter une demande de financement ». La figure présente les relations UML et le tableau formalise le scénario fonctionnel correspondant.

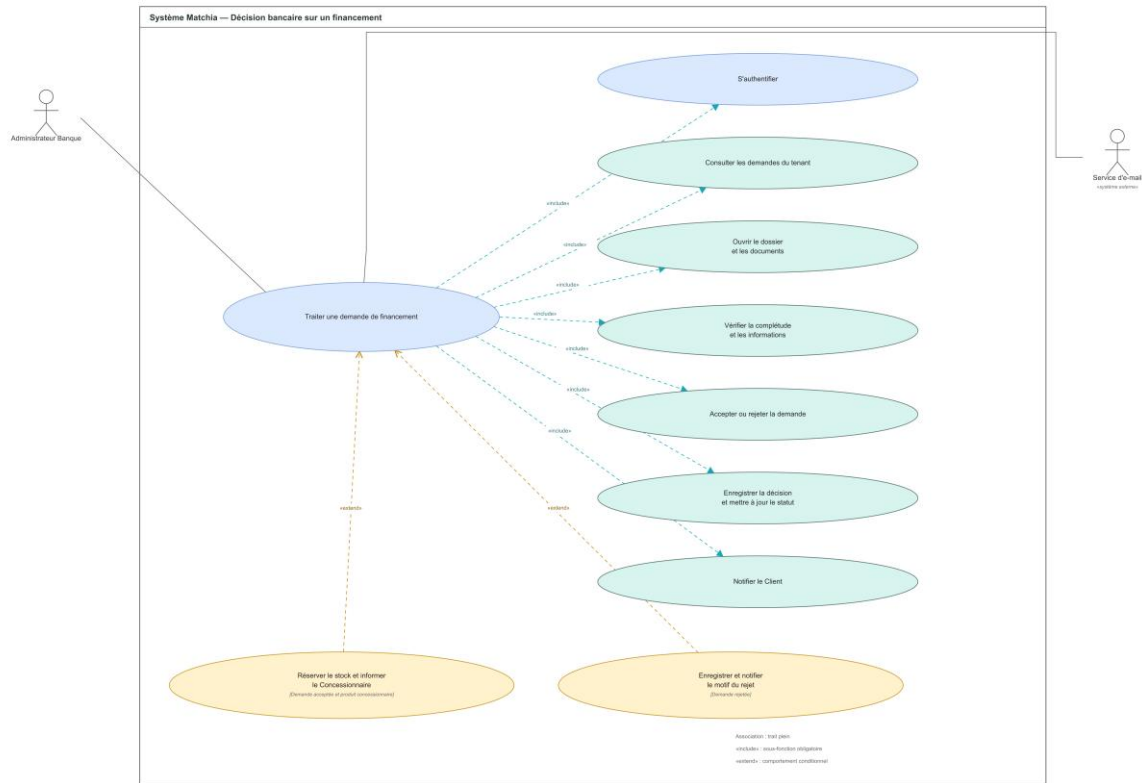


Figure 3.19 — Diagramme de cas d'utilisation détaillé : Traiter une demande de financement

Tableau 3.15 — Description du scénario UC-15

Élément	Description
Cas d'utilisation	UC-15 — Traiter une demande de financement
Acteur principal	Administrateur Banque.
Acteurs secondaires	Client, Concessionnaire et service d'e-mail.
Objectif	Instruire une demande relevant du tenant bancaire, enregistrer la décision et informer les acteurs concernés.
Préconditions	L'Administrateur Banque est authentifié dans son tenant et une demande existe au statut en attente.
Déclencheur	L'Administrateur Banque ouvre une demande de financement depuis la liste de son tenant.
Postconditions en cas de succès	Le dossier est accepté ou rejeté, la décision est persistée et le Client est notifié. Le stock est réservé lorsque les conditions sont réunies.
Postconditions en cas d'échec	Le statut du dossier et le stock restent inchangés.
Scénario nominal	1. L'Administrateur Banque consulte les demandes de son tenant. 2. Il ouvre le dossier et les documents associés. 3. Il vérifie la complétude et les informations fournies. 4. Il accepte ou rejette la demande. 5. Le système enregistre la décision et met à jour le statut. 6. Le service d'e-mail notifie le Client. 7. Si la demande acceptée concerne un produit concessionnaire, le système réserve le stock et informe le Concessionnaire.
Alternatives et exceptions	1. Demande rejetée : la saisie du motif est obligatoire avant validation. 2. Produit concessionnaire et stock insuffisant : la réservation échoue et l'acceptation doit être réévaluée. 3. Dossier appartenant à un autre tenant : l'accès est refusé. 4. Statut déjà modifié : le système empêche une décision concurrente.

4.2.16. UC-16 — Interroger l'assistant IA du SaaS

Ce cas d'utilisation décrit le processus « Interroger l'assistant IA du SaaS ». La figure présente les relations UML et le tableau formalise le scénario fonctionnel correspondant.

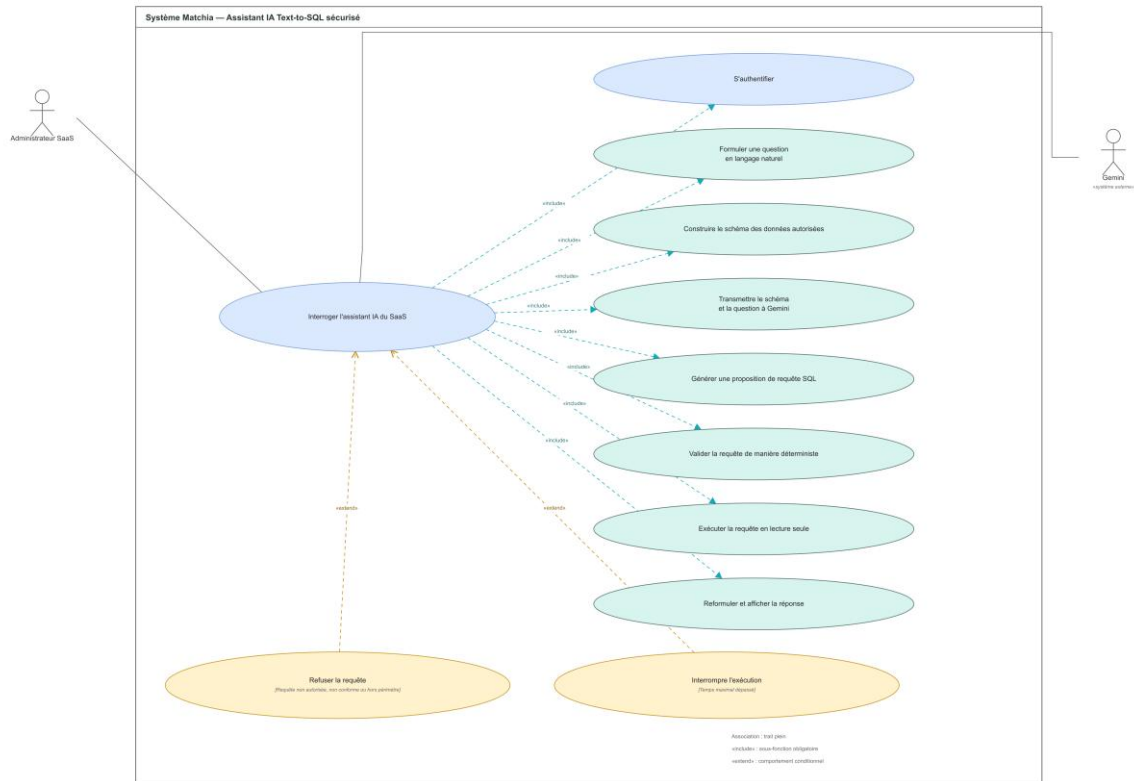


Figure 3.20 — Diagramme de cas d'utilisation détaillé : Interroger l'assistant IA du SaaS

Tableau 3.16 — Description du scénario UC-16

Élément	Description
Cas d'utilisation	UC-16 — Interroger l'assistant IA du SaaS
Acteur principal	Administrateur SaaS.
Acteurs secondaires	Gemini.
Objectif	Obtenir une réponse analytique à partir d'une question en langage naturel, au moyen d'une requête SQL validée et exécutée uniquement en lecture.
Préconditions	L'Administrateur SaaS est authentifié, l'assistant est disponible et le schéma de données autorisé peut être construit.
Déclencheur	L'Administrateur SaaS formule une question dans l'interface de l'assistant.
Postconditions en cas de succès	Une réponse métier est affichée à partir d'une requête autorisée, sans modification des données ni exposition hors périmètre.
Postconditions en cas d'échec	Aucune requête non autorisée n'est exécutée et aucune donnée protégée n'est divulguée.
Scénario nominal	1. L'Administrateur SaaS s'authentifie et formule une question en langage naturel. 2. Le système construit le schéma limité aux données autorisées. 3. Il transmet la question et le schéma à Gemini. 4. Gemini génère une proposition de requête SQL. 5. Le système valide la requête de manière déterministe. 6. La requête autorisée est exécutée en lecture seule avec une durée limitée. 7. Le système reformule le résultat et affiche la réponse.
Alternatives et exceptions	1. Requête non autorisée, non conforme ou hors périmètre : elle est refusée avant exécution. 2. Temps maximal dépassé : l'exécution est interrompue. 3. Gemini indisponible ou réponse inexploitable : un message d'erreur est affiché. 4. Aucun résultat : le système l'indique sans inventer de données.

4.2.17. UC-17 — Utiliser le chatbot public

Ce cas d'utilisation décrit le processus « Utiliser le chatbot public ». La figure présente les relations UML et le tableau formalise le scénario fonctionnel correspondant.

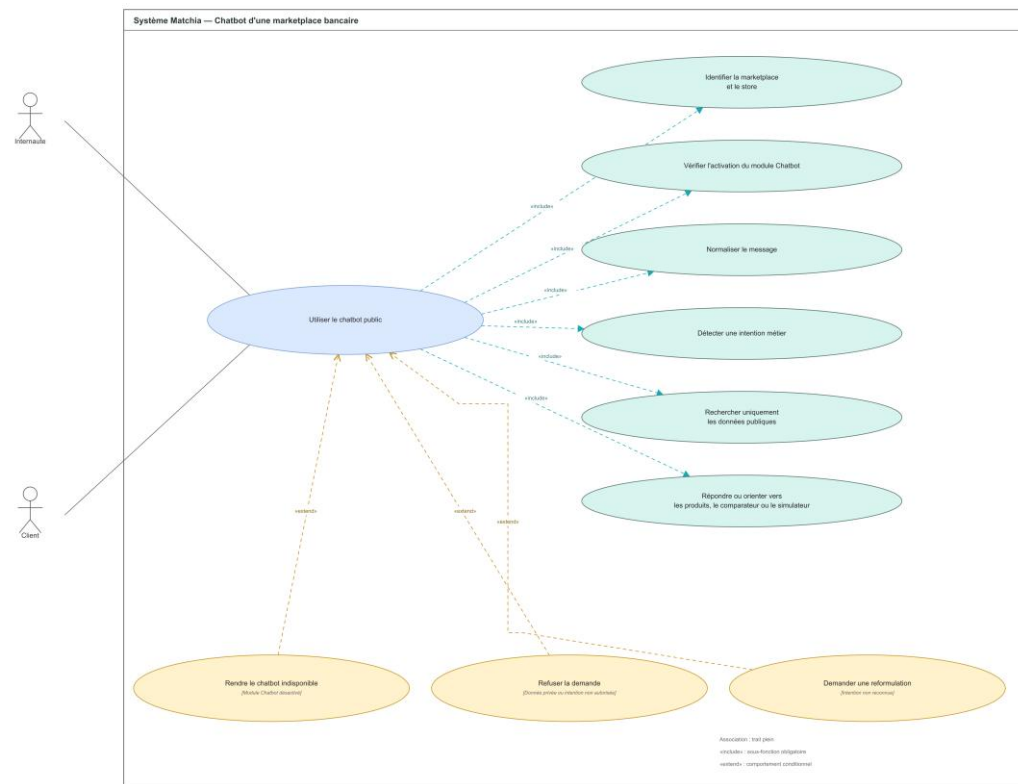


Figure 3.21 — Diagramme de cas d'utilisation détaillé : Utiliser le chatbot public

Tableau 3.17 — Description du scénario UC-17

Élément	Description
Cas d'utilisation	UC-17 — Utiliser le chatbot public
Acteur principal	Internaute ou Client.
Acteurs secondaires	Aucun acteur secondaire direct.
Objectif	Orienter l'utilisateur dans une marketplace bancaire à partir des seules données publiques du tenant et des modules disponibles.
Préconditions	La marketplace et le store sont identifiables et le module Chatbot est actif.
Déclencheur	L'utilisateur envoie un message dans l'interface du chatbot public.
Postconditions en cas de succès	Une réponse publique et contextualisée est affichée, ou l'utilisateur est orienté vers les produits, le comparateur ou le simulateur.
Postconditions en cas d'échec	Aucune donnée privée, non autorisée ou appartenant à un autre tenant n'est révélée.
Scénario nominal	1. Le système identifie la marketplace et le store courants. 2. Il vérifie que le module Chatbot est actif. 3. L'utilisateur saisit son message. 4. Le système normalise le message et détecte l'intention métier. 5. Il recherche uniquement dans les données publiques du tenant. 6. Il affiche la réponse ou oriente l'utilisateur vers un produit, le comparateur ou le simulateur.
Alternatives et exceptions	1. Module Chatbot désactivé : l'interface est rendue indisponible. 2. Demande portant sur une donnée privée ou une intention interdite : le chatbot refuse la demande. 3. Intention non reconnue : le chatbot demande une reformulation. 4. Aucun résultat public pertinent : le chatbot l'indique et propose une orientation générale.

Chapitre 4 : Mise en œuvre du cycle de vie des marketplaces bancaires : onboarding, configuration, abonnement et activation

1. Introduction

Ce chapitre est consacré à la réalisation de la plateforme **Matchia** et à la mise en œuvre de ses principales fonctionnalités. Il présente le parcours complet de création d'une marketplace bancaire, depuis la soumission de la demande jusqu'à son activation et sa configuration par l'Administrateur Banque. Les différents espaces et workflows sont décrits afin de mettre en évidence le fonctionnement concret de la solution et les interactions entre ses principaux acteurs.

2. Authentification, autorisation et contexte multi-tenant

Avant de présenter les différents parcours fonctionnels, il est nécessaire d'introduire les mécanismes qui assurent la sécurisation des accès et la gestion du contexte multi-tenant.

L'accès aux espaces privées repose sur un système d'authentification permettant notamment la connexion, la déconnexion, le renouvellement de la session ainsi que la réinitialisation du mot de passe. Une fois authentifié, l'utilisateur est dirigé vers l'espace correspondant à son rôle.

La plateforme distingue principalement quatre rôles : Administrateur SaaS, Administrateur Banque, Concessionnaire et Client. Chaque rôle dispose d'un périmètre fonctionnel spécifique et ne peut accéder qu'aux fonctionnalités qui lui sont autorisées.

L'Administrateur SaaS dispose d'une vision globale de la plateforme et en assure la supervision générale. L'Administrateur Banque gère uniquement les ressources et services liés à sa propre marketplace. Les Concessionnaires accèdent aux fonctionnalités associées à leurs partenariats bancaires, tandis que les Clients utilisent les services proposés par la marketplace correspondant à leur banque.

Dans le cadre de l'architecture multi-tenant, les utilisateurs de chaque banque sont également associés au contexte de leur établissement. Ce mécanisme permet à plusieurs banques d'utiliser

le même socle applicatif tout en conservant leurs propres utilisateurs, contenus, produits, configurations, stores et modules.

Ces mécanismes constituent la base de sécurité et d'isolation sur laquelle reposent les différents workflows présentés dans la suite de ce chapitre.

Le diagramme d'activité suivant synthétise le contrôle des identifiants et de l'état du compte, le chargement du rôle et des autorisations ainsi que la détermination du contexte du tenant lorsque l'utilisateur est rattaché à une banque ou à une marketplace.

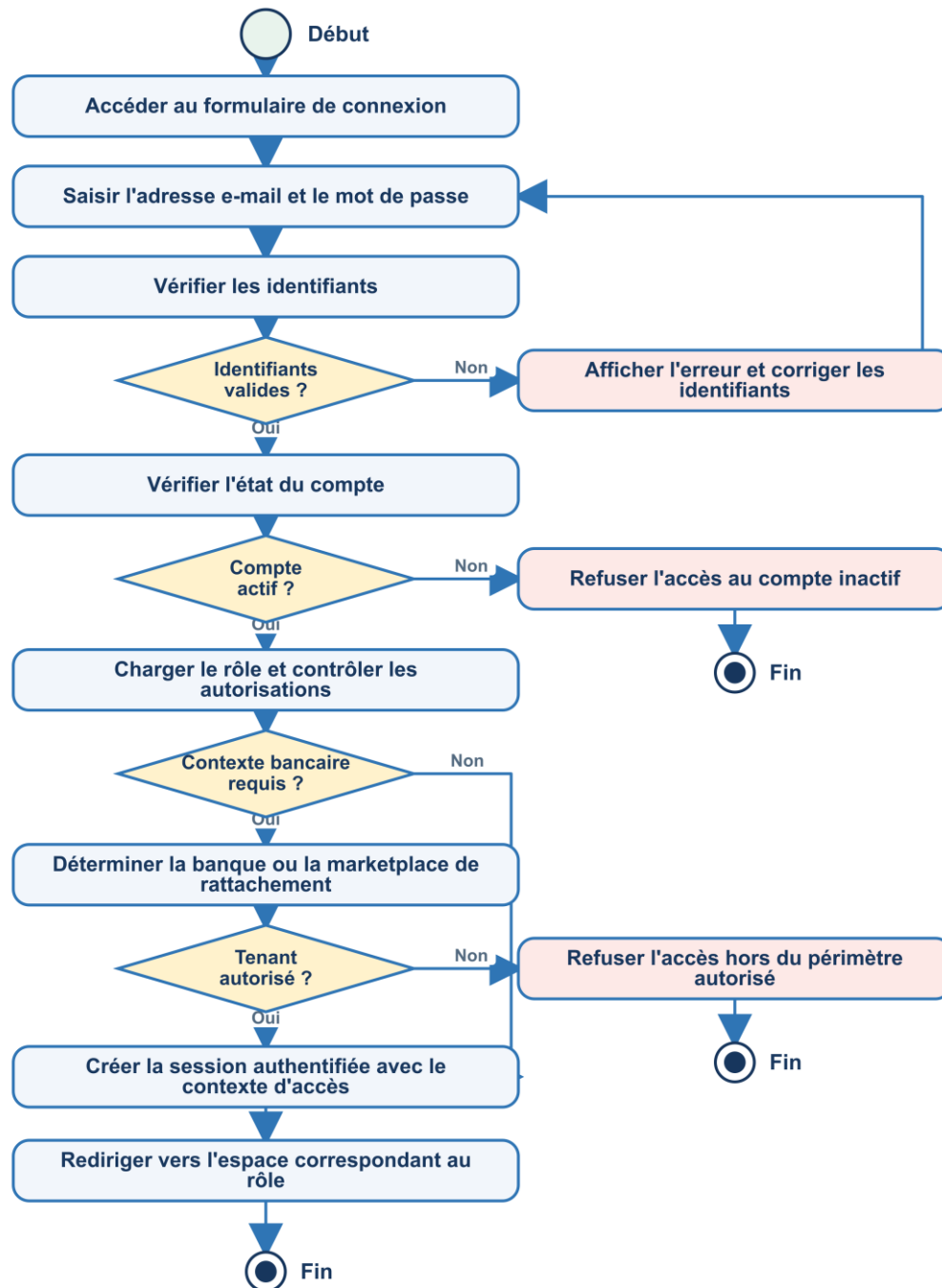


Figure 4.1 — Diagramme d'activité de l'authentification et du contexte d'accès

Le diagramme de séquence suivant détaille les échanges entre l'interface, le service d'authentification, la génération des jetons et la base de données. Il met en évidence le refus des comptes invalides ainsi que la résolution conditionnelle du contexte tenant.

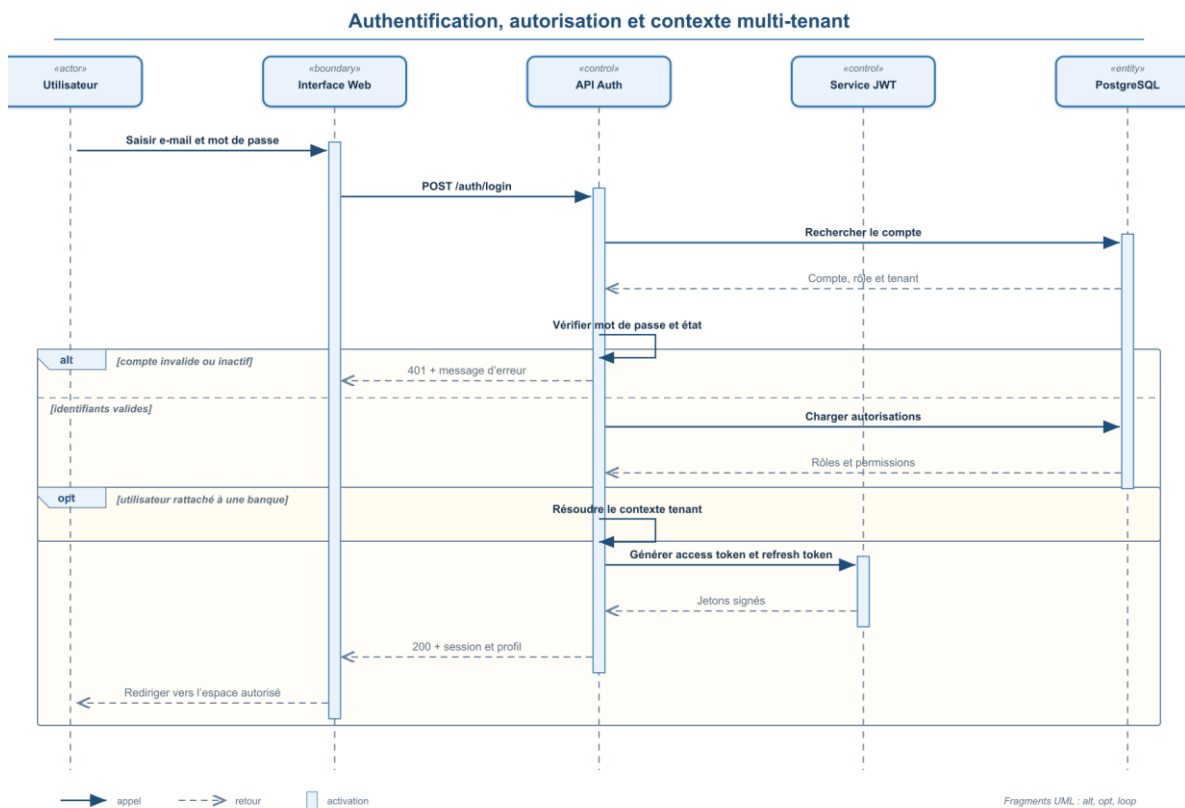


Figure 4.2 — Diagramme de séquence de l'authentification et du contexte multi-tenant

3. Demande de création d'une marketplace bancaire

La première étape du processus consiste pour une banque à soumettre une demande d'adhésion à Matchia. Cette démarche est accessible depuis l'espace public de la plateforme et permet au demandeur de renseigner progressivement les informations nécessaires à la création de sa future marketplace.

Capture accueil public

Afin de faciliter la saisie et de limiter les demandes incomplètes, le formulaire est organisé en plusieurs étapes successives. Le parcours comprend les informations de la banque, les coordonnées du futur administrateur, la configuration initiale de la marketplace, la sélection des stores et modules, puis la validation finale de la demande.

Le processus général peut être représenté par le diagramme d'activité suivant :

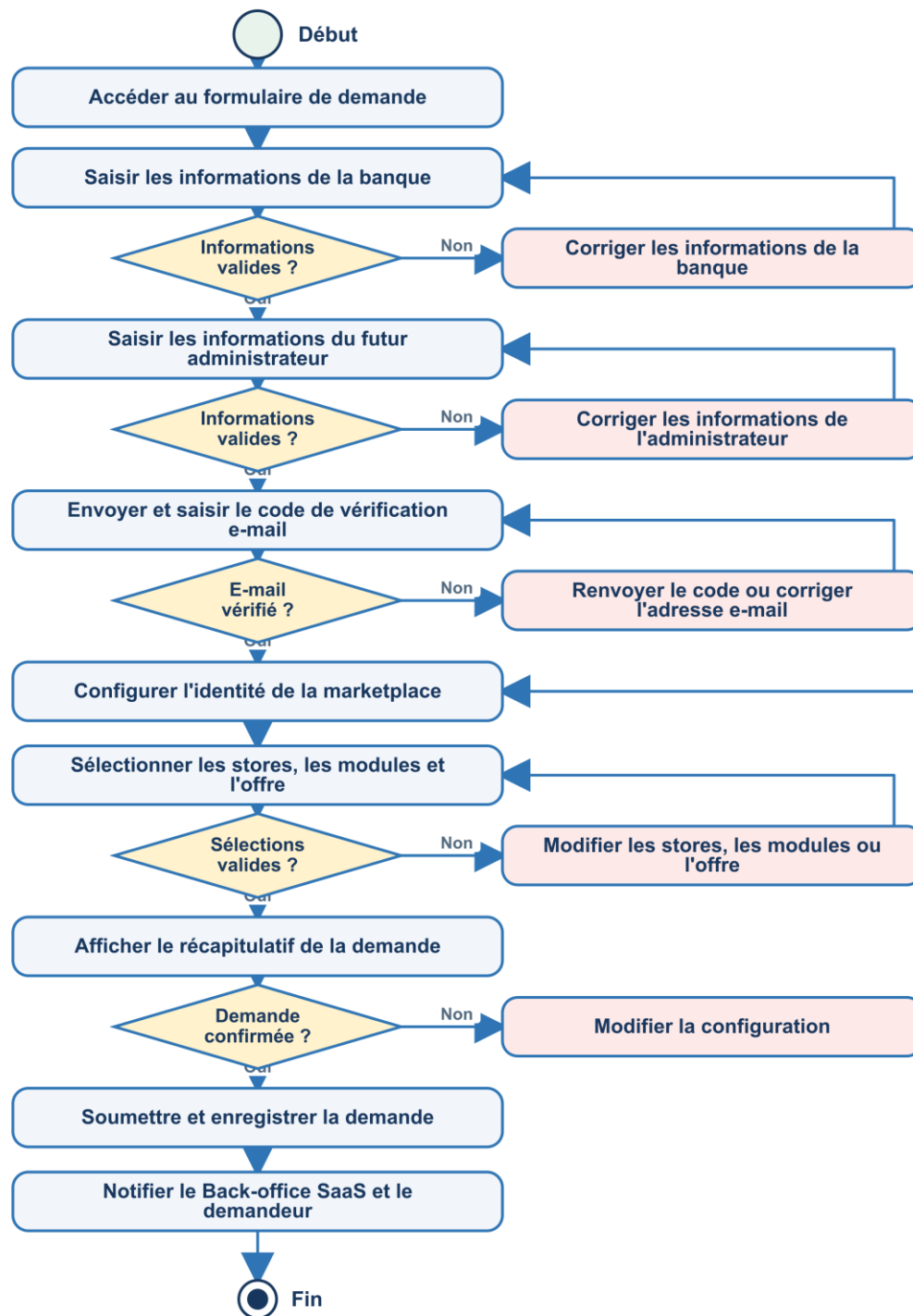


Figure 4.3 — Diagramme d'activité de création d'une demande de marketplace bancaire

Le diagramme de séquence complète cette vue en montrant la vérification de l'adresse e-mail, la validation de la configuration, l'enregistrement de la demande et l'envoi des notifications.

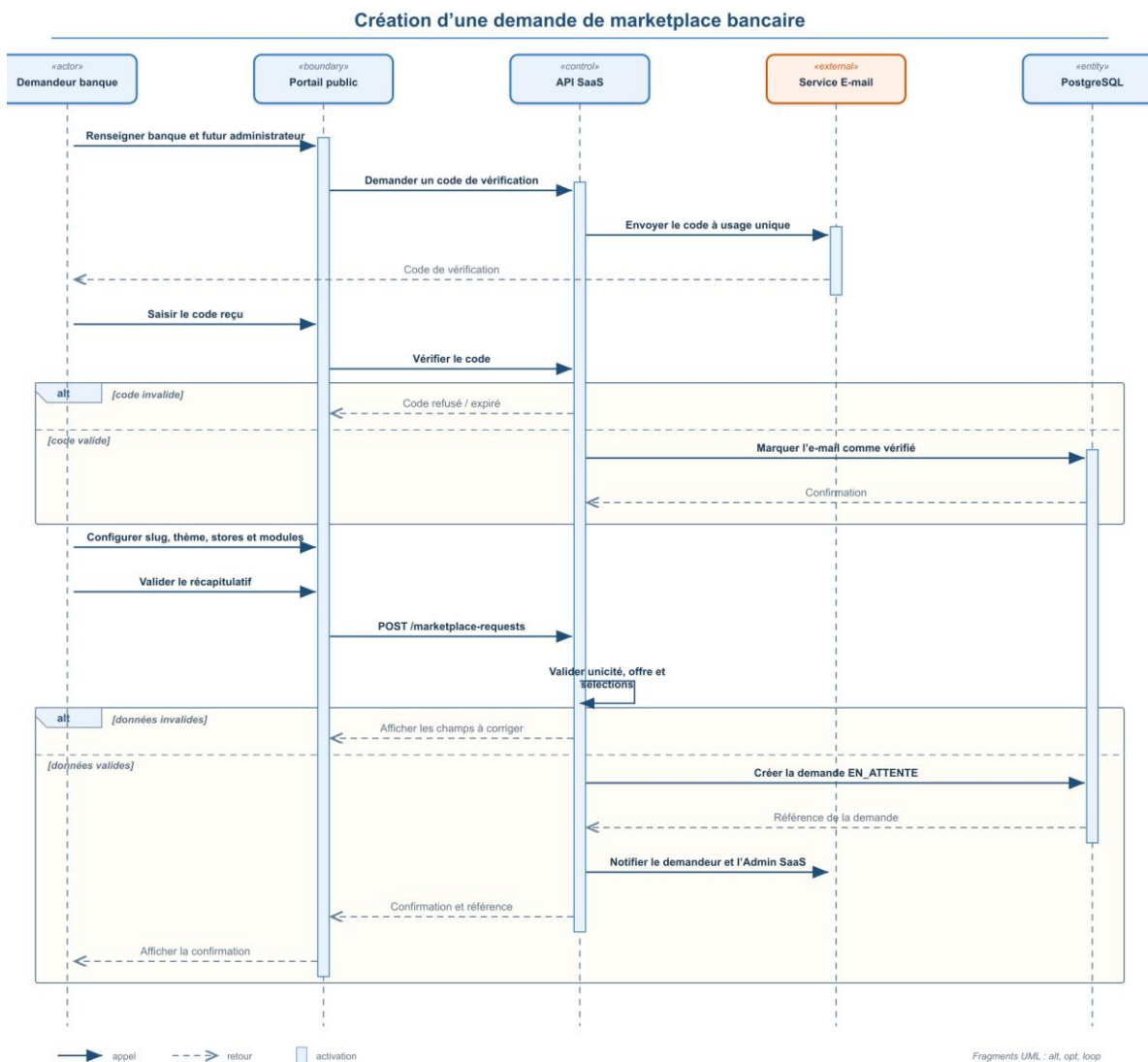


Figure 4.4 — Diagramme de séquence de la création d'une demande de marketplace bancaire

3.1. Renseignement des informations de la banque

La première étape permet de recueillir les informations nécessaires à l'identification de l'établissement bancaire. Le demandeur renseigne notamment le nom de la banque, son adresse électronique, son numéro de téléphone, son site web, une description ainsi que certaines informations complémentaires relatives à l'établissement.

Figure 4.5 — Saisie des informations de la banque

La figure présente le premier écran du formulaire avec les différentes informations demandées ainsi que la zone de téléversement du logo.

3.2. Informations du futur Administrateur Banque et vérification de l'adresse e-mail

La deuxième étape est consacrée au futur administrateur de la marketplace. Le formulaire recueille notamment son nom, son prénom, son adresse électronique, son numéro de téléphone ainsi que son image de profil.

Une vérification de l'adresse électronique est intégrée à ce parcours. Après validation des informations saisies, un **code de vérification** est envoyé à l'adresse indiquée. Le demandeur doit saisir ce code dans le formulaire afin de confirmer que l'adresse lui appartient réellement.

Cette vérification joue un rôle important, car l'adresse validée sera ensuite utilisée dans le processus de communication avec la banque, notamment pour l'envoi des informations relatives au traitement de la demande, au paiement et à l'accès au Back-office Banque.

Figure 4.6 — Vérification de l'adresse électronique du futur Administrateur Banque

La figure doit présenter l'interface de saisie du code ainsi que la confirmation de la vérification de l'adresse électronique. Les informations personnelles utilisées dans les captures doivent être anonymisées.

3.3. Configuration initiale de la marketplace, sélection des stores et modules

Cette étape permet notamment de définir certains éléments propres à l'identité de la marketplace, notamment son identifiant d'accès (*slug*), ses couleurs de personnalisation ainsi que la bannière affichée sur la page d'accueil.

La banque sélectionne ensuite les stores qu'elle souhaite intégrer à sa future marketplace. Chaque store représente un univers fonctionnel dans lequel pourront être proposés des produits et services spécifiques.

Une fois un store sélectionné, la plateforme affiche les modules qui lui sont associés. La banque peut alors choisir les fonctionnalités adaptées à ses besoins et à son offre.

Figure 4.7 — Sélection des stores de la marketplace

La figure présente les stores disponibles et la sélection effectuée par la banque.

3.4. Récapitulatif et soumission de la demande

La dernière étape du formulaire affiche un récapitulatif permettant au demandeur de vérifier l'ensemble des données renseignées avant la validation définitive de sa demande. Cette synthèse regroupe les informations de la banque, les coordonnées du futur administrateur Banque, les éléments de configuration de la marketplace, les stores sélectionnés, les modules associés à chaque store ainsi que les informations relatives à l'offre choisie.

Après la soumission, une notification est créée à l'intention de l'Administrateur SaaS afin de signaler l'arrivée d'une nouvelle demande. Parallèlement, un e-mail de confirmation est envoyé au contact de la banque.

Figure 4.10 — Confirmation de la soumission de la demande

Ces figures permettent respectivement de présenter la synthèse du formulaire et le message confirmant l'enregistrement de la demande.

4. Back-office SaaS : administration et supervision de la plateforme

Après la soumission d'une demande, son traitement est assuré par l'Administrateur SaaS. Le Back-office SaaS constitue l'espace central de gouvernance de la plateforme. Il regroupe les fonctionnalités nécessaires au pilotage des ressources partagées, au suivi des demandes et au contrôle du cycle de vie des services proposés aux banques.

4.1. Tableau de bord

Le tableau de bord fournit une vue synthétique de l'activité de la plateforme à travers des indicateurs relatifs notamment aux banques, aux demandes reçues, aux abonnements et aux

principales opérations à suivre. Il permet ainsi à l'Administrateur SaaS d'identifier rapidement les éléments nécessitant son intervention.

Figure 4.11 — Tableau de bord de l'Administrateur SaaS

4.2. Gestion des stores et des modules

Cette rubrique permet d'administrer le catalogue des stores et des modules proposés aux banques. L'Administrateur SaaS peut consulter les univers fonctionnels disponibles, organiser les modules associés et définir les éléments qui pourront être sélectionnés lors de la création ou de l'évolution d'une marketplace. Cette gestion centralisée garantit la cohérence des services mis à disposition des différents tenants.

4.3. Gestion des demandes de marketplace

La rubrique Demandes centralise les sollicitations reçues pour la création ou l'évolution des services d'une marketplace. Elle facilite leur consultation, leur analyse et leur traitement dans un cadre traçable.

4.3.1. Traitement d'une demande de marketplace

L'Administrateur SaaS dispose d'une interface permettant de consulter les demandes reçues et d'accéder à leur détail. La fiche d'une demande présente les informations de la banque, celles du futur administrateur, la configuration souhaitée ainsi que les stores et modules sélectionnés.

Après vérification, l'administrateur peut approuver ou rejeter la demande. En cas de rejet, il renseigne le motif de la décision ; un e-mail précisant ce motif est ensuite transmis au demandeur. En cas d'approbation, le processus se poursuit vers l'étape de paiement.

Figure 4.12 — Liste des demandes de marketplace dans le Back-office SaaS

Figure 4.13 — Consultation du détail d'une demande

- Le processus décisionnel est également présenté dans le diagramme d'activité suivant.

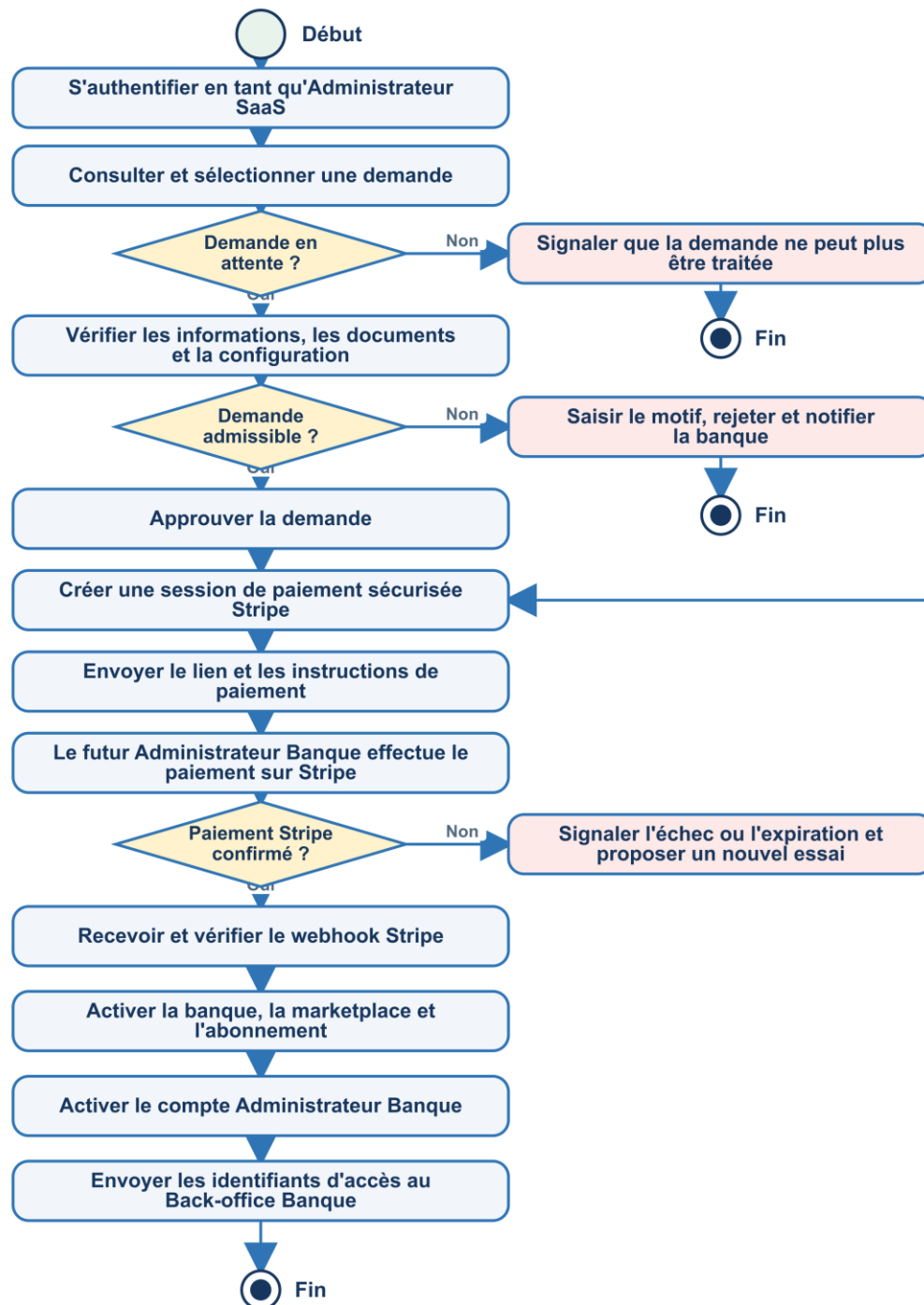


Figure 4.14 — Diagramme d'activité du traitement d'une demande et du paiement Stripe

4.3.2. Approbation et déclenchement du paiement

Lorsqu'une demande est approuvée, la plateforme prépare les éléments nécessaires à la future activation de la banque et de sa marketplace. Ces éléments restent toutefois inactifs tant que le paiement n'a pas été confirmé. À cette étape, un e-mail contenant les instructions de paiement et le lien correspondant est envoyé au contact de la banque.

4.3.3. Paiement sécurisé par Stripe et activation de la marketplace

- Le paiement est une étape déterminante du cycle de vie de la demande. Il est intégré à la plateforme au moyen de Stripe, service de paiement en ligne. Lorsqu'il ouvre le lien reçu, le futur Administrateur Banque accède à une interface de paiement sécurisée dans laquelle il renseigne les informations nécessaires à la transaction. Les données de carte sont traitées par les composants sécurisés de Stripe et ne sont pas enregistrées par l'application.
- Le backend crée l'opération de paiement et l'associe à la demande ainsi qu'à l'abonnement concerné. Après la confirmation de la transaction, il vérifie le statut retourné par Stripe avant de mettre à jour le paiement. Seule une confirmation de paiement réussie déclenche l'activation de la banque, de sa marketplace, de l'abonnement et du compte Administrateur Banque. En cas d'échec ou de paiement en attente, ces ressources restent inactives.
- La plateforme génère ensuite les informations nécessaires à la première connexion et envoie au responsable de la banque un e-mail contenant ses identifiants d'accès au Back-office Banque.

Le workflow complet est représenté par le diagramme de séquence suivant.

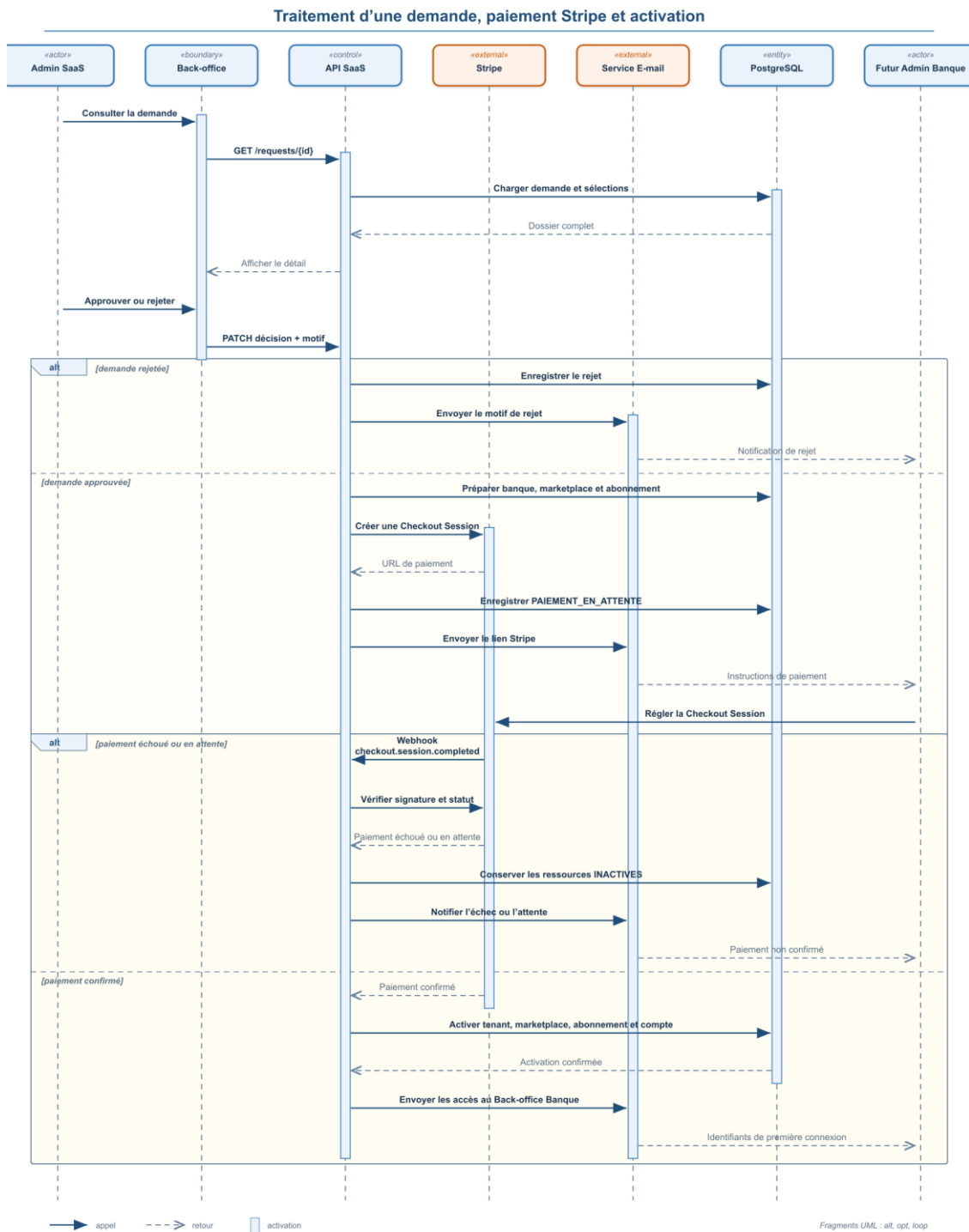


Figure 4.15 — Diagramme de séquence de l'approbation, du paiement et de l'activation

Figure 4.16 — E-mail contenant les instructions de paiement

Figure 4.17 — Confirmation du paiement

Figure 4.18 — Activation de la marketplace et de l'abonnement

Figure 4.19 — E-mail d'accès au Back-office Banque

4.4. Gestion des banques

La rubrique Banques permet à l'Administrateur SaaS de consulter les établissements enregistrés sur la plateforme et de suivre leur état d'activation. Elle offre une vue sur les informations propres à chaque banque et sur la marketplace qui lui est rattachée, tout en respectant le principe d'isolation des données entre tenants.

4.5. Gestion des utilisateurs et des rôles

Cette fonctionnalité permet de consulter les utilisateurs de la plateforme et d'administrer les rôles attribués. Elle contribue à appliquer les règles d'autorisation : chaque utilisateur accède exclusivement aux espaces et aux opérations correspondant à son profil.

4.6. Gestion des concessionnaires

L'Administrateur SaaS peut suivre les demandes d'inscription des concessionnaires, consulter les informations et documents transmis, puis statuer sur leur admissibilité. Cette rubrique permet également de superviser les comptes concessionnaires actifs et leur participation aux marketplaces bancaires.

4.7. Gestion des marketplaces

La rubrique Marketplace permet de visualiser les marketplaces créées pour les banques et d'en suivre le statut. Elle constitue un point de contrôle sur les espaces publics associés aux tenants et sur leur disponibilité après validation de l'abonnement.

4.8. Gestion du contenu des marketplaces

La gestion du contenu permet d'administrer les éléments éditoriaux et visuels utilisés par les marketplaces. L'Administrateur SaaS peut ainsi superviser les contenus publiés tout en laissant à chaque banque un périmètre de personnalisation propre à son espace.

4.9. Gestion des offres et des abonnements

Cette rubrique regroupe la gestion des offres commerciales et le suivi des abonnements. L'abonnement établit le lien entre les services sélectionnés par la banque et leur période

d'utilisation. Lors de la création initiale, il reste en attente du paiement ; après confirmation, il devient actif et les stores et modules souscrits peuvent être exploités.

Le Back-office SaaS permet également de suivre les périodes de validité, les statuts et l'échéance des abonnements, afin d'informer les acteurs concernés lorsqu'un renouvellement est nécessaire. Cette gestion garantit que seuls les services associés à un abonnement valide restent accessibles.

4.10. Audit et logs

La rubrique Audit & Logs fournit une consultation des événements importants réalisés sur la plateforme. Elle contribue à la traçabilité des opérations d'administration, des actions liées aux demandes et des opérations de paiement, ce qui facilite le suivi et l'analyse des incidents.

4.11. Paramètres de la plateforme

Enfin, la rubrique Paramètres centralise les options de configuration générale accessibles à l'Administrateur SaaS. Elle permet de maintenir les réglages nécessaires au fonctionnement cohérent de la plateforme, sans intervenir directement sur les données propres à une banque.

5. Back-office Banque : administration de la marketplace

Une fois connecté, l'Administrateur Banque dispose d'un espace exclusivement associé à son établissement. Le contexte tenant garantit que les informations affichées correspondent uniquement à sa marketplace.

Le tableau de bord fournit une synthèse de l'activité de la banque et constitue le point d'entrée vers les différentes fonctionnalités administratives.

Figure 4.20 — Tableau de bord de l'Administrateur Banque

L'espace bancaire permet à l'Administrateur Banque de gérer la personnalisation de sa marketplace, les stores et les modules activés, les produits et les contenus publiés, ainsi que la configuration du simulateur. Il lui offre également la possibilité d'administrer les concessionnaires et les partenariats, de consulter les clients rattachés à sa banque, de traiter les demandes de financement, de gérer les abonnements et de consulter les notifications reçues.

5.1. Personnalisation de la marketplace

Chaque banque peut personnaliser l'identité visuelle et le contenu de sa marketplace sans modifier le socle applicatif partagé. L'Administrateur Banque dispose ainsi d'outils lui permettant de définir les couleurs principales, le logo, les bannières, les textes d'accueil, les contenus propres à la marketplace ainsi que les contenus associés à chaque store.

Cette personnalisation permet à plusieurs établissements d'utiliser une même application tout en proposant une expérience adaptée à leur identité et à leur offre.

Figure 4.21 — Interface de personnalisation de la marketplace

Figure 4.22 — Gestion des contenus et des bannières

5.2. Gestion des stores, modules et paramètres

La banque peut consulter les stores qui lui sont attribués dans le cadre de son abonnement. Chaque store dispose de modules fonctionnels qui lui sont propres. L'Administrateur Banque peut visualiser les fonctionnalités disponibles, gérer l'activation des modules autorisés et configurer leurs paramètres selon les besoins de sa marketplace.

Les paramètres configurables sont notamment utilisés par le simulateur. La banque peut définir les valeurs nécessaires aux calculs de financement. Ces valeurs sont ensuite exploitées par la marketplace publique lors de l'exécution d'une simulation.

Figure 4.23 — Stores associés à la marketplace

Figure 4.24 — Gestion des modules et de leurs paramètres

5.3. Renouvellement et l'ajout des services

L'Administrateur Banque peut consulter les informations relatives à son abonnement, notamment son statut, sa période de validité, les services associés et son historique.

Lorsque la banque souhaite conserver les mêmes services après leur échéance, elle peut initier une demande de renouvellement. Lorsqu'elle souhaite intégrer de nouveaux stores ou modules, elle soumet une demande d'extension d'abonnement.

Dans les deux cas, la demande est transmise à l'espace SaaS, où elle est examinée et approuvée. Elle se poursuit ensuite par l'étape de paiement ; après confirmation de la transaction, l'abonnement est mis à jour et les nouveaux services sont activés.

Figure 4.25 — Diagramme d'activité du renouvellement et de l'extension d'un abonnement

Figure 4.26 — Consultation de l'abonnement et de son historique

Figure 4.27 — Soumission d'une demande de renouvellement ou d'extension

6. Marketplace publique

Une fois la marketplace active et configurée, elle devient accessible depuis son espace public.

L'interface reprend les paramètres et éléments de personnalisation définis par la banque. Elle présente notamment le logo, les couleurs, les bannières, les contenus ainsi que les stores disponibles.

Figure 4.28 — Page d'accueil d'une marketplace bancaire

La marketplace est organisée autour de plusieurs **stores**, chacun correspondant à un univers fonctionnel spécifique. Chaque store dispose de ses propres modules, définis selon la configuration de la banque. Ces modules déterminent les fonctionnalités accessibles dans le store et peuvent être activés ou désactivés en fonction des services souscrits et des besoins de la marketplace.

Figure 4.29 — Consultation d'un store

7. Conclusion

En conclusion, ce chapitre a permis de présenter la mise en œuvre de la plateforme à travers ses principaux parcours fonctionnels. Il a détaillé le cycle de vie d'une marketplace, depuis la soumission de la demande jusqu'à son activation, sa personnalisation et son exploitation. L'ensemble des fonctionnalités réalisées constitue ainsi une base concrète pour répondre aux besoins des différents acteurs et assurer le bon fonctionnement de la solution.

Chapitre 5 : Réalisation des parcours métier : concessionnaires, financement et assistant IA

1. Introduction

Ce chapitre a présenté les différentes fonctionnalités mises en œuvre pour les Concessionnaires et les Clients, depuis la gestion des produits et des partenariats jusqu'au processus de financement, ainsi que l'intégration de l'assistant IA et du chatbot au sein de la plateforme.

2. Parcours Concessionnaires

Le concessionnaire constitue un acteur important de l'écosystème Matchia puisqu'il peut proposer ses produits auprès de plusieurs banques. Afin de faciliter cette collaboration, la plateforme lui fournit un espace centralisé à partir duquel il peut gérer son activité sans avoir à créer un compte distinct pour chaque banque partenaire. Cette approche permet de conserver un point de gestion unique tout en maintenant la séparation des partenariats et des publications selon chaque banque.

2.1. Inscription et validation du concessionnaire

Le parcours débute par une demande d'inscription accessible depuis l'espace public. Le concessionnaire renseigne les informations relatives à son entreprise, ses coordonnées, le store concerné ainsi que les données nécessaires à l'identification de son activité. Il doit également transmettre les documents justificatifs requis, afin de permettre la vérification de son identité, de la conformité de son activité et de son éligibilité avant le traitement de la demande.

Une fois le formulaire soumis, la demande est enregistrée puis transmise à l'Administrateur SaaS pour traitement. Après analyse, elle peut être approuvée ou rejetée. En cas d'approbation, le système crée le compte du concessionnaire et lui donne accès à son espace personnel. Un e-mail contenant ses identifiants de connexion lui est alors envoyé. En cas de rejet, un e-mail précisant le motif de la décision est transmis au demandeur.

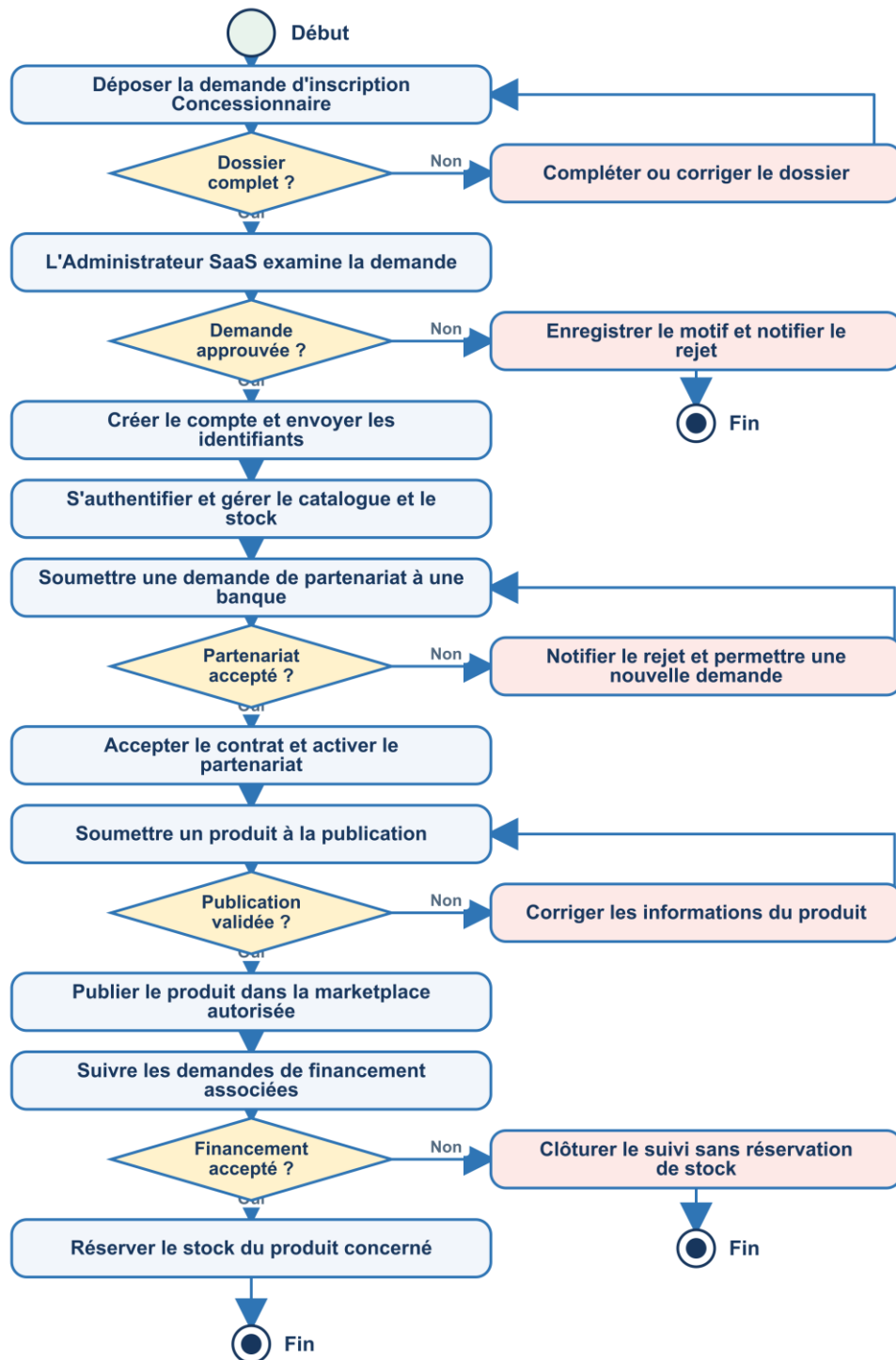


Figure 5.1 — Diagramme d'activité du parcours métier du Concessionnaire

Le diagramme de séquence ci-dessous précise la validation du dossier, sa transmission à l'Administrateur SaaS et les deux issues possibles du traitement.

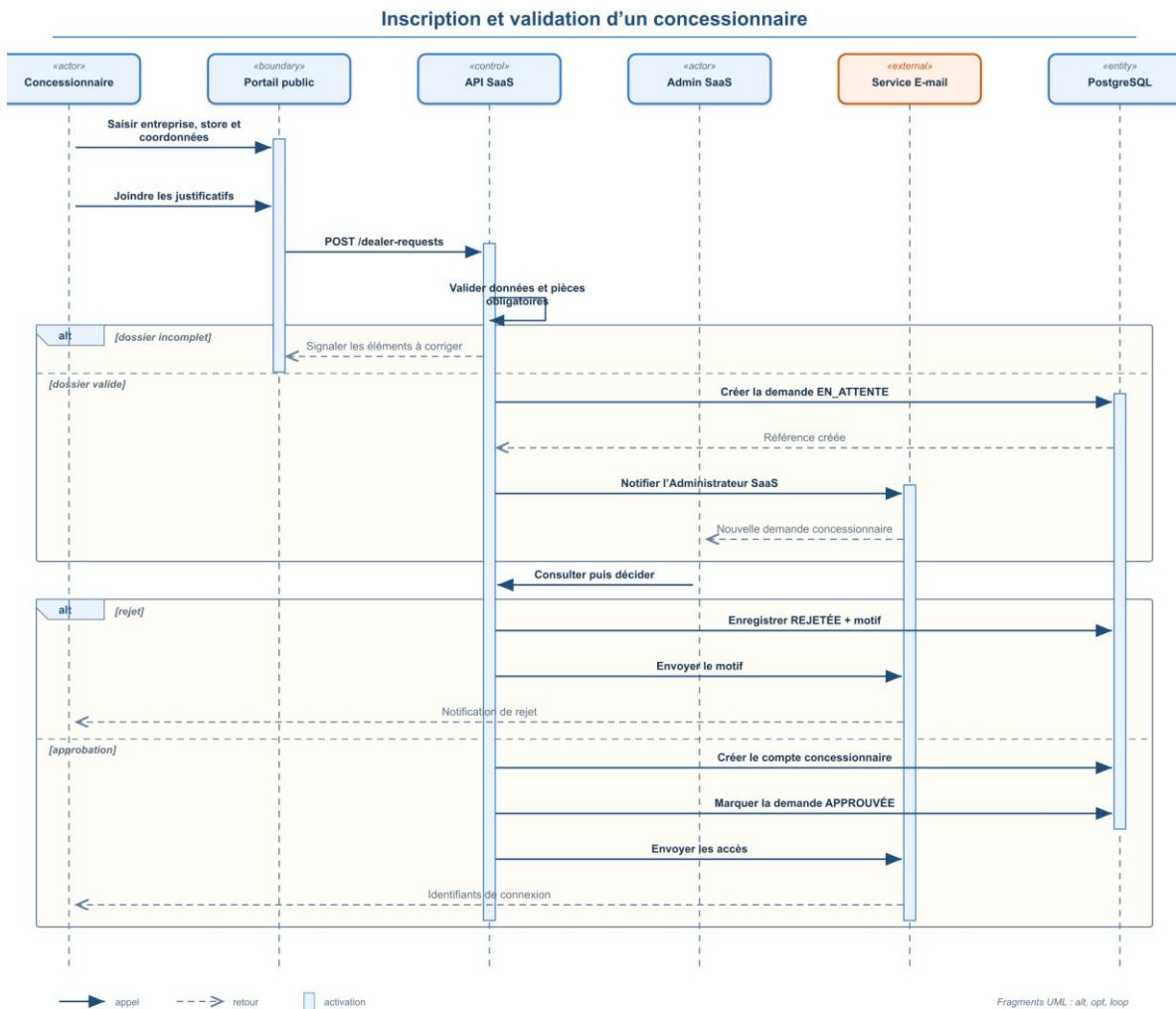


Figure 5.2 — Diagramme de séquence de l'inscription et de la validation d'un concessionnaire

Capture 5.1 — Formulaire d'inscription du concessionnaire

Cette capture présente le formulaire permettant au concessionnaire de renseigner les informations nécessaires à la création de sa demande.

Capture 5.2 — Gestion des demandes concessionnaires par l'Administrateur SaaS

Cette capture illustre l'interface permettant au SaaS de consulter, filtrer et traiter les demandes reçues.

2.2. Partenariats et contrats

Une fois le compte concessionnaire validé, la collaboration avec une banque est organisée à travers un mécanisme de partenariat. Dans Matchia, le partenariat constitue le lien entre un

concessionnaire, une banque et un store donné. Cette modélisation permet à un même concessionnaire de collaborer avec plusieurs banques tout en conservant un compte unique.

La demande de partenariat peut être initiée dans les deux sens. Depuis son espace, le concessionnaire sélectionne la banque et le store auxquels il souhaite proposer sa collaboration ; la banque destinataire peut ensuite consulter la demande et l'approuver ou la rejeter. Inversement, l'Administrateur Banque peut sélectionner un concessionnaire disponible et lui adresser une demande de partenariat. Le concessionnaire reçoit alors cette demande dans son espace et peut à son tour l'accepter ou la rejeter. Dans les deux cas, la décision et, le cas échéant, son motif sont enregistrés afin d'assurer la traçabilité du traitement.

Le contrat définit les conditions de collaboration entre la banque et le concessionnaire. Il précise notamment la période de validité, les commissions éventuellement applicables, les conditions contractuelles ainsi que les modalités de résiliation. Son cycle de vie comprend les états d'attente d'acceptation, d'activation, d'annulation, d'expiration ou de résiliation. L'activation intervient après l'acceptation du concessionnaire et la validation finale de la banque.

Figure 5.3 — Cycle de vie d'un partenariat, d'un contrat et d'une publication

La figure permet de visualiser l'enchaînement entre la création du partenariat, la formalisation du contrat et l'autorisation de publication du produit.

Capture 5.3 — Gestion des partenariats et des contrats

Cette interface permet au concessionnaire de consulter ses relations avec les différentes banques ainsi que les contrats associés.

2.3. Produits et publications multi-banque

Après la validation du partenariat, le concessionnaire accède à un espace centralisé lui permettant de gérer son catalogue. Il peut y créer, modifier et mettre à jour les informations relatives à ses produits, tout en assurant le suivi de leur stock.

Le catalogue des produits est géré indépendamment de leur diffusion auprès des banques. Un même produit peut ainsi être proposé à plusieurs banques sans qu'il soit nécessaire de créer plusieurs copies du produit. Le concessionnaire conserve donc une référence centrale de son catalogue tandis que chaque publication est associée au partenariat et à la marketplace concernée.

Cette organisation permet au concessionnaire de gérer le stock de chaque produit depuis une seule fiche centralisée. Il peut ainsi suivre la quantité totale, la quantité disponible à la vente et la quantité réservée, sans devoir mettre à jour séparément le même produit dans chaque marketplace bancaire. Lorsqu'une information du produit ou de son stock est modifiée, cette mise à jour est automatiquement prise en compte pour l'ensemble de ses publications autorisées.

Cependant, la publication d'un produit reste soumise à plusieurs conditions. Le concessionnaire doit être approuvé, le partenariat et le contrat doivent être valides, le store concerné doit être actif et la publication doit être approuvée par la banque.

Le diagramme de séquence suivant distingue la gestion du produit central, le stockage de ses médias et la demande de publication propre à chaque banque partenaire.

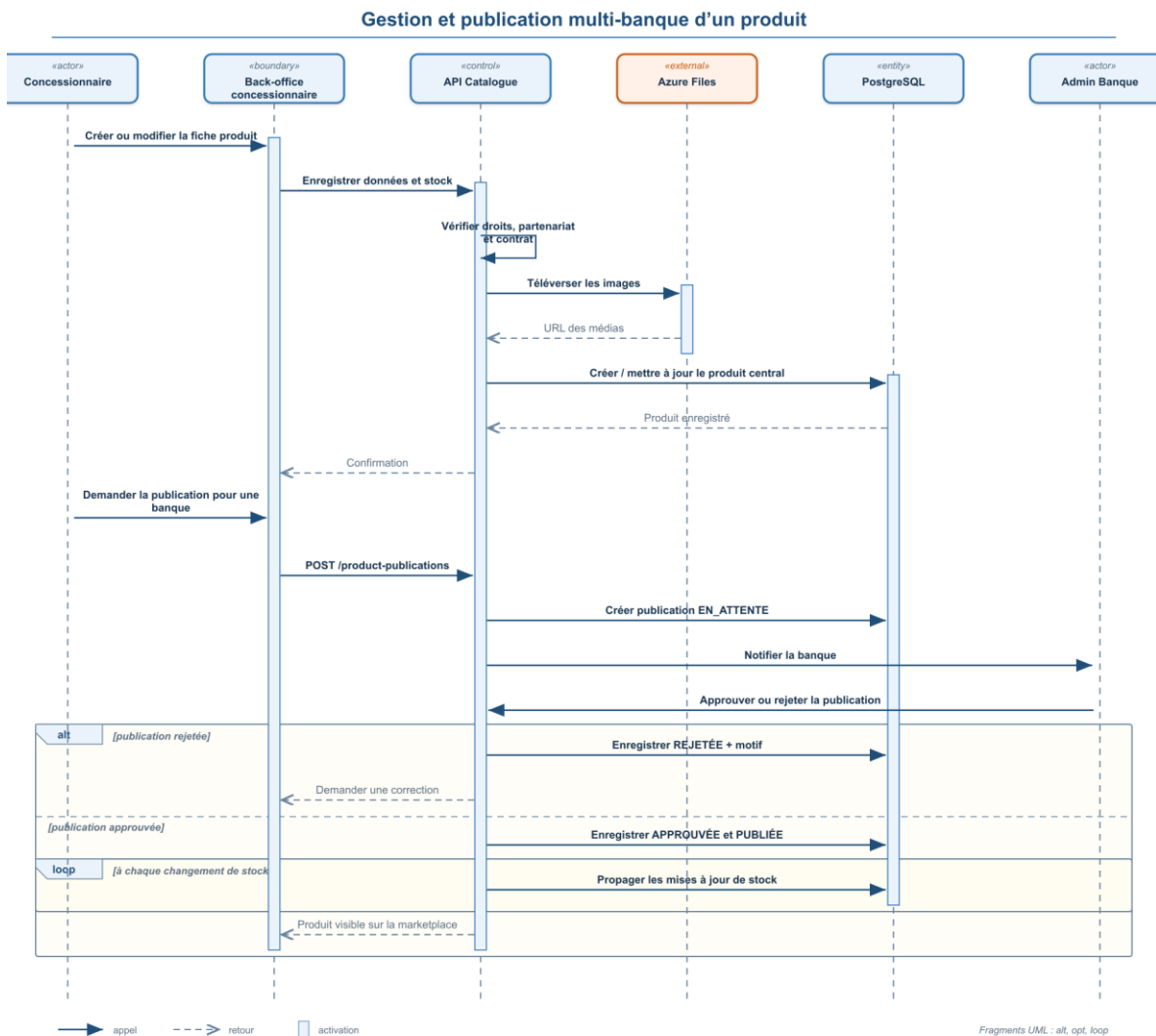


Figure 5.4 — Diagramme de séquence de la gestion et de la publication multi-banque d'un produit

Capture 5.4 — Création d'un produit et gestion du stock

Capture 5.5 — Validation bancaire d'une publication

2.4. Suivi des demandes de financement

Le concessionnaire dispose également d'une section dédiée au suivi des demandes de financement associées à ses produits. Cette fonctionnalité prolonge le principe de compte unique : les demandes provenant de différentes marketplaces partenaires peuvent être consultées depuis le même espace.

La section de suivi permet notamment de consulter les références des dossiers, les produits concernés, la banque associée, le client concerné, le montant demandé, la date de création et le statut du dossier.

Lorsqu'une demande associée à un produit concessionnaire est acceptée par la banque, le stock correspondant peut être réservé automatiquement. En revanche, une demande rejetée ne provoque aucune réservation.

Capture 5.6 — Suivi des demandes de financement par le concessionnaire

3. Parcours client et demande de financement

Le parcours client constitue un processus important de la plateforme. Il permet à un visiteur de découvrir les offres proposées dans la marketplace, de consulter et de comparer les produits disponibles, puis d'effectuer une simulation de financement. Lorsqu'il souhaite poursuivre sa démarche, il peut créer un compte client puis soumettre une demande de financement.

3.1. Consultation, comparaison et simulation

Depuis la marketplace publique, le visiteur peut parcourir les différents stores et consulter les produits proposés par les concessionnaires partenaires. Chaque fiche produit présente les informations utiles à la prise de décision, telles que les caractéristiques, le prix, les images et, selon le store, les conditions d'éligibilité associées.

Lorsque les modules correspondants sont activés, le visiteur peut sélectionner plusieurs produits afin de les comparer selon différents critères. Il peut également utiliser le simulateur de financement pour obtenir une estimation à partir du montant du produit, de la durée souhaitée

et de l'apport personnel. Ces fonctionnalités restent accessibles avant la création d'un compte et permettent au visiteur d'étudier les offres avant d'entamer une demande de financement.

Capture 5.7 — Comparateur /simulateur

3.2. Inscription du client et activation du compte

Lorsqu'un visiteur souhaite déposer une demande de financement, il est orienté vers la création d'un compte client. Cette inscription s'effectue dans le contexte de la marketplace bancaire concernée afin de rattacher le compte au tenant correspondant.

Après validation du formulaire, un code de vérification à six chiffres est envoyé à l'adresse e-mail renseignée. Le compte est activé après la validation de ce code. Le client peut alors se connecter à son espace personnel et reprendre le processus de financement.

Le diagramme de séquence détaille la création initiale du compte inactif, l'envoi du code à six chiffres et l'activation du client dans le tenant de la marketplace consultée.

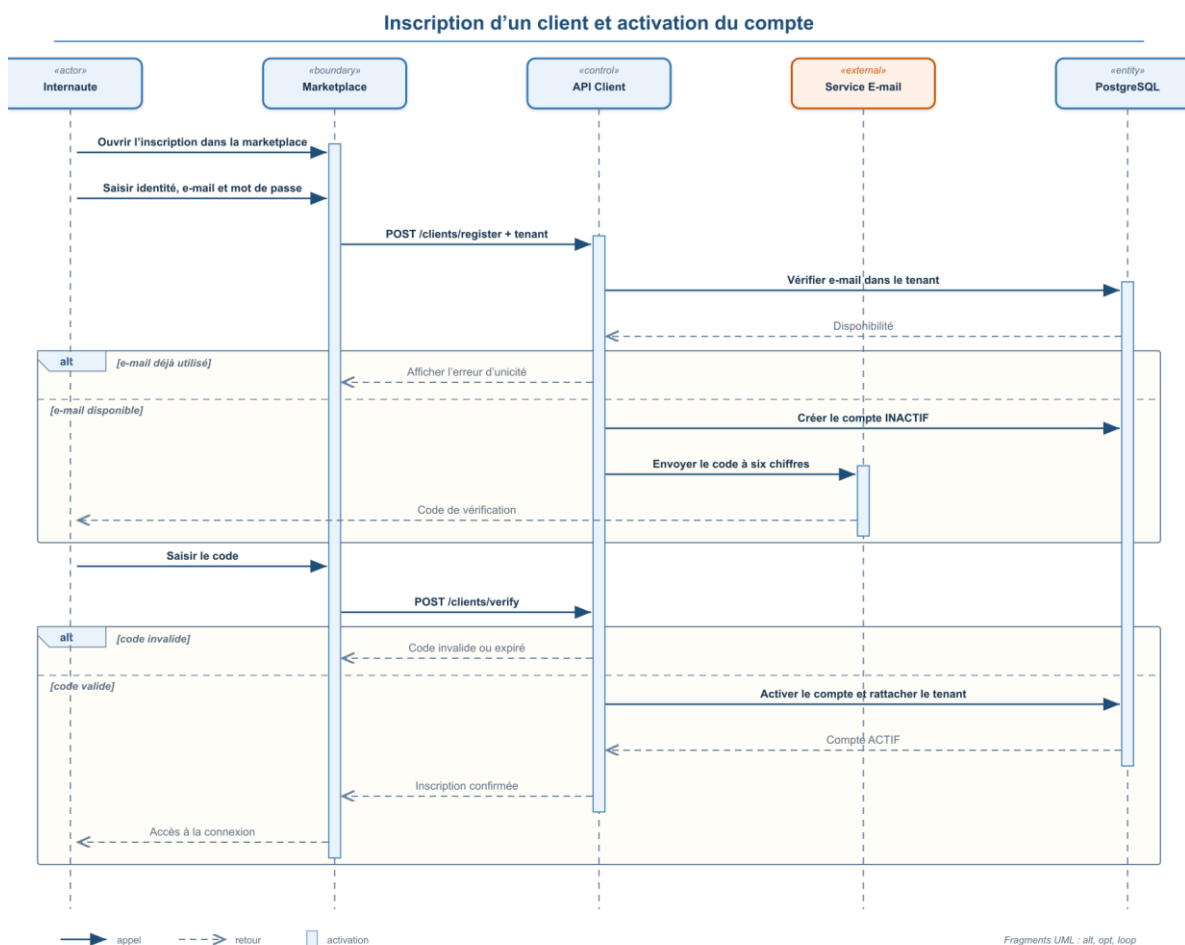


Figure 5.5 — Diagramme de séquence de l'inscription et de l'activation du compte Client

Capture 5.7 — Inscription et vérification du compte client**3.3. Constitution et soumission du dossier**

Une fois authentifié, le client peut commencer sa demande depuis la fiche d'un produit ou à partir du résultat de sa simulation. Le dossier est créé initialement à l'état de brouillon, ce qui lui permet de compléter progressivement les informations requises et de déposer les documents justificatifs demandés.

Avant la soumission définitive, le système vérifie la présence des pièces obligatoires. Une fois le dossier complet, il est transmis à la banque pour traitement.

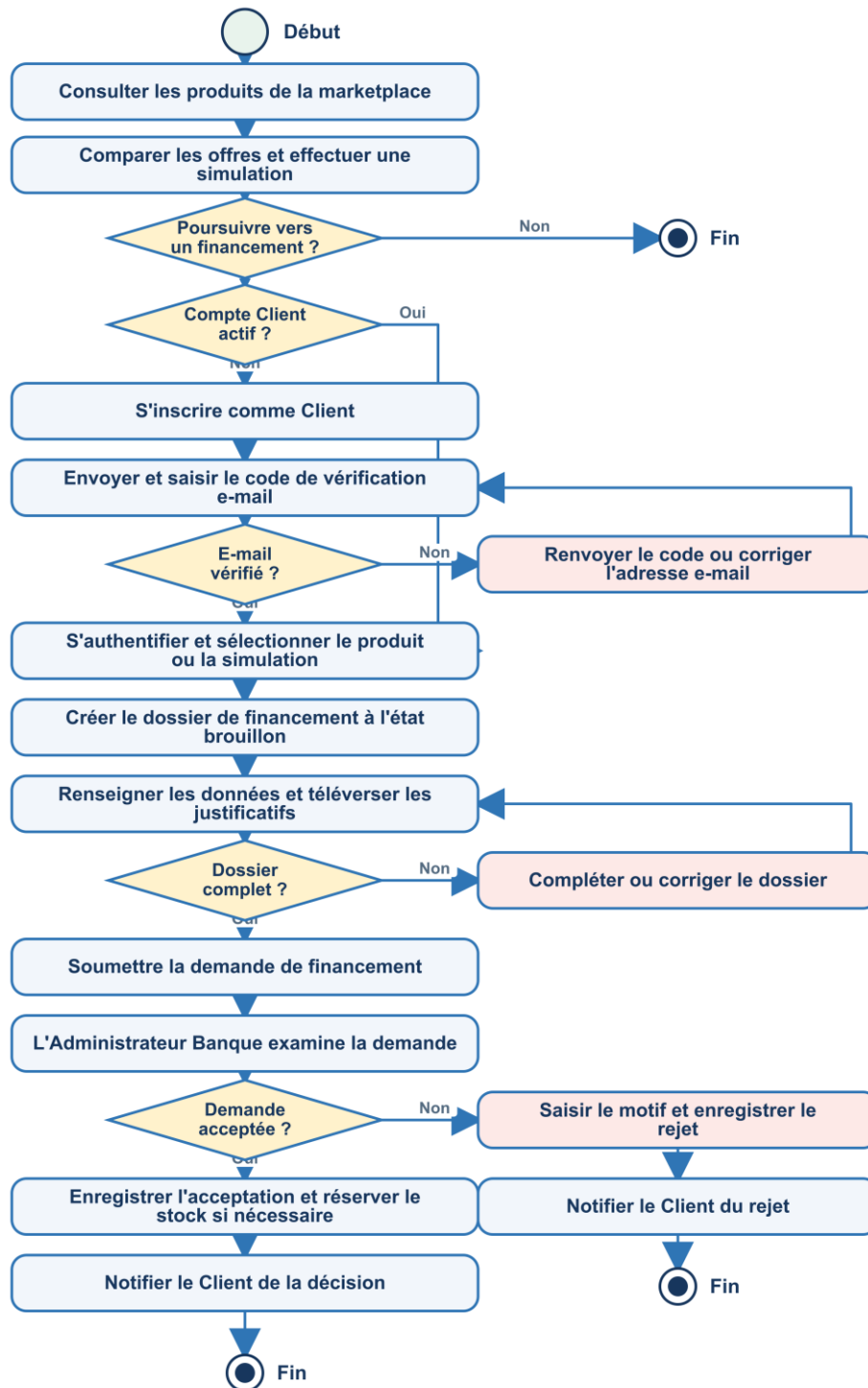


Figure 5.6 — Diagramme d'activité du parcours Client et de la demande de financement

La figure présente les principales étapes du parcours client, depuis la simulation jusqu'à la soumission du dossier et sa transmission à la banque.

3.4. Traitement de la demande par la banque

Après sa soumission, la demande est transmise à la banque concernée. L'Administrateur Banque peut alors consulter le détail du dossier ainsi que les documents fournis par le client avant de rendre sa décision. Lorsqu'elle est en attente de traitement, la demande peut être acceptée ou rejetée, avec l'ajout obligatoire d'un motif de rejet.

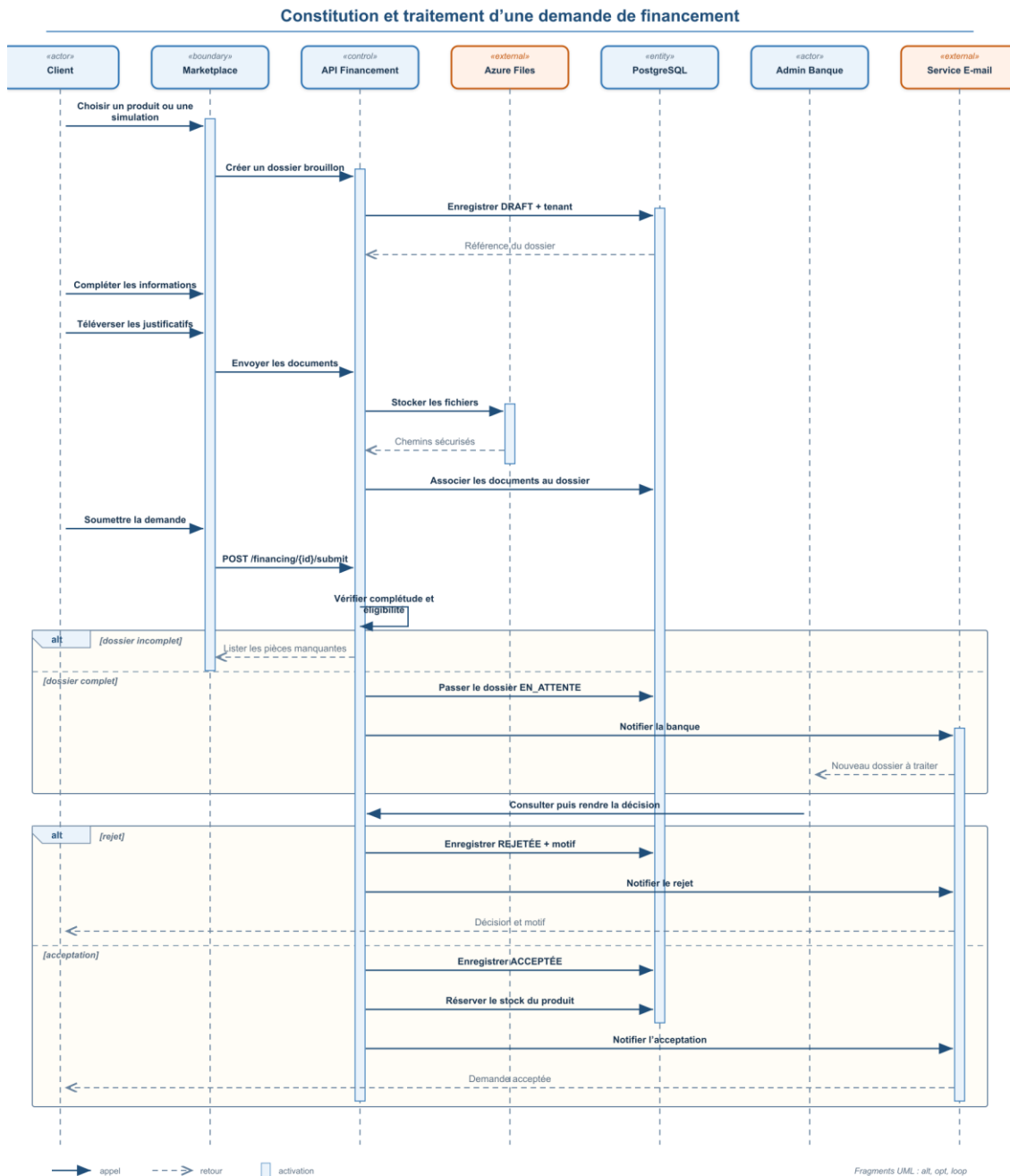


Figure 5.7 — Diagramme de séquence de la constitution et du traitement d'une demande de financement

Capture 5.8 — Simulation et création d'une demande de financement

Capture 5.9 — Téléversement des documents justificatifs

Capture 5.10 — Traitement et décision de la banque

4. Assistants conversationnels de la plateforme

La plateforme Matchia intègre deux solutions conversationnelles complémentaires répondant à des besoins distincts : un **assistant IA** destiné au pilotage du Back-office SaaS et un **chatbot public** accessible aux visiteurs des marketplaces bancaires. Bien que ces deux composants permettent une interaction en langage naturel, ils se distinguent par leurs objectifs, leurs utilisateurs cibles ainsi que par les mécanismes techniques sur lesquels ils reposent.

4.1. Assistant IA pour le pilotage du SaaS

L'assistant IA est intégré au Back-office SaaS et est destiné principalement à l'Administrateur SaaS. Il facilite l'accès aux informations de pilotage en permettant à l'administrateur de formuler directement ses questions en langage naturel. Ces interrogations peuvent notamment concerner les banques, les marketplaces, les demandes, les abonnements, les paiements, les stores ou encore les modules disponibles sur la plateforme.

Cet assistant s'appuie sur le modèle **Gemini**, appartenant à la famille des **LLM**. Grâce à ses capacités de traitement du langage naturel (**NLP**), le modèle analyse la question formulée par l'utilisateur, en détermine le sens et génère une requête permettant de récupérer les informations correspondantes dans la base de données. Les résultats obtenus sont ensuite reformulés afin de fournir une réponse compréhensible et directement exploitable par l'administrateur.

4.2. Fonctionnement et sécurisation de l'assistant IA

Lorsqu'une question est envoyée par l'administrateur, le backend commence par construire un schéma dynamique contenant uniquement les tables, les colonnes et les relations autorisées. Ce schéma, accompagné de la question formulée en langage naturel, est ensuite transmis au modèle Gemini.

À partir de ces informations, Gemini génère une proposition de requête SQL correspondant à la demande. Toutefois, cette requête n'est jamais exécutée directement sur la base de données. Elle est préalablement analysée par un validateur déterministe, chargé de vérifier qu'elle respecte les règles de sécurité définies par la plateforme.

Par ailleurs, les informations sensibles telles que les mots de passe, les tokens, les secrets, les clés, les données de vérification ainsi que certaines informations sont exclues du schéma

transmis au modèle. Après validation, la requête est exécutée en lecture seule sur PostgreSQL, avec un contrôle du temps maximal d'exécution.

Les résultats obtenus sont ensuite transmis à Gemini afin de générer une réponse claire, précise et adaptée au contexte métier.

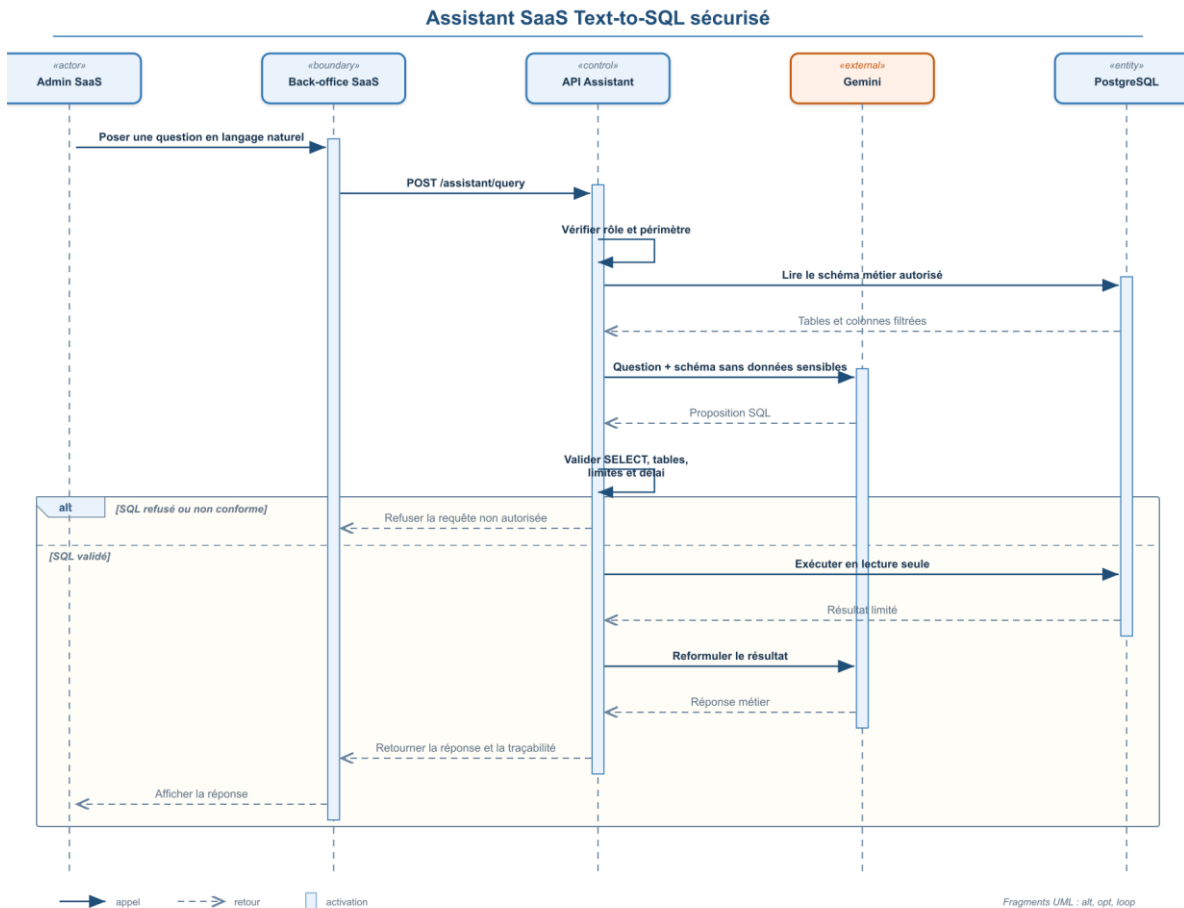


Figure 5.8 — Diagramme de séquence de l'assistant IA Text-to-SQL sécurisé

4.3. Chatbot public des marketplaces

Le chatbot public constitue le second composant conversationnel de la plateforme. Il est destiné aux visiteurs des marketplaces bancaires et apparaît uniquement lorsque le module Chatbot est activé pour le store consulté. Son rôle principal consiste à accompagner le visiteur dans la découverte des produits et à faciliter son accès aux différentes fonctionnalités disponibles au sein de la marketplace.

Ce chatbot public est un système conversationnel basé sur des règles métier. Son fonctionnement consiste à normaliser le message reçu, à rechercher des mots-clés et à identifier une intention prédéfinie afin de fournir une réponse adaptée.

Lorsqu'un visiteur envoie un message, le système identifie tout d'abord la marketplace et le store dans lesquels la conversation est effectuée. Le chatbot analyse ensuite le contenu du message afin d'en déterminer l'intention. Celle-ci peut notamment correspondre à la recherche d'un produit, à la consultation de sa disponibilité ou de ses caractéristiques, à l'identification du concessionnaire qui le propose, à une demande de comparaison ou encore à une orientation vers le simulateur de financement.

L'accès aux données reste limité au périmètre public de la marketplace consultée. Le chatbot refuse donc toute demande non autorisée.

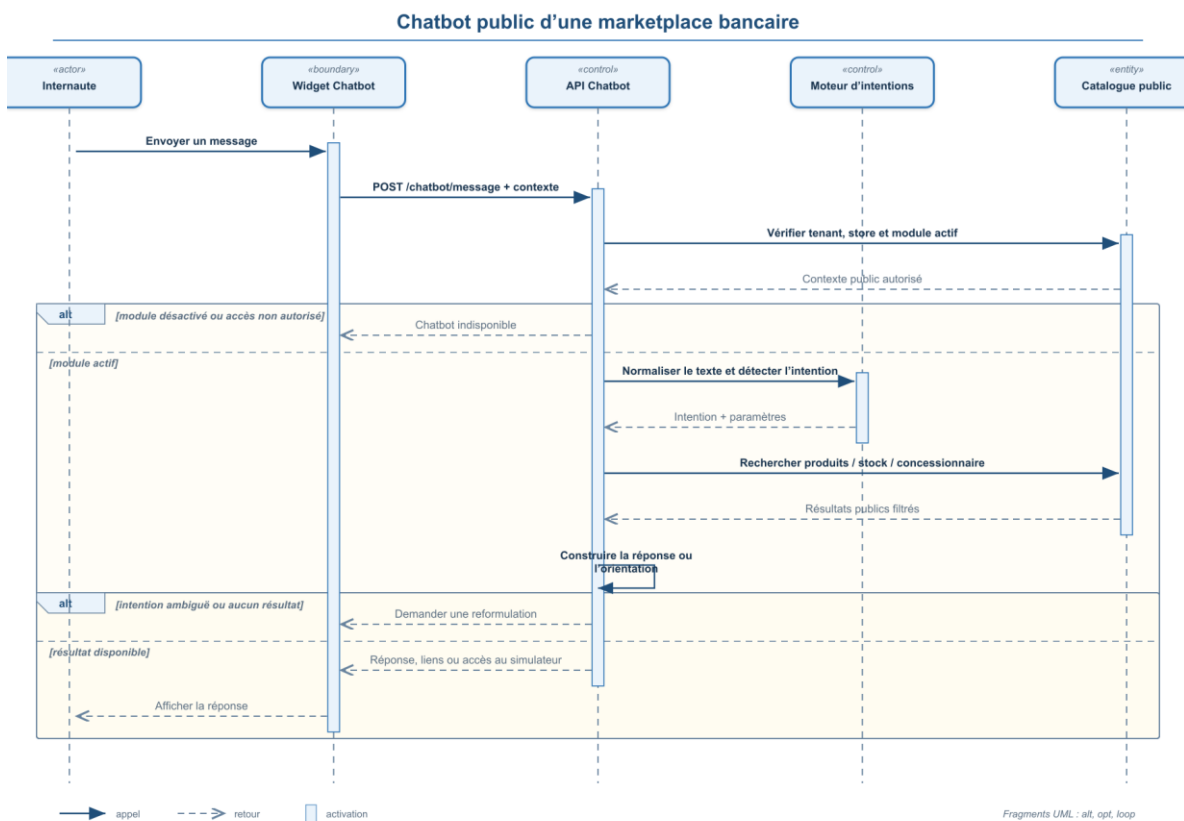


Figure 5.9 — Diagramme de séquence du chatbot public d'une marketplace bancaire

5. Conclusion

En conclusion, ce chapitre a permis de mettre en évidence l'ensemble des fonctionnalités métier développées pour les Concessionnaires et les Clients, ainsi que les mécanismes facilitant leurs

interactions avec les banques. Il a également présenté l'apport de l'assistant IA et du chatbot dans l'amélioration de l'accès à l'information et de l'accompagnement des utilisateurs.

Chapitre 6 : Mise en place de la démarche DevOps, du pipeline CI/CD et déploiement de Matchia sur Microsoft Azure

1. Introduction

Après la conception et le développement de la plateforme, une démarche de livraison et de déploiement a été mise en place afin que chaque nouvelle version puisse être construite, vérifiée et déployée de manière reproductible. L'objectif ne se limite pas à rendre l'application accessible dans le Cloud : il consiste également à automatiser les contrôles qui précèdent la mise en production.

La solution repose sur GitHub pour la centralisation du code, Docker pour la conteneurisation, Jenkins pour l'orchestration du pipeline, SonarQube pour l'analyse de la qualité, Docker Hub pour la distribution des images et Microsoft Azure pour l'hébergement. Azure CLI permet enfin au pipeline d'appliquer les mises à jour sur les services Cloud.

La figure suivante présente la chaîne globale allant du push du développeur jusqu'au déploiement des conteneurs dans Microsoft Azure.

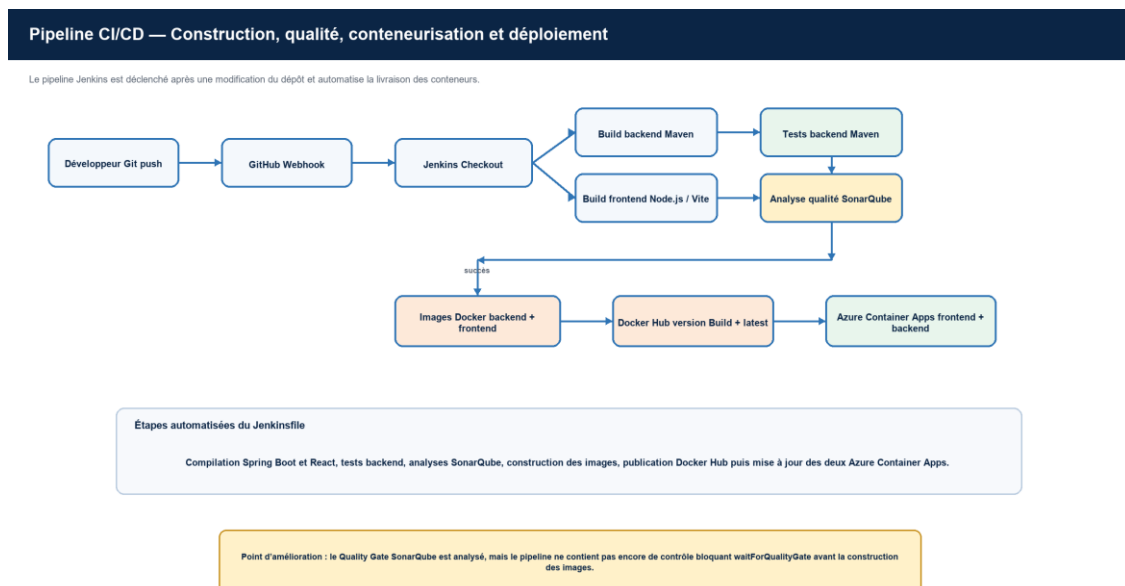


Figure 6.1 — Diagramme du pipeline CI/CD de la plateforme

Figure 6.1 — Vue générale de la chaîne DevOps et CI/CD de Matchia

2. Démarche DevOps et principes de la CI/CD

2.1. Démarche DevOps

Le DevOps est une démarche qui associe étroitement les activités de développement et d'exploitation afin d'améliorer la collaboration, la qualité des livraisons et la fiabilité des déploiements. Dans ce projet, il se traduit par un processus automatisé qui enchaîne la construction, les tests, l'analyse du code, la création d'images Docker et le déploiement des versions validées.

2.2. Intégration et déploiement continus

L'intégration continue soumet chaque modification à des opérations automatiques de compilation, de test et de contrôle de qualité. Le déploiement continu prolonge cette vérification en construisant les images Docker, en les publiant dans un registre puis en mettant à jour les services Azure. Une étape critique en échec interrompt le pipeline et empêche le déploiement d'une version non validée.

3. Préparation et validation de la conteneurisation

Avant d'automatiser la livraison, les différentes étapes techniques ont été mises en place et validées séparément. Les constructions du backend et du frontend, la création des images Docker et l'exécution locale des conteneurs ont d'abord été réalisées manuellement. Cette phase a permis de vérifier les Dockerfiles, les variables d'environnement, les ports exposés et la communication entre les services avant de confier leur enchaînement à Jenkins.

3.1. Gestion des versions avec Git et GitHub

Git assure le suivi local des évolutions du projet tandis que GitHub centralise le dépôt distant. Le dépôt rassemble le backend Spring Boot, le frontend React, le fichier docker-compose.yml et le Jenkinsfile. Il devient ainsi le point d'entrée du pipeline : un push sur la branche suivie par Jenkins déclenche la récupération de la nouvelle version du code.

La figure 6.2 met en évidence l'organisation du dépôt et les principaux fichiers nécessaires à la construction et au déploiement de l'application.

Organisation du dépôt GitHub Matchia

```
Matchia/
├── MatchiaBackend/    API Spring Boot et tests Maven
├── MatchiaFrontend/   application React TypeScript
├── docker-compose.yml orchestration locale des conteneurs
└── Jenkinsfile        pipeline CI/CD versionné
```

Figure 6.2 — Organisation du dépôt GitHub de la plateforme Matchia

3.2. Conteneurisation du backend Spring Boot

Le backend est développé avec Spring Boot et Java 17. Son Dockerfile suit une construction en plusieurs étapes : Maven compile d'abord l'application, puis une image Java d'exécution plus légère ne conserve que le fichier JAR produit. Le service expose l'API REST sur le port 8081. L'image a été construite et exécutée manuellement lors de la phase initiale afin de valider le démarrage de l'API avant son intégration au pipeline.

```

PS D:\PFE M2\Plateforme SaaS\MatchiaBackend> docker build -t matchia-backend .
[+] Building 111.9s (15/15) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                  0.0s
=> => transferring dockerfile: 737B                                  0.0s
=> [internal] load metadata for docker.io/library/eclipse-temurin:17-jre 2.4s
=> [internal] load metadata for docker.io/library/maven:3.9-eclipse-temurin-17 2.4s
=> [internal] load .dockerignore                                     0.0s
=> => transferring context: 2B                                         0.0s
=> [build 1/6] FROM docker.io/library/maven:3.9-eclipse-temurin-17@sha256:4815718812bbf1113ec6cf2b958be328d90 30.9s
=> => resolve docker.io/library/maven:3.9-eclipse-temurin-17@sha256:4815718812bbf1113ec6cf2b958be328d90265cebb 0.0s
=> sha256:14a96477f37298a0892b0baacd8ba56ab1d2d3093abb5cf9b386b9fa21aa66 1558 / 1558 0.1s
=> sha256:6a172338dc8967fe98e4f79f58c1da535d8eed9b1c5e9f1e96825b659d41168 3588 / 3588 0.2s
=> sha256:583b4ddfc1e57d25486aebc4c781079bec1deee5db9cf1618c854ef6d282bde3 9.36MB / 9.36MB 4.8s
=> sha256:eac13abce8a9aae584d015c38d7bb63f9a491b95a1f61fcd401ea44617c3f1 8498 / 8498 0.3s
=> sha256:04c5cfe1b038c02d41e5fa0fb1952474bec8da575c28a7a75d910979809a72b 22.55MB / 22.55MB 8.9s
=> sha256:66110416a118e8961eba6236a8da2d3358de4c9090c1adcc92bfe8cebfb8baca 2.28kB / 2.28kB 0.5s
=> sha256:54ad9a5b2563e73118cd3c060d02c6f626d4bd5fcdab6f1c29963c0ab463532e 1698 / 1698 0.3s
=> sha256:2d129e9ac955a75cd701f995eea0f7468f4824abc484b3de8f8a90f3d78c251 145.91MB / 145.91MB 23.6s
=> sha256:4765da19721820a8d81b0abcaebadd8e2d40a808b0bdc6cd783e6dd951be97f6b 25.33MB / 25.33MB 9.8s
=> sha256:966c395d29cb24a3fa77e04f32076f4c5f778319d4132dae4f47e2aa7b9da4924 29.75MB / 29.75MB 8.7s
=> extracting sha256:966c395d29cb24a3fa77e04f32076f4c5f778319d4132dae4f47e2aa7b9da4924 0.9s
=> extracting sha256:4765da19721820a8d81b0abcaebadd8e2d40a808b0bdc6cd783e6dd951be97f6b 0.7s
=> extracting sha256:2d129e9ac955a75cd701f995eea0f7468f4824abc484b3de8f8a90f3d78c251 1.2s
=> extracting sha256:54ad9a5b2563e73118cd3c060d02c6f626d4bd5fcdab6f1c29963c0ab463532e 0.0s
=> extracting sha256:66110416a118e8961eba6236a8da2d3358de4c9090c1adcc92bfe8cebfb8baca 0.0s
=> extracting sha256:04c5cfe1b038c02d41e5fa0fb1952474bec8da575c28a7a75d910979809a72b 0.5s
=> extracting sha256:583b4ddfc1e57d25486aebc4c781079bec1deee5db9cf1618c854ef6d282bde3 0.1s
=> extracting sha256:eac13abce8a9aae584d015c38d7bb63f9a491b95a1f61fcd401ea44617c3f1 0.0s
=> extracting sha256:6a172338dc8967fe98e4f79f58c1da535d8eed9b1c5e9f1e96825b659d41168 0.0s
=> extracting sha256:14a96477f37298a0892b0baacd8ba56ab1d2d3093abb5cf9b386b9fa21aa66 0.0s
=> [stage-1 1/3] FROM docker.io/library/eclipse-temurin:17-jre@sha256:e4f018a55645ad204892e44eb35437518d7e108ba 16.9s
=> => resolve docker.io/library/eclipse-temurin:17-jre@sha256:e4f018a55645ad204892e44eb35437518d7e108ba2a2dce305 0.0s
=> sha256:26df7c06c61d8c89938b275761188d37d909096584bae567db05ad2959e406df 2.28kB / 2.28kB 0.2s
=> sha256:08758b58d388b0203443825c9cdee0fa66e88ea686000d8fc0072e19cfb109e 1598 / 1598 0.2s
=> sha256:dd424028a6d5ec8f9fd4f51e23e6d19301d3297d4f97af4636caf6d9e8522a47 47.57MB / 47.57MB 15.8s
=> sha256:5908a9e4b3f7100211678e03fbc6d80ac7783c4f6ca348a05fb07e06d24c58b1 29.39MB / 29.39MB 12.6s
=> extracting sha256:5908a9e4b3f7100211678e03fbc6d80ac7783c4f6ca348a05fb07e06d24c58b1 0.7s
=> extracting sha256:dd424028a6d5ec8f9fd4f51e23e6d19301d3297d4f97af4636caf6d9e8522a47 0.8s
=> extracting sha256:08758b58d388b0203443825c9cdee0fa66e88ea686000d8fc0072e19cfb109e 0.0s
=> extracting sha256:26df7c06c61d8c89938b275761188d37d909096584bae567db05ad2959e406df 0.0s
=> [internal] load build context                                     0.2s
=> => transferring context: 976.71kB                                  0.1s
=> [stage-1 2/3] WORKDIR /app                                       0.2s
=> [build 2/6] WORKDIR /app                                         0.4s
=> [build 3/6] COPY pom.xml                                          0.0s
=> [build 4/6] RUN mvn dependency:go-offline -B                      62.0s
=> [build 5/6] COPY src ./src                                       0.1s
=> [build 6/6] RUN mvn clean package -DskipTests                   12.1s
=> [stage-1 3/3] COPY --from=build /app/target/*.jar app.jar       0.2s
=> exporting to image                                               3.3s
=> => exporting layers                                               2.7s
=> => exporting manifest sha256:2b3b3eb6fe87e2a08451c486f023d5e3f95c011409814c7797479a586f9ef 0.0s
=> => exporting config sha256:72b6b6c6561f69966bb979e892b4804e24fdd5fcd408b124326a9a917531c21f9f 6.0s
=> => exporting attestation manifest sha256:88018306261bd6e8a168a4304e5d15985f1d658a8c653e4759f48773c3a309ef5 0.0s
=> => exporting manifest list sha256:cfc6604c83bcd6a0135597ed9293e696f09bf1255fae4c9b92b063fbd2eed 0.0s
=> => naming to docker.io/library/matchia-backend:latest            0.0s
=> => unpacking to docker.io/library/matchia-backend:latest         0.5s

```

Figure 6.3 — Dockerfile utilisé pour la conteneurisation du backend Matchia

3.3. Conteneurisation du frontend React

Le frontend React, TypeScript et Vite est également construit en plusieurs étapes. Node.js installe les dépendances et génère les fichiers statiques de production ; ceux-ci sont ensuite servis par Nginx. La variable VITE_API_URL est fournie au moment du build afin d'associer le frontend à l'URL du backend correspondant à l'environnement cible. La construction et l'exécution de cette image ont été vérifiées manuellement avant leur automatisation.

```

PS C:\WINDOWS\system32> cd "D:\PFE M2\Plateforme SaaS\MatchiaFrontend"
PS D:\PFE M2\Plateforme SaaS\MatchiaFrontend> docker build -t matchia-frontend .
[+] Building 39.7s (16/16) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 903B                                0.0s
=> [internal] load metadata for docker.io/library/node:22-alpine   0.9s
=> [internal] load metadata for docker.io/library/nginx:alpine     0.9s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 148B                                     0.0s
=> [build 1/6] FROM docker.io/library/node:22-alpine@sha256:c610fcd5b4740dd70c284ed3cb16bb857e0f7166196e36a5  0.0s
=> => resolve docker.io/library/node:22-alpine@sha256:c610fcd5b4740dd70c284ed3cb16bb857e0f7166196e36a5501df7  0.0s
=> [internal] load build context                                   0.0s
=> => transferring context: 11.00kB                                  0.0s
=> [stage-1 1/4] FROM docker.io/library/nginx:alpine@sha256:4a73073bd557c65b759505da037898b61f1be6cbcc3c2c3aeac2  0.0s
=> => resolve docker.io/library/nginx:alpine@sha256:4a73073bd557c65b759505da037898b61f1be6cbcc3c2c3aeac22d2a470c  0.0s
=> CACHED [stage-1 2/4] RUN rm /etc/nginx/conf.d/default.conf     0.0s
=> CACHED [stage-1 3/4] COPY nginx.conf /etc/nginx/conf.d/default.conf 0.0s
=> CACHED [build 2/6] WORKDIR /app                                 0.0s
=> CACHED [build 3/6] COPY package*.json ./                        0.0s
=> [build 4/6] RUN npm install                                     20.1s
=> [build 5/6] COPY . .                                           0.4s
=> [build 6/6] RUN npm run build                                   16.2s
=> [stage-1 4/4] COPY --from=build /app/dist /usr/share/nginx/html 0.1s
=> exporting to image                                             0.8s
=> => exporting layers                                             0.4s
=> => exporting manifest sha256:92784c13e652126fca54174cd2d5645ca6ea77ced48d624b9d81647814c78490  0.0s
=> => exporting config sha256:39f0bd257e66656f6de09a8f97006a6dd4e9f7b1b145f28438fdaa21d2cd352d  0.0s
=> => exporting attestation manifest sha256:5e20057af14e9dfa0c311893c8a29101570346f19ac91e7a3153a8c6bf7004e7 0.0s
=> => exporting manifest list sha256:4bc2ac01e79a7a2cf2d14954ee31fdd07c26f43556749b47b2d33d66e5c04fe3  0.0s
=> => naming to docker.io/library/matchia-frontend:latest        0.0s
=> => unpacking to docker.io/library/matchia-frontend:latest     0.1s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/iwrgvkd0ggq0w2kjtlnqds1dx
PS D:\PFE M2\Plateforme SaaS\MatchiaFrontend>

```

Figure 6.4 — Dockerfile utilisé pour la conteneurisation du frontend Matchia

3.4. Orchestration locale avec Docker Compose

Docker Compose décrit les services frontend et backend, les ports exposés, les variables d'environnement, le volume des fichiers téléversés et le réseau de communication. La commande `docker compose up -d` permet de démarrer un environnement local cohérent. Cette exécution manuelle a permis de vérifier que les deux conteneurs communiquent correctement et que l'application est fonctionnelle avant l'automatisation de la chaîne CI/CD.

```

PS D:\PFE M2\Plateforme SaaS> docker compose config
name: platformesaas
services:
  backend:
    build:
      context: D:\PFE M2\Plateforme SaaS\backend
      dockerfile: Dockerfile
    container_name: matchia-backend
    environment:
      SERVER_PORT: "8081"
      SPRING_DATASOURCE_PASSWORD: "123456"
      SPRING_DATASOURCE_URL: jdbc:postgresql://host.docker.internal:5432/matchiaSaaS
      SPRING_DATASOURCE_USERNAME: yasmine
    networks:
      matchia-network: null
    ports:
      - mode: ingress
        target: 8081
        published: "8081"
        protocol: tcp
    restart: unless-stopped
  frontend:
    build:
      context: D:\PFE M2\Plateforme SaaS\frontend
      dockerfile: Dockerfile
    args:
      VITE_API_URL: http://lvh.me:8081
    container_name: matchia-frontend
    depends_on:
      backend:
        condition: service_started
        required: true
    networks:
      matchia-network: null
    ports:
      - mode: ingress
        target: 80
        published: "5173"
        protocol: tcp
    restart: unless-stopped
    networks:
      matchia-network:
        name: platformesaas_matchia-network
        driver: bridge
PS D:\PFE M2\Plateforme SaaS> |

```

Figure 6.5 — Configuration locale des services Matchia avec Docker Compose

4. Mise en place du pipeline CI/CD avec Jenkins

Une fois les étapes de construction et de conteneurisation validées, Jenkins a été configuré pour les enchaîner automatiquement. Le pipeline reprend donc un processus déjà testé, en garantissant qu'il est exécuté de manière identique à chaque nouvelle version du code.

4.1. Rôle de Jenkins

Jenkins est le composant central de la chaîne CI/CD. Exécuté dans un conteneur Docker, il récupère le code depuis GitHub et coordonne les outils nécessaires au pipeline : Maven, Node.js, Docker, SonarQube, Docker Hub et Azure CLI. Les accès aux services externes sont gérés à l'aide de credentials configurés dans Jenkins, afin de ne pas intégrer les informations d'authentification dans le dépôt source.

```

PS D:\PFE M2\Plateforme SaaS> docker compose -f docker-compose.jenkins.yml build
#1 [internal] load local bake definitions
#1 reading from stdin 530B 0.0s done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile.jenkins
#2 transferring dockerfile: 957B 0.0s done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/jenkins/jenkins:lts-jdk21
#3 DONE 2.2s

#4 [internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s

#5 [1/3] FROM docker.io/jenkins/jenkins:lts-jdk21@sha256:8547df3b0db2803d158ecc9499207a056bb30c23fddc18bb5b4a4dc14e77dd09
#5 resolve docker.io/jenkins/jenkins:lts-jdk21@sha256:8547df3b0db2803d158ecc9499207a056bb30c23fddc18bb5b4a4dc14e77dd09 0.0s done
#5 DONE 0.3s

#5 [1/3] FROM docker.io/jenkins/jenkins:lts-jdk21@sha256:8547df3b0db2803d158ecc9499207a056bb30c23fddc18bb5b4a4dc14e77dd09
#5 sha256:adea3ef8b4db3e85e81608a94ff829a46d51669a65671056c1752bd8b8da8232 0B / 391B 0.2s
#5 sha256:adea3ef8b4db3e85e81608a94ff829a46d51669a65671056c1752bd8b8da8232 391B / 391B 0.2s done
#5 sha256:18e2bde91fb067df55af69149ac5e3edaf075d403d079d1b471fad9ac8b4fe62 0B / 1.29kB 0.3s
#5 sha256:a5e3afd1e0b3cf2f1b5d1230490dec35079d8e941c321618ed2efef69ba9bfd 0B / 2.65kB 0.2s
#5 sha256:ae71a7561ea537a28198d728269b680eda28ec3bb9fa45ec2583730670554f66 0B / 100.98MB 0.2s
#5 sha256:18e2bde91fb067df55af69149ac5e3edaf075d403d079d1b471fad9ac8b4fe62 1.29kB / 1.29kB 0.4s done
#5 sha256:f19a623914320692b9eb745387631946e7f30fb87f577b2d2a5d25b4d02ea044 0B / 72.24MB 0.2s
#5 sha256:8957c4df8f9d89bcbce692b371fe0add0af6f61d75267a0589ec43bdaa94be79 0B / 6.45MB 0.2s
#5 sha256:a5e3afd1e0b3cf2f1b5d1230490dec35079d8e941c321618ed2efef69ba9bfd 2.65kB / 2.65kB 0.7s done
#5 sha256:844d30ade61d27e39771ded7f6e50a42d294ac1b6c467e6caf9920a3ae2f44b1 0B / 188B 0.2s
#5 sha256:844d30ade61d27e39771ded7f6e50a42d294ac1b6c467e6caf9920a3ae2f44b1 188B / 188B 0.4s done
#5 sha256:8957c4df8f9d89bcbce692b371fe0add0af6f61d75267a0589ec43bdaa94be79 1.05MB / 6.45MB 1.2s

```

Figure 6.6 — Interface du serveur Jenkins configuré pour le projet Matchia

4.2. Pipeline as Code avec le Jenkinsfile

La définition du pipeline est versionnée dans le Jenkinsfile placé à la racine du dépôt. Cette approche, appelée Pipeline as Code, garantit que les évolutions de la chaîne CI/CD sont suivies avec les évolutions applicatives. Le pipeline comporte les étapes Checkout, Build Backend, Build Frontend, Tests Backend, analyses SonarQube, Docker Build, publication des images puis déploiements du backend et du frontend sur Azure.

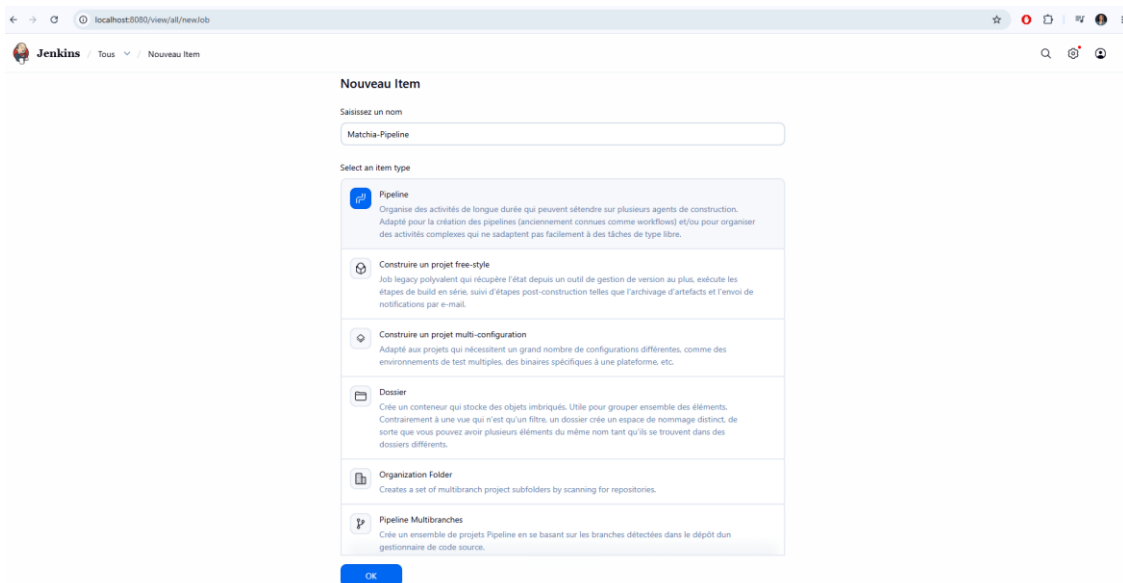


Figure 6.7 — Définition du pipeline CI/CD de Matchia dans Jenkins

4.3. Déclenchement automatique avec GitHub Webhook et Cloudflare Tunnel

Un Webhook GitHub notifie Jenkins lorsqu'un push est réalisé sur le dépôt. Le chemin `/github-webhook/` est configuré afin que Jenkins puisse recevoir cet événement et démarrer automatiquement le pipeline. Ce mécanisme évite le lancement manuel d'un build après chaque modification.

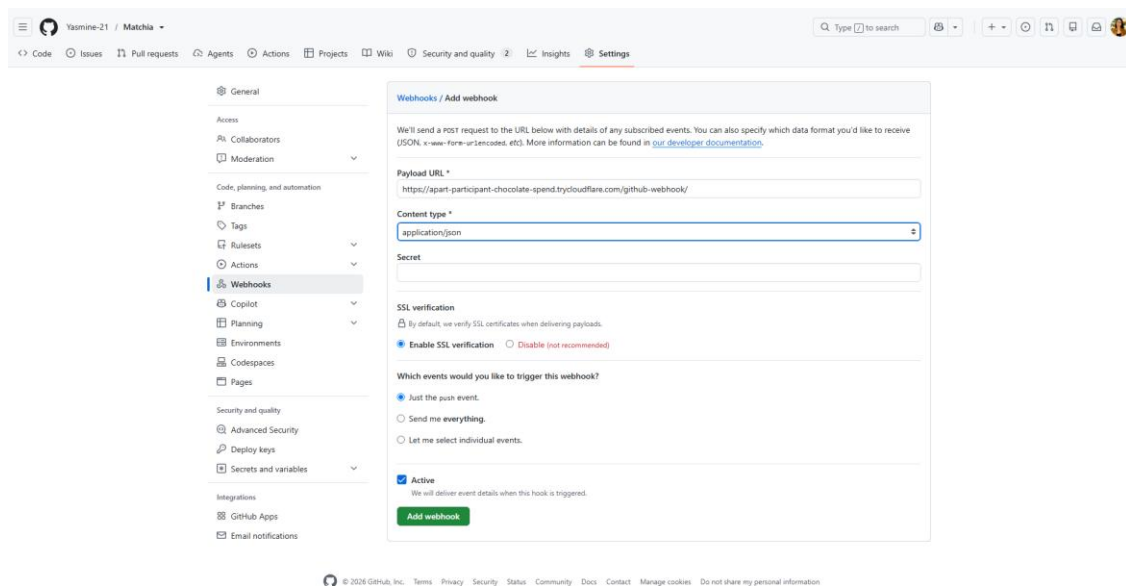


Figure 6.8 — Configuration du Webhook GitHub déclenchant Jenkins

Jenkins étant exécuté localement pendant la phase de mise en place, son adresse localhost n'était pas accessible depuis GitHub. Un Cloudflare Tunnel a donc été utilisé pour exposer temporairement le serveur Jenkins par une adresse HTTPS publique et rediriger les événements vers l'instance locale.


```

PS C:\WINDOWS\system32> cloudflared tunnel --url http://localhost:8080
1826-08-23T12:42:59Z INF Thank you for trying Cloudflare Tunnel. Doing so, without a Cloudflare account, is a quick way to experiment and try it out. However, be aware that these account-less Tunnels have no uptime guarantee, are subject to the Cloudflare Online Services Terms of Use (https://www.cloudflare.com/website-terms/), and Cloudflare reserves the right to investigate your use of Tunnels for violations of such terms. If you intend to use Tunnels in production you should use a pre-created named tunnel by following: https://developers.cloudflare.com/cloudflare-one/connections/connect-apps
1826-08-23T12:42:59Z INF Requesting new quick Tunnel on trycloudflare.com...
1826-08-23T12:42:59Z INF Your quick Tunnel has been created! Visit it at (it may take some time to be reachable): https://apart-participant-chocolate-spend.trycloudflare.com
1826-08-23T12:42:59Z INF Cannot determine default configuration path. No file [config.yml config.yaml] in [~/cloudflared ~/.cloudflared-warp ~/.cloudflared-warp]
1826-08-23T12:42:59Z INF Version 2026.8.2 (Checksum c29ee2b121f5b36a6d2eed69fd9787da7e7b8c510fa50aaa130337f931357b5)
1826-08-23T12:42:59Z INF GOOS: windows, GOVersion: go1.26.4, GoArch: amd64
1826-08-23T12:42:59Z INF Settings: map[ha-connections:1 protocol:quic url:http://localhost:8080]
1826-08-23T12:42:59Z INF cloudflared will not automatically update on Windows systems.
1826-08-23T12:42:59Z INF Generated Connector ID: 39e59983-7908-4228-ae63-b4d12a389a81
1826-08-23T12:42:59Z INF Initial protocol quic
1826-08-23T12:42:59Z INF ICMP proxy will use 192.168.1.14 as source for IPv4
1826-08-23T12:42:59Z INF ICMP proxy will use fe8b::9502:c97f:43:4f6d in zone MI-FI as source for IPv6
1826-08-23T12:43:00Z INF cloudflared does not support loading the system root certificate pool on Windows. Please use --origin-ca-pool <PATH> to specify the path to the certificate pool
1826-08-23T12:43:00Z INF ICMP proxy will use 192.168.1.14 as source for IPv4
1826-08-23T12:43:00Z INF Tunnel connection curve preferences: [X25519MLKEM768 CurveID(63874) CurveP256] connIndex=0 event=0 ip=198.41.280.43
1826-08-23T12:43:00Z INF ICMP proxy will use fe8b::9502:c97f:43:4f6d in zone MI-FI as source for IPv6
1826-08-23T12:43:00Z INF Starting metrics server on 127.0.0.1:20241/metrics
1826-08-23T12:43:00Z INF
+-----+
| COMPONENT | TARGET | STATUS | DETAILS |
+-----+
| DNS Resolution | region1.v2.argotunnel.com | PASS | DNS Resolved successfully |
| DNS Resolution | region2.v2.argotunnel.com | PASS | DNS Resolved successfully |
| UDP Connectivity | region1.v2.argotunnel.com | PASS | QUIC connection successful |
| UDP Connectivity | region2.v2.argotunnel.com | PASS | QUIC connection successful |
| TCP Connectivity | region1.v2.argotunnel.com | PASS | HTTP/2 connection successful |
| TCP Connectivity | region2.v2.argotunnel.com | PASS | HTTP/2 connection successful |
| Cloudflare API | api.cloudflare.com:443 | PASS | API is reachable |
+-----+
SUMMARY: Environment is healthy. cloudflared will use 'quic' as primary protocol.
1826-08-23T12:43:00Z INF precheck component="DNS Resolution" details="DNS Resolved successfully" run_id=f88af3b1-478a-41f2-bddf-a9c9f3c38c46 status=pass target=region1.v2.argotunnel.com
1826-08-23T12:43:00Z INF precheck component="DNS Resolution" details="DNS Resolved successfully" run_id=f88af3b1-478a-41f2-bddf-a9c9f3c38c46 status=pass target=region2.v2.argotunnel.com
1826-08-23T12:43:00Z INF precheck component="UDP Connectivity" details="QUIC connection successful" run_id=f88af3b1-478a-41f2-bddf-a9c9f3c38c46 status=pass target=region1.v2.argotunnel.com
1826-08-23T12:43:00Z INF precheck component="UDP Connectivity" details="QUIC connection successful" run_id=f88af3b1-478a-41f2-bddf-a9c9f3c38c46 status=pass target=region2.v2.argotunnel.com
1826-08-23T12:43:00Z INF precheck component="TCP Connectivity" details="HTTP/2 connection successful" run_id=f88af3b1-478a-41f2-bddf-a9c9f3c38c46 status=pass target=region1.v2.argotunnel.com
1826-08-23T12:43:00Z INF precheck component="TCP Connectivity" details="HTTP/2 connection successful" run_id=f88af3b1-478a-41f2-bddf-a9c9f3c38c46 status=pass target=region2.v2.argotunnel.com
1826-08-23T12:43:00Z INF precheck component="Cloudflare API" details="API is reachable" run_id=f88af3b1-478a-41f2-bddf-a9c9f3c38c46 status=pass target=api.cloudflare.com:443
1826-08-23T12:43:00Z INF precheck complete has_fail=false run_id=f88af3b1-478a-41f2-bddf-a9c9f3c38c46 suggested_protocol=quic
1826-08-23T12:43:00Z INF Registered tunnel connection connIndex=0 connection=d13adfa-baee-471e-8d88-7b52a31697ef event=0 ip=198.41.280.43 location=ars86 protocol=quic

```

Figure 6.9 — Exposition temporaire de Jenkins au moyen de Cloudflare Tunnel

5. Exécution automatisée du pipeline CI/CD

Le pipeline est déclenché à chaque push reçu depuis GitHub. Il applique ensuite, dans un ordre contrôlé, les étapes suivantes : récupération du code, construction du backend et du frontend, exécution des tests, analyse de qualité, création des images Docker, publication dans Docker Hub et déploiement des nouvelles versions sur Microsoft Azure. En cas d'échec d'une étape critique, les étapes suivantes ne sont pas exécutées.

5.1. Récupération du code et construction des applications

Après le checkout, Jenkins construit le backend avec Maven puis le frontend avec Node.js et npm. Le backend est préparé avec la commande Maven de construction définie dans le Jenkinsfile ; le frontend exécute npm ci puis npm run build. Lors du build de production, VITE_API_URL reçoit l'adresse publique du backend Azure afin que le frontend déployé communique avec l'API cible.

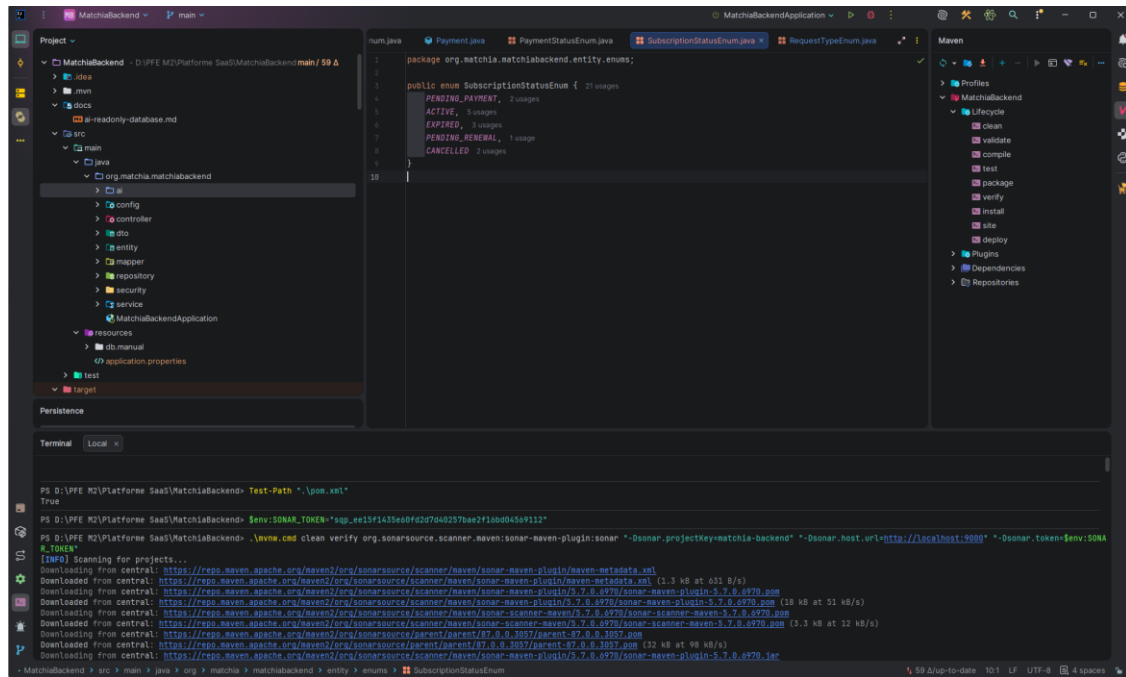


Figure 6.10 — Exécution de la phase de construction du backend dans le pipeline

5.2. Exécution automatique des tests backend

Les tests backend sont exécutés avant la création des images Docker. Ils constituent une barrière de qualité : si l'exécution échoue, Jenkins interrompt le pipeline et les opérations de publication et de déploiement ne sont pas réalisées. Les rapports de couverture générés avec JaCoCo sont également utilisables lors de l'analyse SonarQube.

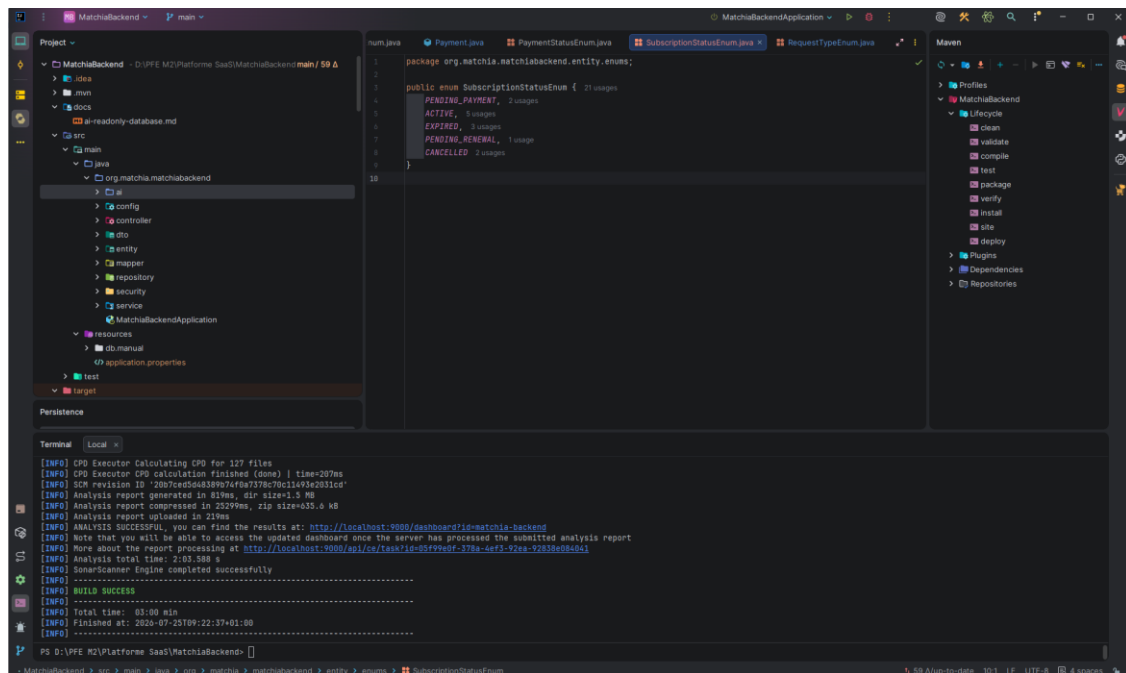


Figure 6.11 — Résultat de l'exécution automatique des tests backend

5.3. Analyse statique avec SonarQube

SonarQube complète les tests en réalisant une analyse statique distincte sur le backend et le frontend. Les indicateurs suivis portent notamment sur les bugs, les vulnérabilités, les duplications, les odeurs de code et la couverture disponible. Cette étape aide à détecter les défauts qui ne sont pas nécessairement révélés par les tests fonctionnels.

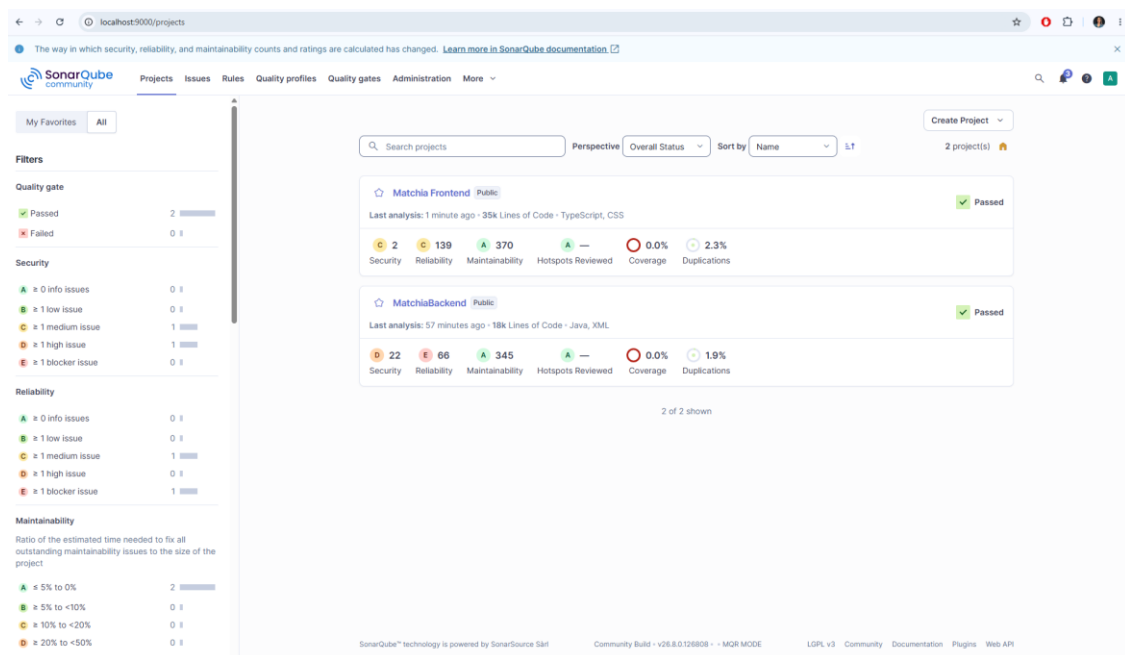


Figure 6.12 — Projets Matchia analysés dans SonarQube

5.4. Construction et publication des images Docker

Après la validation des builds, des tests et de l'analyse de qualité, Jenkins utilise les Dockerfiles du projet pour construire une image dédiée au backend et une autre au frontend. Les images sont étiquetées avec le numéro du build Jenkins et avec le tag latest, ce qui assure la traçabilité entre une exécution du pipeline et l'artefact produit.

Jenkins s'authentifie ensuite auprès de Docker Hub et publie les images du projet. Docker Hub joue ainsi le rôle de registre distant entre l'environnement de construction et les Azure Container Apps, qui peuvent récupérer la version à déployer.

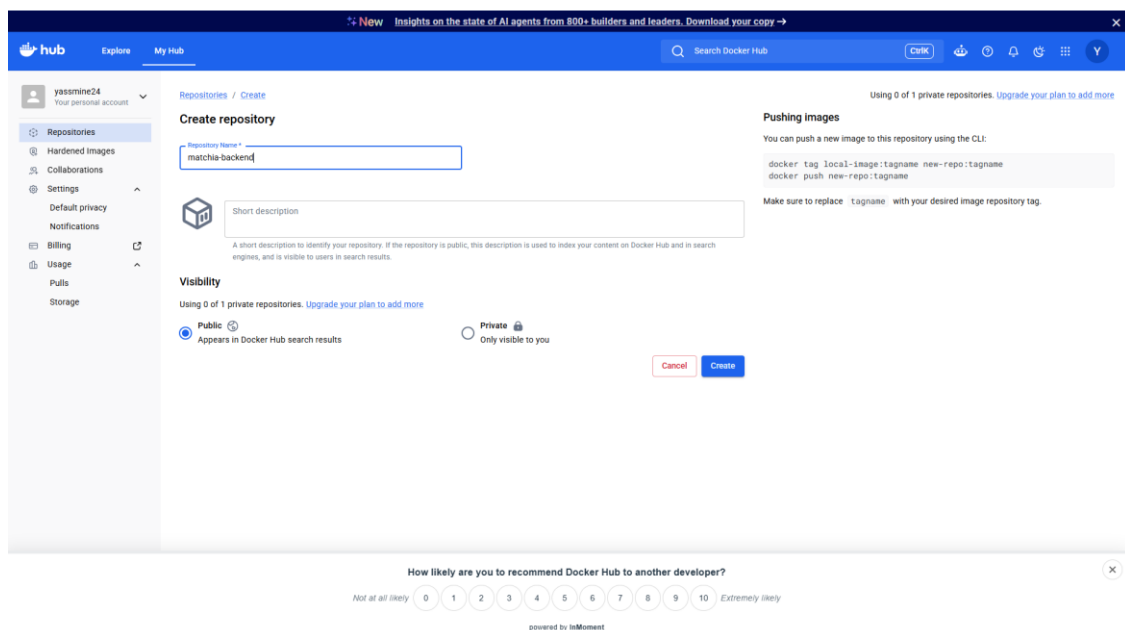


Figure 6.13 — Registre Docker Hub utilisé pour publier les images Matchia

6. Déploiement de Matchia sur Microsoft Azure

6.1. Préparation de l'environnement Cloud

Microsoft Azure a été retenu afin d'héberger les composants de la plateforme dans un environnement Cloud. L'architecture répartit les responsabilités entre Azure Container Apps pour les conteneurs applicatifs, Azure Database for PostgreSQL pour les données métier et Azure Files pour les fichiers persistants. Cette séparation préserve les données et les fichiers lors du remplacement d'une révision de conteneur.

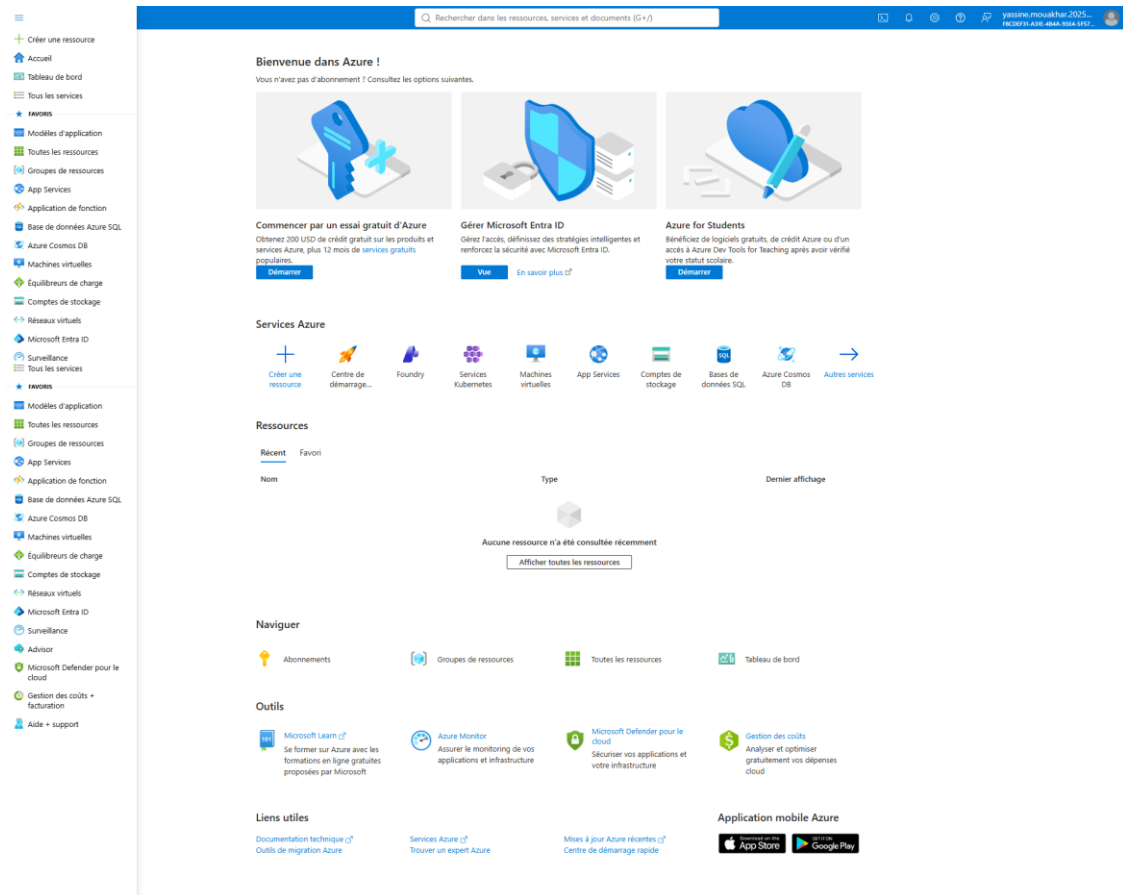


Figure 6.14 — Ressources Azure mises en place pour l'hébergement de Matchia

6.2. Base de données PostgreSQL managée

La base de données PostgreSQL est hébergée dans Azure Database for PostgreSQL Flexible Server. Le backend y accède au moyen d'une chaîne JDBC fournie par des variables d'environnement, sans inscrire les paramètres de connexion dans le code source. La migration initiale peut être réalisée avec `pg_dump` puis `pg_restore` afin de conserver la continuité des données entre l'environnement local et l'environnement Cloud.

Microsoft Azure

Accueil > Gestionnaire des ressources > Groupes de ressources > rg-matchia > Marketplace > Azure Database for PostgreSQL Flexible Server

Nouveau serveur flexible Azure Database pour PostgreSQL

Microsoft

⚠ Les noms de serveur, la méthode de connectivité réseau et la redondance de sauvegarde ne peuvent pas être modifiés une fois le serveur créé. Vérifiez soigneusement ces options avant l'allocation.

⚠ Le changement des options de base peut réinitialiser vos sélections. Passez en revue toutes les options avant de créer la ressource.

De base Mise en réseau Sécurité Balises Vérifier > créer

Créez une base de données Azure pour le serveur flexible PostgreSQL. [En savoir plus >](#)

Saviez-vous que les nouveaux utilisateurs d'Azure peuvent utiliser PostgreSQL – Serveur flexible gratuitement pendant 750 heures en utilisant un compte gratuit Azure ? [En savoir plus >](#)

Détails du projet

Sélectionnez l'abonnement pour gérer les coûts et les ressources déployées. Utilisez les groupes de ressources comme les dossiers pour organiser et gérer toutes vos ressources.

Abonnement * Azure for Students

Groupe de ressources * rg-matchia [Créer nouveau](#)

Détails du serveur

Entrez les paramètres obligatoires de ce serveur, notamment en choisissant un emplacement et en configurant les ressources de calcul et de stockage.

Nom du serveur * matchia-postgresql

Région * France Central

Version de PostgreSQL 18

Type de charge de travail ☒ Dev/Test ☐ Production

ⓘ Cette instance est destinée à un usage de développement en dehors d'un environnement de production.

Coûts estimés

Calcul USD 13.87/mois
Libérez jusqu'à 750 heures.
Standard_B1ms (1 cœur virtuel) 13.87

Stockage USD 4.26/mois
Gratuit jusqu'à 32 Go
32 Go (USD 0.13 par Go) 32 x 0.13

Bande passante
Le transfert de données sortantes entre des services situés dans des régions différentes entraîne des frais supplémentaires. Tous les transferts de données entrants sont gratuits. [En savoir plus >](#)

Estimation du total USD 18.13/mois

Des frais s'appliquent si vous utilisez au-dessus des limites mensuelles gratuites. Vérifiez votre utilisation > de services gratuits. Les frais finaux apparaîtront dans votre devise locale.

Vérifier > créer

Suivant : Mise en réseau >

Figure 6.15 — Configuration de la base PostgreSQL Azure de Matchia

6.3. Stockage persistant avec Azure Files

Les conteneurs applicatifs étant remplaçables, les fichiers déposés dans leur système de fichiers interne ne doivent pas constituer le stockage permanent. Azure Files fournit donc un partage monté dans le backend, notamment pour conserver les documents justificatifs, les images de produits, les logos et les autres contenus téléversés. Le montage du partage garantit que ces fichiers demeurent disponibles après la création d'une nouvelle révision.

Microsoft Azure

Accueil > Gestionnaire des ressources > Groupes de ressources > rg-matchia > Marketplace > Azure Database for PostgreSQL Flexible Server > Nouveau serveur flexible Azure Database pour PostgreSQL

Calcul + stockage

Cluster

Un cluster est un ensemble de plusieurs nœuds (machines) qui permet une mise à l'échelle flexible et une capacité de données élevée. [Comparer les options de cluster](#)

Options de cluster

- ☒ Serveur
- ☐ Cluster élastique

Pour une capacité supérieure à un seul nœud, le cluster élastique permet un partage basé sur les schémas et les lignes d'une base de données distribuées sur un cluster.

Compute

Les ressources de calcul sont pré-allouées et facturées à l'heure, en fonction des vCores configurés.

Notez que la haute disponibilité est prise en charge uniquement pour les niveaux Usage général et Mémoire optimisée.

Compute tier

- ☒ Burstable (1 à 20 vCores) : est destiné à un usage de développement en dehors d'un environnement de production
- ☐ Usage général (2-96 vCores) – Configuration équilibrée pour les charges de travail les plus courantes
- ☐ Mémoire optimisée (2-96 vCores) – Idéal pour les charges de travail qui nécessitent un rapport mémoire/CPU élevé

Taille de calcul

Standard_B1ms (1 vCore, 2 mémoire Gio, 640nombre maximal d'IOPS)

Stockage

Le stockage que vous provisionnez est la quantité de capacité de stockage disponible pour votre serveur, et est facturé par Gio/mois.

Notez que le stockage ne peut pas faire l'objet d'un scale-down une fois le serveur créé.

Type de stockage

SSD Premium

Les serveurs avec Expansible activés ont des options de type de stockage limitées

Taille du stockage

32 Gio

Niveau de performance

P4 (120 IOPS)

Coûts estimés

Calcul USD 13.87/mois

Libérez jusqu'à 750 heures.

Standard_B1ms (1 cœur virtuel) 13.87

Stockage USD 4.26/mois

Gratuit jusqu'à 32 Gio

32 Gio (USD 0.13 par Gio) 32 x 0.13

Bande passante

Le transfert de données sortantes entre des services situés dans des régions différentes entraîne des frais supplémentaires. Tous les transferts de données entrants sont gratuits. [En savoir plus](#)

Estimation du total USD 18.13/mois

Des frais s'appliquent si vous utilisez au-dessus des limites mensuelles gratuites. Vérifiez votre utilisation de services gratuits. Les frais finaux apparaîtront dans votre devise locale.

[Enregistrer](#)

Figure 6.16 — Ressource Azure de stockage persistant associée à Matchia

6.4. Hébergement avec Azure Container Apps

Deux Azure Container Apps indépendantes exécutent l'application : matchia-frontend sert les fichiers React compilés à travers Nginx, tandis que matchia-backend expose les API REST Spring Boot sur le port 8081. L'Ingress Azure rend les services accessibles en HTTPS et les deux composants peuvent être mis à jour indépendamment.

Microsoft Azure

Accueil > rg-matcha > Marketplace > Application de conteneur

Créer Application de conteneur

Informations de base Conteneur Entrée Étiquettes Vérifier > créer

Sélectionnez une image de démarrage rapide pour votre conteneur ou désélectionnez une image de démarrage rapide pour utiliser un conteneur existant.

Utiliser l'image de démarrage rapide ☐

Détails du conteneur

Nom * matcha-frontend

Source de l'image

☐ Azure Container Registry

☒ Docker Hub ou d'autres registres

Type d'image

☒ Public

☐ Privé

Serveur de connexion au registre * docker.io

Image et balise * yassminaz2/matcha-frontend:latest

Remplacement de commande Example: /bin/bash

Remplacement des arguments Example: -c while true; do echo hello; sleep 10; done

Fonctionnalités spécifiques à la pile de développement

Lorsque vous sélectionnez une pile de développement spécifique, vous obtenez des fonctionnalités supplémentaires adaptées à cette pile, en optimisant Container Apps pour qu'elles s'exécutent pour vos paramètres uniques.

Pile de développement Non spécifiée

Allocation des ressources de conteneur

Choisissez le profil de charge de travail pour cette application. Vous pouvez ajuster l'allocation de processeur et de mémoire pour cette application jusqu'à la limite du profil de charge de travail. [En savoir plus](#)

Profil de charge de travail * Consommation : jusqu'à 4 processeurs virtuels, 8 Go de mémoire

GPU ☐

Processeur et mémoire * 0.5 cœurs de processeur, mémoire 1 Go

Variables d'environnement

Nom	Valeur	Supprimer
Enter a name	Enter a value	

Vérifier > créer < précédent Suivant > Entrée

Figure 6.17 — Configuration de l'Azure Container App du frontend

Microsoft Azure

Accueil > rg-matcha > Marketplace > Application de conteneur

Créer Application de conteneur

Informations de base Conteneur Entrée Étiquettes Vérifier > créer

Sélectionnez une image de démarrage rapide pour votre conteneur ou désélectionnez une image de démarrage rapide pour utiliser un conteneur existant.

Utiliser l'image de démarrage rapide ☐

Détails du conteneur

Nom * matcha-backend

Source de l'image

☐ Azure Container Registry

☒ Docker Hub ou d'autres registres

Type d'image

☒ Public

☐ Privé

Serveur de connexion au registre * docker.io

Image et balise * yassminaz2/matcha-backend:latest

Remplacement de commande Example: /bin/bash

Remplacement des arguments Example: -c while true; do echo hello; sleep 10; done

Fonctionnalités spécifiques à la pile de développement

Lorsque vous sélectionnez une pile de développement spécifique, vous obtenez des fonctionnalités supplémentaires adaptées à cette pile, en optimisant Container Apps pour qu'elles s'exécutent pour vos paramètres uniques.

Pile de développement Non spécifiée

Allocation des ressources de conteneur

Choisissez le profil de charge de travail pour cette application. Vous pouvez ajuster l'allocation de processeur et de mémoire pour cette application jusqu'à la limite du profil de charge de travail. [En savoir plus](#)

Profil de charge de travail * Consommation : jusqu'à 4 processeurs virtuels, 8 Go de mémoire

GPU ☐

Processeur et mémoire * 0.5 cœurs de processeur, mémoire 1 Go

Variables d'environnement

Nom	Valeur	Supprimer
SPRING_DATASOURCE_URL	jdbc:postgresql://matcha-post...	
SPRING_DATASOURCE_USERNAME	matchaadmin	
SPRING_DATASOURCE_PASSWORD	Ssd@2020	
Enter a name	Enter a value	

Vérifier > créer < précédent Suivant > Entrée

Figure 6.18 — Configuration de l'Azure Container App du backend

6.5. Déploiement automatisé avec Azure CLI

Après la publication réussie des images dans Docker Hub, Azure CLI assure la dernière étape du pipeline. Jenkins s'authentifie à Azure, sélectionne l'environnement configuré puis exécute une commande `az containerapp update` pour mettre à jour successivement la Container App du

backend et celle du frontend. Une nouvelle révision est créée avec le numéro du build ; cette révision fournit une version identifiable et permet de suivre le résultat du déploiement.

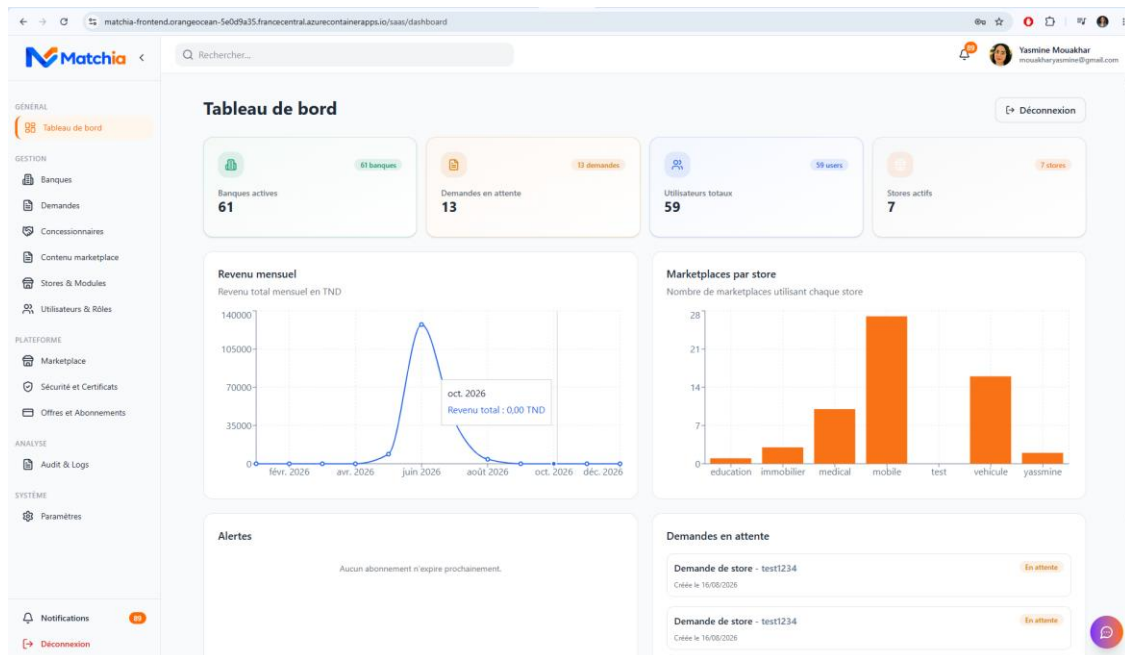


Figure 6.19 — Création d'une nouvelle révision Azure Container Apps après déploiement

7. Architecture complète et validation du processus

L'architecture finale relie le poste du développeur, GitHub, le Webhook, Jenkins, SonarQube, Docker Hub et les ressources Azure. À chaque push, Jenkins récupère le code, construit les deux composants, exécute les tests et les analyses, publie les images puis demande à Azure Container Apps d'utiliser les nouvelles versions. Le backend déployé communique avec PostgreSQL Azure et avec le stockage persistant Azure Files.

Architecture de déploiement de Matchia sur Microsoft Azure

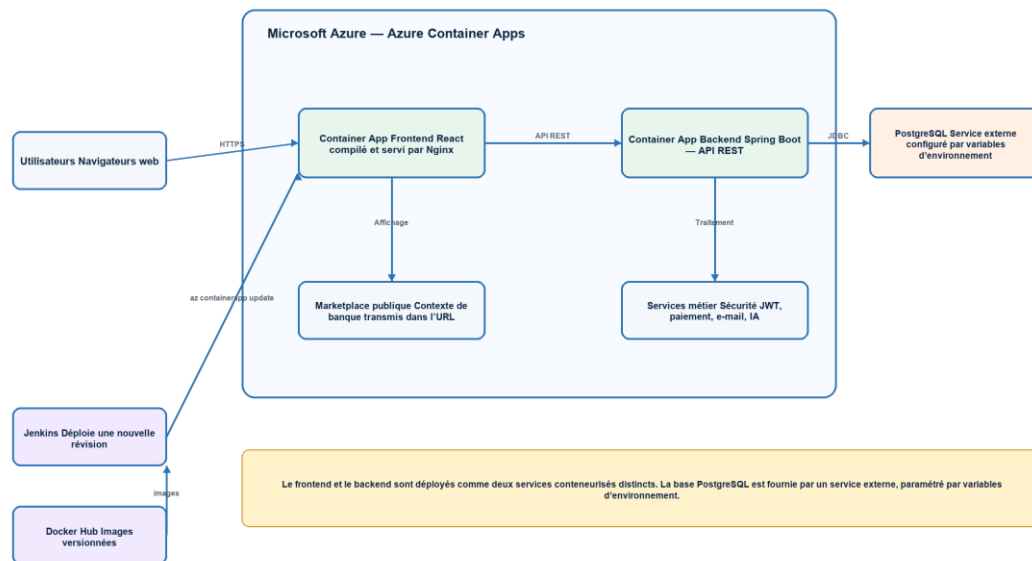


Figure 6.20 — Architecture globale de la chaîne CI/CD et du déploiement Cloud de Matchia

7.1. Validation du processus complet

La validation de la chaîne consiste à effectuer une modification contrôlée du code, à l'enregistrer puis à exécuter un git push sur la branche suivie. Le Webhook déclenche Jenkins, qui exécute successivement les étapes de construction, de test, d'analyse, de création d'images, de publication et de déploiement. La disponibilité d'une nouvelle révision des Container Apps après une exécution réussie confirme le fonctionnement de bout en bout du processus automatisé.

8. Conclusion

Ce chapitre a présenté la mise en place du pipeline CI/CD de Matchia, depuis la validation manuelle initiale de la conteneurisation jusqu'au déploiement automatisé sur Microsoft Azure. Jenkins orchestre désormais une chaîne complète, contrôlée et reproductible : récupération du code, construction, tests, analyse de qualité, publication des images Docker et mise à jour des services Cloud.